

Full Stack Developer Prep

Priyanka R

Senior Full Stack Developer · 4+ years

Stack covered: Java 17 · Spring Boot · Angular 16 · TypeScript · JavaScript · HTML/CSS · MongoDB · MySQL · Microservices · AWS · plus coding problems, project deep-dives & HR. **Includes the Berriobot assessment guide.**

Table of Contents

Overview & Study Plan

00 Self-Introduction

01 JavaScript

02 HTML & CSS

03 TypeScript

04 Angular

05 Core Java

06 Spring & Spring Boot

07 AWS (Basics)

08 Microservices (Basics)

09 Coding Problems

10 Projects Deep-Dive

11 HR & Behavioral

12 Java 17 Features

13 MongoDB

14 BerriBot Assessment Guide

Priyanka R — Interview Preparation Pack

A complete, resume-tailored interview study kit for a **Full Stack Developer (Angular + Java/Spring Boot, 4+ years)** — covering technical + managerial + HR rounds.

Prepared from your resume: Angular, Ionic, RxJS, NgRx, Angular Formly, TypeScript, JavaScript, HTML/CSS, Java, Spring Boot, Microservices, AWS (basics), MySQL/Postgres/Neo4j — and your projects **RoboGebra**, **Subsea**, **Backoffice Migration**, **EasyVisa**.

🎯 **Targeting the Berribot assessment?** Start with [14 — Berribot Assessment Guide](#) (Senior Full Stack: Java 17, Spring Boot, Angular 16, MongoDB, MySQL, Microservices, AWS S3). It maps the whole stack to these files, flags your gaps (MongoDB & Java 17), and gives a focused 5-day plan.



Contents

#	File	What's inside
00	Self Introduction	Ready-to-memorize "Tell me about yourself" (30s / 60s / 2-min) + cheat sheet
01	JavaScript	Closures, async/event loop, ES6+, debounce/throttle, output-based Qs
02	HTML & CSS	Semantics, box model, Flexbox/Grid, responsive, specificity
03	TypeScript	Types vs interfaces, generics, utility types, decorators
04	Angular	Your core skill — lifecycle, RxJS, NgRx, forms, DI, routing, change detection
05	Core Java	OOP, collections, HashMap internals, Java 8 streams, multithreading
06	Spring & Spring Boot	IoC/DI, REST, JPA, auto-config, exception handling, security basics
07	AWS (Basics)	EC2, S3, Lambda, RDS, CloudWatch — fundamentals
08	Microservices (Basics)	Monolith vs micro, Spring Cloud, patterns, communication
09	Coding Problems	35+ problems with JS & Java solutions + complexity
10	Projects Deep-Dive	Tough "project difficulty" Q&A on your 4 real projects
11	HR & Behavioral	Why join, strengths/weaknesses, STAR answers, salary, notice
12	Java 17 Features	 Records, sealed classes, pattern matching, switch expressions, text blocks
13	MongoDB	 Document model, CRUD, aggregation, indexing, Spring Data MongoDB (<i>your gap</i>)
14	Berribot Assessment Guide	🎯 Stack map, gap analysis, full-stack S3 scenario, 5-day plan



Suggested 7-day study plan

Day	Focus
1	Self-intro (00) + Projects deep-dive (10) — these are asked in every round
2	JavaScript (01) + TypeScript (03)
3	Angular (04) — spend the most time here
4	Core Java (05) + Spring Boot (06)
5	Coding problems (09) — practice writing on paper / whiteboard
6	AWS (07) + Microservices (08) + HTML/CSS (02)
7	HR/behavioral (11) + revise weak areas + mock interview



Interview-day checklist

- [] Can deliver the 60-second intro smoothly without notes
- [] Can draw the architecture of **RoboGebra** and **EasyVisa** on a whiteboard
- [] Have **one strong "challenge I solved"** story ready per project (STAR format)
- [] Can explain RxJS `switchMap` vs `mergeMap` and the `takeUntil` unsubscribe pattern
- [] Can explain Spring Boot auto-configuration and `@SpringBootApplication`
- [] Can code FizzBuzz, palindrome, two-sum, and reverse-a-string from memory
- [] Have 2–3 thoughtful questions ready to ask the interviewer
- [] Be honest about **AWS & Microservices = basic/working knowledge** — don't oversell



How to use this pack

1. **Read actively** — say answers out loud, don't just skim.
2. **For project questions, replace any generic phrasing with your real details** — interviewers can tell the difference.
3. **Practice the coding problems by typing them yourself**, then check against the solution.
4. **Time your self-introduction** — aim for under 90 seconds.



The technical answers are accurate study material. The project answers are written to match your resume — review them and adjust any specifics (team size, exact metrics) to your real experience before the interview.

Good luck, Priyanka! 🚀

Self-Introduction — "Tell Me About Yourself"

This is the very first thing most interviewers ask. Nail it, and you set a confident tone for the rest of the interview. Below are three versions (60s, 30s, 2-min), a cheat sheet of anchor facts, and delivery tips. Practice out loud until it feels natural — not memorized word-for-word, but smooth.

60-Second Version (Primary — memorize this one)

"Hi, I'm Priyanka, a Full Stack Developer with over four years of experience building web applications, primarily on the Angular front end and Java with Spring Boot on the back end.

I currently work as a Software Engineer at Provility Software Solutions in Chennai, where I've delivered across several products. My most recent and favourite project is RoboGebra — an AI-driven math learning platform built with Angular, Ionic, Java and MySQL. There I built a personalized learning dashboard, a study reminder system backed by REST APIs, a quiz module, and I contributed to an AI math explanation engine that gives bilingual explanations in Tamil and English, along with real-time math computation and visualization.

Before that, I worked on Subsea — a marine operations management product using Angular and .NET on Azure — and on EasyVisa, a visa management platform for attorneys, where I built a document portal and reusable Angular Formly components with nested reactive forms. I've also led an Angular version migration, upgrading four repositories to Angular 16.

On the technical side I'm strong in JavaScript, TypeScript, RxJS, NgRx and Redux on the front end, and Spring Boot, microservices and SQL databases like MySQL and Postgres on the back end. I enjoy owning a feature end to end — from API to UI.

I'm now looking for a role with more scale and variety, which is exactly why I'm excited about Wipro."

30-Second Version (Short / when time is tight)

"I'm Priyanka, a Full Stack Developer with over four years of experience, currently a Software Engineer at Provility Software Solutions in Chennai. I specialize in Angular on the front end and Java with Spring Boot on the back end. My latest project is RoboGebra, an AI-driven math learning platform where I built the learning dashboard, a quiz module, and a reminder system using REST APIs. I've also worked across .NET, microservices and cloud on AWS and Azure. I'm looking for a larger platform to grow, which is what draws me to Wipro."

2-Minute Detailed Version (when asked to "walk me through your background")

"Sure. I'm Priyanka, a Full Stack Developer with just over four years of professional experience. I completed my B.Tech in Information Technology from Anna University, IRTT Erode, in 2020, and I've been with Provility Software Solutions in Chennai as a Software Engineer since October 2021.

Over these four years I've worked across the full stack — Angular, Ionic, React and TypeScript on the front end, and Java, Spring Boot, Grails/Groovy and ASP.NET Core on the back end, with MySQL, Postgres and Neo4j as databases. I've also worked with AWS services like EC2, S3, Lambda and RDS, and with Azure.

To give you a sense of my work through a few projects:

I started on EasyVisa, a visa management platform for attorneys, built with Angular, Grails, Groovy, Postgres and Neo4j. There I built a document portal module and a lot of reusable UI — custom Angular Formly components, nested reactive forms, an image cropper — and I wrote unit tests for the components I owned. That's where I really matured as a front-end engineer.

I then led a focused migration project, upgrading four repositories to Angular 16, which sharpened my understanding of the framework internals and dependency management.

After that I moved to Subsea, a marine equipment and operations management product on Angular and .NET, running with Docker on Azure. I built the Schedule-Manager module with .NET APIs and a Kendo UI grid, including category and subcategory management, plus a number of bug fixes.

My current project, since late 2024, is RoboGebra — an AI-driven math learning platform on Angular, Ionic, Java and MySQL. I built a personalized learning dashboard, a study reminder system with REST APIs, a study list module and a quiz module, and I contributed to the AI math explanation engine that produces bilingual Tamil and English explanations, plus real-time math computation and visualization. It's been the most rewarding project because it combines clean front-end work with meaningful AI-driven features.

Outside the code, I'm comfortable in agile teams using Jira, Git, Bitbucket and GitHub. What I enjoy most is owning a feature end to end and seeing it actually help users. I'm now looking for a role with broader scale and exposure to enterprise-level work, which is exactly why I'm interested in Wipro."

Cheat Sheet — Anchor Facts

- **Role / experience:** Full Stack Developer, 4+ years.
- **Current job:** Software Engineer, Provility Software Solutions, Chennai (Oct 2021 – Present).
- **Core stack:** Angular (+ Ionic, NgRx, Formly), Java + Spring Boot, TypeScript/JavaScript, RxJS, Redux.
- **Also:** React/Formik, ASP.NET Core, Grails/Groovy, Microservices, Spring Reactive.
- **Databases:** MySQL, Postgres, Neo4j.
- **Cloud:** AWS (EC2, S3, Lambda, RDS, CloudWatch), Azure.
- **Tools:** Jira, GitHub, Bitbucket, IntelliJ IDEA, Visual Studio.
- **Flagship project:** RoboGebra — AI math learning platform (Angular, Ionic, Java, MySQL); dashboard, reminders/REST APIs, quiz module, bilingual AI explanation engine, real-time math visualization.
- **Other projects:** Subsea (Angular/.NET/Azure/Docker, Schedule-Manager + Kendo UI), Angular 16 migration (4 repos), EasyVisa (Angular/Grails/Groovy/Postgres/Neo4j, document portal + Formly components).
- **Education:** B.Tech IT, Anna University / IRTT Erode, 2020, 77.1%.
- **Languages:** English (professional), Tamil (native).

- **The closing line:** "Looking for more scale and variety — which is why I'm excited about Wipro."
-

Tone & Delivery Tips

- **Lead with the headline.** First sentence = role + years + core stack. The interviewer decides in 10 seconds whether you sound senior; give them the answer immediately.
- **Present → past → future.** Current role first, then a quick arc of past projects, then close with why you're here (Wipro). This structure feels intentional, not rambling.
- **Pick ONE flagship.** Spend the most words on RoboGebra. Mention others briefly. Don't list every project equally — it flattens your story.
- **Talk in outcomes, not just tech.** "I built a quiz module and a bilingual explanation engine" beats "I used Angular and Java."
- **Don't recite your resume line by line.** They have it. Give the narrative version.
- **End with intent.** Always close on why you're interviewing — it bridges naturally into "Why Wipro?"
- **Pace and pause.** Speak slightly slower than feels natural. Pause after your headline sentence so it lands.
- **Confidence over modesty.** Use "I built / I led / I delivered," not "I helped with" or "I was involved in."
- **Keep it tight.** If you're past 75 seconds on the 60s version, you're over-explaining. Practice with a timer.
- **Smile and breathe.** This is the one answer you can fully prepare — let that calm carry you into the harder questions.

JavaScript — Interview Q&A

Lateral hire (4+ years) Full Stack Developer prep. Concise but complete answers with code examples.

Fundamentals

Q: What is the difference between `var`, `let`, and `const`?

- `var`: function-scoped, hoisted and initialized to `undefined`, can be re-declared and re-assigned. Attaches to `window` when declared globally.
- `let`: block-scoped, hoisted but in the **Temporal Dead Zone (TDZ)** until declared, can be re-assigned but not re-declared in the same scope.
- `const`: block-scoped, must be initialized at declaration, cannot be re-assigned. The binding is constant, **not the value** — object/array contents can still mutate.

```
function demo() {  
  if (true) {  
    var a = 1; // function scoped  
    let b = 2; // block scoped  
    const c = 3; // block scoped  
  }  
  console.log(a); // 1  
  // console.log(b); // ReferenceError  
}  
  
const obj = { n: 1 };  
obj.n = 2; // OK — mutating, not re-assigning  
// obj = {}; // TypeError
```

Q: What is hoisting?

Hoisting is JavaScript moving **declarations** to the top of their scope during the compile phase. `var` declarations are hoisted and initialized to `undefined`. Function declarations are hoisted entirely (callable before definition). `let` / `const` are hoisted but stay in the TDZ — accessing them before declaration throws. Function **expressions** and arrow functions are not hoisted as callable.

```
console.log(x); // undefined (var hoisted)  
var x = 5;  
  
foo(); // "hi" (function declaration hoisted)  
function foo() { console.log("hi"); }  
  
// bar(); // ReferenceError (TDZ)  
let bar = () => {};
```


Q: Explain scope in JavaScript.

Scope determines variable accessibility. Types:

- **Global scope:** accessible everywhere.
- **Function scope:** variables declared inside a function (`var`).
- **Block scope:** `{ }` blocks for `let` / `const` .
- **Lexical scope:** inner functions access variables of their outer (enclosing) functions, determined by where code is written, not where it's called. This is the basis of closures.

```
let g = "global";
function outer() {
  let o = "outer";
  function inner() {
    console.log(g, o); // can read both via lexical scope
  }
  inner();
}
```

Q: What are the data types in JavaScript?

Primitives (immutable, copied by value): `string` , `number` , `bigint` , `boolean` , `undefined` , `null` , `symbol` .

Non-primitive (copied by reference): `object` (includes arrays, functions, dates).

```
typeof "a";           // "string"
typeof 1;              // "number"
typeof 10n;            // "bigint"
typeof true;           // "boolean"
typeof undefined;      // "undefined"
typeof null;           // "object" (historical bug)
typeof Symbol();        // "symbol"
typeof {};              // "object"
typeof function(){};    // "function"
```

Q: Difference between `==` and `===` ?

- `==` (loose equality) compares **after type coercion**.
- `===` (strict equality) compares value **and** type, no coercion. Prefer `===` .

```
0 == "0";           // true (coerced)
0 === "0";          // false (different types)
null == undefined;  // true
null === undefined; // false
NaN === NaN;        // false (use Number.isNaN)
```

Q: What is type coercion?

Automatic conversion of values from one type to another. **Implicit** happens during operations (`+` , `==` , template strings); **explicit** is manual (`Number()` , `String()`). The `+` operator favors string concatenation if either operand is a string; other arithmetic operators coerce to numbers.

```
"5" + 1;    // "51" (number -> string)
"5" - 1;    // 4    (string -> number)
true + 1;   // 2
[] + {};    // "[object Object]"
Number(""); // 0
```

Q: What are truthy and falsy values?

Falsy values (coerce to `false`): `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy — including `"0"`, `"false"`, `[]`, `{}`, and functions.

```
if ([]) console.log("empty array is truthy"); // logs
if (!"") console.log("empty string is falsy"); // logs
Boolean("0"); // true
```

Functions & Closures

Q: What is a closure? Give an example.

A closure is a function that **retains access to its lexical scope** even after the outer function has returned. It "remembers" the variables of the environment in which it was created. Used for data privacy, factories, and stateful functions.

```
function counter() {
  let count = 0;           // private state
  return function () {
    count++;
    return count;
  };
}
const inc = counter();
inc(); // 1
inc(); // 2 -- count persists in the closure
```

Q: What is an IIFE?

Immediately Invoked Function Expression — a function defined and executed at once. Historically used to create a private scope and avoid polluting the global namespace (before block scope / modules existed).

```
(function () {
  const secret = "hidden";
  console.log("runs immediately");
})();

// arrow version
(() => console.log("iife"))();
```

Q: What are higher-order functions?

Functions that take other functions as arguments and/or return functions. `map`, `filter`, `reduce`, `forEach` are built-in examples.

```
const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2); // [2,4,6,8]
const evens = nums.filter(n => n % 2 === 0); // [2,4]
const sum = nums.reduce((acc, n) => acc + n, 0); // 10

function withLogging(fn) { // returns a function
  return (...args) => {
    console.log("calling with", args);
    return fn(...args);
  };
}
```

Q: What is currying?

Transforming a function with multiple arguments into a sequence of functions each taking a single argument. Enables partial application and reusable specialized functions.

```
const add = a => b => c => a + b + c;
add(1)(2)(3); // 6

const multiply = a => b => a * b;
const double = multiply(2);
double(5); // 10
```

Q: Explain `call`, `apply`, and `bind`.

All set the `this` context of a function.

- `call(thisArg, arg1, arg2)` : invokes immediately, args listed individually.
- `apply(thisArg, [args])` : invokes immediately, args as an array.
- `bind(thisArg, ...args)` : returns a **new function** with `this` bound (does not invoke).

```
const person = { name: "Priya" };
function greet(greeting, punct) {
  return `${greeting}, ${this.name}${punct}`;
}
greet.call(person, "Hi", "!"); // "Hi, Priya!"
greet.apply(person, ["Hello", "."]); // "Hello, Priya."
const bound = greet.bind(person, "Hey");
bound("?"); // "Hey, Priya?"
```

Q: Difference between arrow functions and normal functions?

- **this binding**: arrow functions have no own `this` — they inherit it lexically from the enclosing scope. Normal functions get `this` based on how they're called.
- Arrows have no **arguments object** (use rest params), cannot be used as **constructors** (no `new`), and have no **prototype**.
- Arrows can't be used as object methods that need dynamic `this`.

```
const obj = {
  val: 42,
  normal: function () { return this.val; }, // this = obj
  arrow: () => this.val,                  // this = outer (undefined)
};
obj.normal(); // 42
obj.arrow();  // undefined
```

Q: How is `this` determined in JavaScript?

Depends on the call site:

1. **Default:** global object (or `undefined` in strict mode).
2. **Implicit:** the object before the dot (`obj.method()` → `obj`).
3. **Explicit:** `call` / `apply` / `bind` .
4. **new** : the newly created instance.
5. **Arrow function:** inherited lexically (none of the above apply).

```
function show() { return this; }
const o = { show };
o.show();           // o (implicit)
show.call({a:1});   // {a:1} (explicit)
```

Objects & Prototypes

Q: What is prototypal inheritance?

Objects inherit properties/methods from other objects via the **prototype** link. When a property isn't found on an object, the engine walks up the prototype chain until found or `null` is reached.

```
const animal = {
  eat() { return "eating"; },
};
const dog = Object.create(animal);
dog.bark = () => "woof";
dog.eat(); // "eating" (inherited)
dog.bark(); // "woof"
```

Q: What is the prototype chain?

The series of links between objects via `[[Prototype]]` (accessible as `__proto__` / `Object.getPrototypeOf`). Lookups traverse this chain. It ends at `Object.prototype` , whose prototype is `null` .

```
const arr = [1, 2];
// arr -> Array.prototype -> Object.prototype -> null
arr.hasOwnProperty("length"); // method found on Object.prototype
Object.getPrototypeOf(arr) === Array.prototype; // true
```

Q: What does `Object.create` do?

Creates a new object with the specified prototype object. Useful for clean prototypal inheritance without constructor functions.

```
const proto = { greet() { return "hi"; } };
const obj = Object.create(proto);
obj.greet(); // "hi"
Object.getPrototypeOf(obj) === proto; // true
Object.create(null); // object with no prototype
```

Q: Difference between shallow copy and deep copy?

- **Shallow copy:** copies top-level properties; nested objects are still shared by reference. (`{...obj}`, `Object.assign`, `Array.slice`).
- **Deep copy:** recursively copies all nested levels — fully independent. (`structuredClone`, recursion, or `JSON.parse(JSON.stringify())` with limitations).

```
const original = { a: 1, nested: { b: 2 } };

const shallow = { ...original };
shallow.nested.b = 99;
original.nested.b; // 99 -- shared reference!

const deep = structuredClone(original);
deep.nested.b = 5;
original.nested.b; // unchanged
```

Asynchronous JavaScript

Q: What is a callback and what is callback hell?

A **callback** is a function passed to another function to run later (often after async work). **Callback hell** ("pyramid of doom") is deeply nested callbacks that are hard to read and maintain. Solved by Promises / async-await.

```
// callback hell
getUser(id, (user) => {
  getOrders(user, (orders) => {
    getDetails(orders[0], (details) => {
      console.log(details); // deeply nested
    });
  });
});
```

Q: What is a Promise and what are its states?

A Promise represents the eventual result of an async operation. States: **pending** → **fulfilled** (resolved) or **rejected**. It's settled once fulfilled or rejected and cannot change again.

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve("done"), 100);
});
p.then(val => console.log(val))
  .catch(err => console.error(err))
  .finally(() => console.log("cleanup"));
```

Q: What is promise chaining?

Returning a value or another promise from `.then` so the next `.then` receives it, flattening async sequences. Errors propagate to the nearest `.catch`.

```
fetch("/user")
  .then(res => res.json())
  .then(user => fetch(`/orders/${user.id}`))
  .then(res => res.json())
  .then(orders => console.log(orders))
  .catch(err => console.error(err)); // catches any step
```

Q: What is async/await?

Syntactic sugar over promises for writing async code that reads synchronously. `async` functions always return a promise; `await` pauses until the awaited promise settles. Use `try/catch` for error handling.

```
async function loadOrders(id) {
  try {
    const user = await fetch(`/user/${id}`).then(r => r.json());
    const orders = await fetch(`/orders/${user.id}`).then(r => r.json());
    return orders;
  } catch (err) {
    console.error("failed", err);
  }
}
```

Q: Difference between `Promise.all`, `Promise.race`, and `Promise.allSettled`?

- **`Promise.all`**: resolves when **all** fulfill (array of results); rejects fast if **any** rejects.
- **`Promise.race`**: settles as soon as the **first** promise settles (fulfilled or rejected).
- **`Promise.allSettled`**: waits for **all** to settle; never rejects — returns `{status, value/reason}` for each.
- (**`Promise.any`**: first **fulfilled**; rejects only if all reject.)

```
const a = Promise.resolve(1);
const b = Promise.reject("err");
const c = Promise.resolve(3);

Promise.all([a, c]).then(console.log); // [1, 3]
Promise.race([a, b]).then(console.log); // 1 (a is faster/first)
Promise.allSettled([a, b]).then(console.log);
// [{status:"fulfilled",value:1},{status:"rejected",reason:"err"}]
```

Q: Explain the event loop.

JavaScript is single-threaded. The **call stack** runs synchronous code. Async callbacks wait in queues. The **event loop** continuously checks: if the stack is empty, it processes all **microtasks** (promises, `queueMicrotask`), then **one** **macrotask** (`setTimeout`, I/O, events), repeating. This enables non-blocking concurrency.

```
console.log("1");
setTimeout(() => console.log("2"), 0); // macrotask
Promise.resolve().then(() => console.log("3")); // microtask
console.log("4");
// Output: 1, 4, 3, 2
```

Q: Difference between microtask and macrotask queues?

- **Microtasks:** promise callbacks, `queueMicrotask`, `MutationObserver`. Drained **completely** after each task, before rendering.
- **Macrotasks:** `setTimeout`, `setInterval`, I/O, UI events. One processed per loop iteration. Microtasks always run before the next macrotask, even `setTimeout(0)`.

```
setTimeout(() => console.log("macro"), 0);
Promise.resolve().then(() => console.log("micro"));
// micro then macro
```

Q: Difference between `setTimeout` and `setInterval` ?

- `setTimeout(fn, delay)` : runs `fn` **once** after the delay.
 - `setInterval(fn, delay)` : runs `fn` **repeatedly** every delay until `clearInterval`.
- Delays are minimums, not guarantees (depends on stack/queue). A recursive `setTimeout` is often preferred over `setInterval` for guaranteed gaps between executions.

```
const t = setTimeout(() => console.log("once"), 1000);
clearTimeout(t);

let n = 0;
const i = setInterval(() => {
  if (++n === 3) clearInterval(i);
  console.log("tick", n);
}, 1000);
```

ES6+ Features

Q: What is destructuring?

Extracting values from arrays or properties from objects into distinct variables, with support for defaults, renaming, and nesting.

```
const [first, , third = 0] = [1, 2]; // first=1, third=0
const { name, age: years = 18 } = { name: "Priya" };
// name="Priya", years=18

function print({ id, role = "user" }) { // in params
  console.log(id, role);
}
```

Q: Explain spread and rest operators.

Both use `...`.

- **Spread**: expands an iterable/object into individual elements (copying, merging, passing args).
- **Rest**: collects multiple elements into a single array/object (function params, destructuring).

```
// spread
const merged = [...[1, 2], ...[3, 4]]; // [1,2,3,4]
const clone = { ...{ a: 1 }, b: 2 }; // {a:1, b:2}
Math.max(...[5, 9, 2]); // 9

// rest
function sum(...nums) { return nums.reduce((a, b) => a + b, 0); }
const [head, ...tail] = [1, 2, 3]; // head=1, tail=[2,3]
```

Q: What are template literals?

Backtick strings supporting interpolation (``${}``), multi-line strings, and tagged templates.

```
const name = "Priya";
const msg = `Hello ${name}, total = ${2 + 3}`;
const multi = `line1
line2`;
```

Q: What are default parameters?

Function parameters with fallback values used when the argument is `undefined`.

```
function greet(name = "Guest", greeting = "Hi") {
  return `${greeting}, ${name}`;
}
greet(); // "Hi, Guest"
greet("Priya"); // "Hi, Priya"
```

Q: How do ES modules (import/export) work?

Modules have their own scope. Use `export` to expose and `import` to consume. **Named exports** (multiple per file) and a single **default export**. Imports are hoisted and live (read-only bindings).

```
// math.js
export const PI = 3.14;
export function add(a, b) { return a + b; }
export default function multiply(a, b) { return a * b; }

// app.js
import multiply, { PI, add } from "./math.js";
import * as math from "./math.js";
```

Q: Difference between `Map` / `Set` and objects/arrays?

- **Map**: key-value pairs with **any** key type (objects, functions), maintains insertion order, has `.size`, easily iterable. Better for frequent additions/removals.
- **Set**: collection of **unique** values; great for deduplication and membership checks.


```
const m = new Map();
m.set("a", 1).set({ id: 1 }, "obj key");
m.get("a"); // 1
m.size;     // 2

const s = new Set([1, 1, 2, 3]);
[...s];     // [1, 2, 3]
s.has(2);   // true
```

Q: What is optional chaining (`?.`)?

Safely accesses nested properties/methods that may be `null` / `undefined` , short-circuiting to `undefined` instead of throwing.

```
const user = { profile: { address: null } };
user.profile?.address?.city; // undefined (no error)
user.getName?.();           // undefined if method absent
user.list?.[0];             // safe index access
```

Q: What is the nullish coalescing operator (`??`)?

Returns the right operand only when the left is `null` or `undefined` (unlike `||` , which triggers on any falsy value like `0` or `""`).

```
const count = 0;
count || 10; // 10 (0 is falsy)
count ?? 10; // 0 (0 is not null/undefined)

const name = null ?? "Guest"; // "Guest"
```

Q: What are generators?

Functions (`function*`) that can pause and resume using `yield` , producing values lazily on demand. Calling returns an iterator; `.next()` advances execution. Useful for lazy sequences, custom iterators, and (historically) async flows.

```
function* idGenerator() {
  let id = 1;
  while (true) yield id++;
}

const gen = idGenerator();
gen.next().value; // 1
gen.next().value; // 2

function* range(start, end) {
  for (let i = start; i < end; i++) yield i;
}

[...range(1, 4)]; // [1, 2, 3]
```

Advanced

Q: Difference between debounce and throttle (with implementation)?

- **Debounce:** delays execution until a pause of `delay` ms after the last call. Good for search input, resize-end.
- **Throttle:** ensures execution at most once per `delay` interval. Good for scroll, mousemove.

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

function throttle(fn, limit) {
  let inThrottle = false;
  return function (...args) {
    if (!inThrottle) {
      fn.apply(this, args);
      inThrottle = true;
      setTimeout(() => (inThrottle = false), limit);
    }
  };
}
```

Q: What is memoization? Implement it.

Caching the results of expensive function calls keyed by arguments, returning the cached result on repeat inputs.

```
function memoize(fn) {
  const cache = new Map();
  return function (...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const result = fn.apply(this, args);
    cache.set(key, result);
    return result;
  };
}

const slowSquare = n => { /* heavy */ return n * n; };
const fastSquare = memoize(slowSquare);
fastSquare(4); // computes
fastSquare(4); // cached
```

Q: Implement a deep clone function.

Recursively copy nested structures. (`structuredClone` is the built-in modern option, but interviewers often want the manual version.)

```
function deepClone(value) {
  if (value === null || typeof value !== "object") return value;
  if (value instanceof Date) return new Date(value);
  if (Array.isArray(value)) return value.map(deepClone);
  return Object.fromEntries(
    Object.entries(value).map(([k, v]) => [k, deepClone(v)])
  );
}

const a = { x: 1, nested: { y: 2 }, arr: [1, 2] };
const b = deepClone(a);
b.nested.y = 99;
a.nested.y; // 2 (independent)
```

Q: Implement a generic curry function.

Collect arguments until the original function's arity is satisfied, then invoke.

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) return fn.apply(this, args);
    return (...next) => curried.apply(this, [...args, ...next]);
  };
}

const add = (a, b, c) => a + b + c;
const cAdd = curry(add);
cAdd(1)(2)(3); // 6
cAdd(1, 2)(3); // 6
cAdd(1)(2, 3); // 6
```

Q: What is event delegation?

Attaching a single listener to a common ancestor instead of many child listeners, relying on event bubbling. The handler inspects `event.target`. Improves performance and handles dynamically added elements.

```
document.getElementById("list").addEventListener("click", (e) => {
  if (e.target.matches("li")) {
    console.log("clicked", e.target.textContent);
  }
});
// works even for <li> added later
```

Q: Explain event bubbling and capturing.

DOM event propagation has three phases:

1. **Capturing** (top → target): listeners with `{capture: true}`.
2. **Target**: the event reaches the actual element.
3. **Bubbling** (target → top): default phase for most handlers.

Use `stopPropagation()` to halt and `event.target` (origin) vs `event.currentTarget` (handler's element).

```
parent.addEventListener("click", () => console.log("parent"), true); // capture
child.addEventListener("click", () => console.log("child"));           // bubble
// click child -> "parent" (capture) then "child" (target/bubble)
child.addEventListener("click", e => e.stopPropagation());
```

Tricky Output-Based Questions

Q: What is the output?

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

Output: 3 3 3. `var` is function-scoped, so all callbacks share the same `i`, which is 3 by the time the timeouts run. Fix with `let` (block-scoped, new binding per iteration) → prints 0 1 2.

Q: What is the output?

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
```

Output: 1 4 3 2. Synchronous logs (1, 4) run first. The microtask queue (promise → 3) drains before macrotasks (setTimeout → 2).

Q: What is the output?

```
console.log(typeof null);
console.log(0.1 + 0.2 === 0.3);
console.log([] + []);
console.log(!"false");
```

Output: "object", false, "", true.

- `typeof null` is "object" (legacy bug).
- Floating-point: `0.1 + 0.2` = `0.30000000000000004`.
- `[] + []` coerces both arrays to empty strings → "".
- The non-empty string `"false"` is truthy → `!!` gives `true`.

Q: What is the output?

```
const obj = {
  name: "Priya",
  regular: function () { return this.name; },
  arrow: () => this.name,
};
console.log(obj.regular());
console.log(obj.arrow());
```

Output: "Priya", undefined . regular is called as a method so this is obj . The arrow function has no own this — it inherits from the enclosing (module/global) scope where this.name is undefined .

Q: What is the output?

```
let a = { n: 1 };  
let b = a;  
a.n = 2;  
a = { n: 3 };  
console.log(b.n);
```

Output: 2 . b references the original object. a.n = 2 mutates that shared object (so b.n becomes 2). Re-assigning a to a new object doesn't affect b , which still points to the first object.

HTML & CSS — Interview Q&A

HTML

Q: What is semantic HTML and why does it matter?

Semantic HTML uses tags that describe the meaning of the content rather than just its appearance — `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`, `<figure>`. It matters because:

- **Accessibility**: screen readers can navigate landmarks (e.g. jump to `<nav>` or `<main>`).
- **SEO**: search engines understand page structure and rank content better.
- **Maintainability**: code is self-documenting vs a sea of `<div>` s.

```
<!-- Non-semantic -->
<div class="header"><div class="nav">...</div></div>

<!-- Semantic -->
<header><nav>...</nav></header>
```

Q: What is the difference between block and inline elements?

- **Block** elements start on a new line and take the full available width; height/width and vertical margins/padding apply. Examples: `<div>`, `<p>`, `<h1>`, `<ul>`, `<section>`.
- **Inline** elements flow within text, take only as much width as their content, and ignore width/height and top/bottom margins. Examples: `<span>`, `<a>`, `<strong>`, `<img>` (inline-block-ish).
- **inline-block** flows inline but respects width/height/margins.

Q: What is the difference between `<div>` and `<span>` ?

Both are generic, non-semantic containers with no default styling. `<div>` is a **block** element used to group larger sections/layout; `<span>` is an **inline** element used to style or target a small piece of text inside a line.

```
<div>A block container</div>
<p>Highlight <span class="warn">this word</span> only.</p>
```

Q: What are `data-*` attributes?

Custom attributes to store extra data on an element without misusing class/id. Read in JS via `dataset` or in CSS via attribute selectors.

```
<button data-user-id="42" data-role="admin">Edit</button>
```

```
el.dataset.userId // "42"
el.dataset.role   // "admin"
```

Commonly used to bind config to elements (e.g. analytics, widget options).

Q: What are meta tags and which are important?

`<meta>` tags live in `<head>` and provide metadata about the document. Key ones:

- `charset` — character encoding.
- `viewport` — responsive scaling (essential for mobile).
- `description` — SEO snippet shown in search results.
- Open Graph (`og:title` , `og:image`) — social media previews.

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="Concise page summary for SEO.">
```

Q: What is the difference between `async` and `defer` on a script tag?

Both download the script without blocking HTML parsing. The difference is execution timing:

- **async**: executes as soon as it's downloaded, possibly before parsing finishes; order not guaranteed. Good for independent scripts (analytics).
- **defer**: executes only after HTML parsing completes, in document order. Good for scripts that depend on the DOM or each other.

```
<script src="analytics.js" async></script>
<script src="app.js" defer></script>
```

Without either, the script blocks parsing while it downloads and runs.

Q: localStorage vs sessionStorage vs cookies?

Feature	localStorage	sessionStorage	cookies
Capacity	~5–10 MB	~5 MB	~4 KB
Lifetime	Until cleared	Until tab closes	Set by <code>expires</code> / <code>max-age</code>
Sent to server	No	No	Yes, on every request
Scope	Per origin	Per tab	Per domain/path

Use `localStorage` for persistent client-only data, `sessionStorage` for per-tab temporary data, cookies for auth tokens/server-read data (prefer `HttpOnly` , `Secure` , `SameSite`).

Q: What are common HTML5 form input types?

`text` , `email` , `password` , `number` , `tel` , `url` , `date` , `time` , `color` , `range` , `checkbox` , `radio` , `file` , `search` . They give built-in validation and mobile keyboards.

```
<input type="email" required>
<input type="number" min="1" max="10" step="1">
<input type="date">
```

Pair with `required` , `pattern` , `min` / `max` for native validation and always use `<label for>` for accessibility.

Q: What are the basics of accessibility (ARIA)?

ARIA (Accessible Rich Internet Applications) adds semantics for assistive tech when native HTML isn't enough. Rules of thumb:

- **Use native elements first** (`<button>` over `<div role="button">`).
- `aria-label` / `aria-labelledby` name an element.
- `role` describes a widget's purpose.
- `aria-hidden="true"` hides decorative content from screen readers.
- `aria-live` announces dynamic updates.

```
<button aria-label="Close dialog">&times;</button>
<div role="alert" aria-live="assertive">Saved!</div>
```

Q: What is `<!DOCTYPE html>` and why is it needed?

It tells the browser to render in **standards mode** rather than quirks mode (legacy IE-compatible behavior). `<!DOCTYPE html>` is the short HTML5 doctype. Without it, the box model and CSS can behave inconsistently. It must be the very first line of the document.

Q: What is an iframe and what are its concerns?

An `<iframe>` embeds another HTML document inside the current page (e.g. maps, videos, payment widgets).

Concerns:

- **Security**: use the `sandbox` attribute to restrict capabilities and `allow` to whitelist features.
- **Performance**: each iframe loads a separate document; use `loading="lazy"`.
- **SEO**: content inside iframes isn't attributed to your page.

```
<iframe src="https://example.com" sandbox="allow-scripts" loading="lazy" title="Embedded report"></iframe>
```

Q: What are some on-page SEO basics?

- One descriptive `<title>` and a `<meta name="description">` per page.
- A single `<h1>`, with logical `<h2>` / `<h3>` hierarchy.
- Semantic structure and descriptive `alt` text on images.
- Clean, readable URLs and `<a>` text (avoid "click here").
- Canonical tags to avoid duplicate content, fast load times, mobile-friendliness, and structured data (schema.org).

CSS

Q: Explain the CSS box model.

Every element is a box with four layers, from inside out: **content** → **padding** → **border** → **margin**. Total rendered width depends on `box-sizing`:

- `content-box` (default): width = content only; padding/border add on top.
- `border-box`: width includes content + padding + border (more predictable).

```
*, ::before, ::after { box-sizing: border-box; }
```


Q: What is margin collapsing?

When vertical margins of adjacent or parent/child block elements touch, they **collapse** into a single margin equal to the larger value (not the sum). It happens between adjacent siblings, between empty blocks, and parent/first-child. It does **not** happen horizontally, with flex/grid items, or when separated by padding/border/overflow.

```
/* two stacked <p> with margin 20px and 30px → gap is 30px, not 50px */
```

Q: Explain CSS position values.

- **static**: default; normal flow, ignores top/left/etc.
- **relative**: offset from its normal position; still occupies original space; creates a positioning context.
- **absolute**: removed from flow; positioned relative to nearest positioned ancestor.
- **fixed**: removed from flow; positioned relative to the viewport; stays on scroll.
- **sticky**: hybrid — acts relative until a scroll threshold, then sticks like fixed within its container.

```
.header { position: sticky; top: 0; }
```

Q: What are the main CSS display types?

- **block** / **inline** / **inline-block** — flow behavior (see HTML section).
- **none** — removes element from layout entirely.
- **flex** — one-dimensional flexible layout.
- **grid** — two-dimensional layout.
- **table**, **contents** — less common specialized values.

Q: How is CSS specificity calculated?

Specificity is a 4-part value (a, b, c, d):

- a = inline styles
- b = IDs
- c = classes, attributes, pseudo-classes
- d = elements, pseudo-elements

Higher wins; ties go to the later rule. The universal selector ***** adds nothing.

```
#nav .item a    /* (0,1,1,1) */  
.item a         /* (0,0,1,1) → loses */
```

Q: What does **!important** do and when should you use it?

!important overrides normal specificity and cascade, forcing a declaration to win (an **!important** later in the cascade beats an earlier one). Avoid it — it breaks the natural cascade and is hard to override. Legitimate uses: overriding third-party/library styles (e.g. Bootstrap, Kendo) you can't change, or utility classes. Prefer raising specificity properly first.

Q: Inline vs internal vs external CSS?

- **Inline**: **style** attribute on the element — highest specificity, not reusable, hard to maintain.
- **Internal**: **<style>** block in **<head>** — page-scoped, no extra request.

- **External:** `<link>` to a `.css` file — reusable, cacheable, best for real projects.

```
<link rel="stylesheet" href="styles.css">
```

Q: Difference between pseudo-classes and pseudo-elements?

- **Pseudo-class** (single colon `:`) targets a **state** of an element: `:hover`, `:focus`, `:nth-child()`, `:first-child`, `:checked`.
- **Pseudo-element** (double colon `::`) styles a **part** of an element or inserts generated content: `::before`, `::after`, `::first-line`, `::placeholder`.

```
a:hover { color: blue; }
.card::before { content: "★"; }
```

Q: em vs rem vs px vs % ?

- **px:** absolute, fixed size.
- **em:** relative to the **parent's** font-size (compounds when nested).
- **rem:** relative to the **root** (`<html>`) font-size — predictable, great for scalable typography/spacing.
- **%:** relative to the parent's corresponding dimension.

```
html { font-size: 16px; }
.title { font-size: 2rem; } /* 32px */
.box { width: 50%; } /* half of parent width */
```

Q: What is z-index and stacking context?

z-index controls the front-to-back order of positioned/flex/grid items. It only works within the same **stacking context**. A new stacking context is created by, e.g., a positioned element with a **z-index**, **opacity < 1**, **transform**, **filter**, or **position: fixed**. A child's z-index can never escape its parent's stacking context, which is the usual cause of "z-index not working".

```
.modal { position: fixed; z-index: 1000; }
```

Layout

Q: Explain Flexbox and its main properties.

Flexbox is a one-dimensional layout system (a row OR a column). On the container:

- **display: flex**
- **flex-direction** — row | column
- **justify-content** — alignment along the main axis
- **align-items** — alignment along the cross axis
- **flex-wrap** — wrap onto multiple lines
- **gap** — spacing between items

On items: **flex-grow**, **flex-shrink**, **flex-basis** (shorthand **flex**), **align-self**.

```
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
}
```

Q: Explain CSS Grid basics.

Grid is a two-dimensional layout system (rows AND columns simultaneously). Define tracks on the container and place items into the grid.

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
  grid-template-rows: auto;
  gap: 20px;
}
.feature { grid-column: span 2; } /* item spans 2 columns */
```

`fr` = fraction of available space; `repeat()`, `minmax()`, and `auto-fit/auto-fill` enable responsive grids.

Q: Flexbox vs Grid — when to use which?

- **Flexbox:** one-dimensional layouts — navbars, toolbars, button groups, distributing items in a single row/column. Content-driven sizing.
- **Grid:** two-dimensional layouts — page layouts, card galleries, dashboards where you control rows and columns together. Layout-driven.

They compose well: use Grid for the overall page, Flexbox for components inside cells.

Q: How do you center a div?

Multiple ways:

```
/* Flexbox (most common) */
.parent { display: flex; justify-content: center; align-items: center; }

/* Grid */
.parent { display: grid; place-items: center; }

/* Absolute + transform */
.child {
  position: absolute; top: 50%; left: 50%;
  transform: translate(-50%, -50%);
}

/* Horizontal block centering */
.child { margin: 0 auto; width: 300px; }
```

Responsive

Q: What are media queries?

Media queries apply CSS conditionally based on device characteristics — width, height, orientation, resolution, or `prefers-color-scheme`. They're the core of responsive design.

```
@media (max-width: 768px) {  
  .sidebar { display: none; }  
}  
@media (min-width: 1024px) {  
  .container { max-width: 960px; }  
}
```

Q: What is mobile-first design?

Write base styles for small screens first, then layer enhancements for larger screens using `min-width` media queries. Benefits: simpler base CSS, better performance on mobile, and progressive enhancement. (Bootstrap is mobile-first.)

```
.card { width: 100%; } /* mobile default */  
@media (min-width: 768px) {  
  .card { width: 50%; } /* tablet+ */  
}
```

Q: Why is the viewport meta tag needed?

Without it, mobile browsers render at a default ~980px width and scale down, so media queries don't trigger correctly. The viewport meta tells the browser to match the device's width and set initial zoom.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Q: What are responsive units?

Units that scale with context instead of being fixed:

- `%` — relative to parent.
- `rem` / `em` — relative to font-size.
- `vw` / `vh` — 1% of viewport width/height.
- `vmin` / `vmax` — relative to smaller/larger viewport dimension.
- `clamp(min, preferred, max)` — fluid sizing with bounds.

```
.title { font-size: clamp(1.5rem, 4vw, 3rem); }
```

Q: How does the Bootstrap grid system work?

Bootstrap uses a 12-column, flexbox-based, mobile-first grid:

- `.container` (or `.container-fluid`) wraps the grid.
- `.row` is a flex row of columns.
- `.col-*` define widths; columns in a row should add up to 12.
- Responsive breakpoint infixes: `col-sm-`, `col-md-`, `col-lg-`, `col-xl-`, `col-xxl-`.

```

<div class="container">
  <div class="row">
    <div class="col-12 col-md-8">Main</div>
    <div class="col-12 col-md-4">Sidebar</div>
  </div>
</div>

```

On mobile each is full width (12); from `md` up, they split 8/4. Use `g-*` for gutters and offset/order utilities for fine control.

Modern / Advanced

Q: What are CSS variables (custom properties)?

Reusable values declared with `--name` and read with `var()`. Unlike preprocessor variables, they're live in the DOM, cascade, inherit, and can be changed at runtime via JS or media queries — great for theming.

```

:root {
  --primary: #0d6efd;
  --space: 8px;
}
.btn { background: var(--primary); padding: var(--space); }

```

```
document.documentElement.style.setProperty('--primary', '#198754');
```

Q: Transitions vs animations?

- **Transitions** animate a property between two states, triggered by a change (e.g. `:hover`). Simple, one-shot.
- **Animations** use `@keyframes` for multi-step, looping, or self-starting motion with fine control.

```

.btn { transition: background 0.3s ease; }

@keyframes pulse { 0%,100% { opacity: 1; } 50% { opacity: 0.5; } }
.loader { animation: pulse 1s infinite; }

```

Q: What is the CSS transform property?

`transform` applies 2D/3D visual changes — `translate`, `scale`, `rotate`, `skew` — without affecting document flow (no reflow). It's GPU-accelerated and composited, so it's the performant way to animate position/size.

```
.card:hover { transform: scale(1.05) rotate(2deg); }
```

Q: opacity vs visibility vs display:none — differences?

Property	In layout?	Clickable?	Animatable?
<code>opacity: 0</code>	Yes (space kept)	Yes	Yes
<code>visibility: hidden</code>	Yes (space kept)	No	Limited
<code>display: none</code>	No (removed)	No	No

Use `display:none` to fully remove, `visibility:hidden` to hide but keep space, `opacity:0` for fade animations.

Q: What is BEM naming?

BEM (Block, Element, Modifier) is a CSS naming convention for predictable, scoped class names:

- **Block** — standalone component: `.card`
- **Element** — a part of the block (double underscore): `.card__title`
- **Modifier** — a variant (double hyphen): `.card--featured`

```
<div class="card card--featured">
  <h2 class="card__title">...</h2>
</div>
```

It keeps specificity flat and avoids naming collisions.

Q: What causes reflow and repaint?

- **Reflow (layout)**: recalculating element geometry/positions. Triggered by changing width/height, adding/removing DOM nodes, font changes, or reading layout properties like `offsetHeight`. Expensive.
- **Repaint**: redrawing pixels without layout changes — e.g. `color`, `background`, `visibility`.

Minimize reflows: batch DOM changes, animate `transform` / `opacity` (compositor-only) instead of `top` / `width`, and avoid layout thrashing (read-then-write loops).

Q: What is the critical rendering path?

The sequence the browser follows to turn HTML/CSS/JS into pixels:

1. Parse HTML → **DOM**.
2. Parse CSS → **CSSOM**.
3. Combine into the **Render Tree**.
4. **Layout** (compute geometry).
5. **Paint** and **Composite**.

CSS is render-blocking and JS can block parsing, so optimize by minimizing/inlining critical CSS, deferring non-critical JS (`async` / `defer`), and reducing resource sizes to render faster.

TypeScript — Interview Q&A

Lateral interview prep (4+ years, Angular developer). Focused, practical answers with TypeScript code examples.

Basics

Q: What is TypeScript and what are its advantages over JavaScript?

TypeScript is a strongly-typed **superset of JavaScript** developed by Microsoft. Any valid JS is valid TS. It adds static typing, interfaces, generics, enums, decorators and modern features that compile down to plain JavaScript.

Advantages:

- **Static type checking** catches errors at compile time, not runtime.
- **Better tooling** — autocomplete, refactoring, inline docs (IntelliSense).
- **Self-documenting** code via types and interfaces.
- **Safer refactoring** in large codebases (Angular apps especially).
- Supports modern ES features and **decorators** that Angular relies on.

```
function add(a: number, b: number): number {  
  return a + b;  
}  
add(2, 3);           // OK  
// add(2, "3");    // Compile-time error
```

Q: How does TypeScript compile / how does it run?

TypeScript code (`.ts`) is **transpiled** to JavaScript (`.js`) by the TypeScript compiler (`tsc`), because browsers and Node only understand JS. Types are **erased** during compilation — they exist only at compile time and have no runtime cost. In Angular, the CLI (`ng build`) runs this compilation under the hood.

```
tsc app.ts           # produces app.js  
tsc --watch          # recompile on change
```

Q: What are the key tsconfig.json options you should know?

`tsconfig.json` configures the compiler. Important options:

```
{
  "compilerOptions": {
    "target": "ES2022",          // JS version to output
    "module": "ESNext",         // module system
    "strict": true,             // enables all strict type checks
    "noImplicitAny": true,      // error on implicit 'any'
    "strictNullChecks": true,   // null/undefined must be handled
    "outDir": "./dist",         // output folder
    "sourceMap": true,          // for debugging
    "experimentalDecorators": true // required by Angular
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules"]
}
```

`strict: true` is the most important — it turns on a family of checks (`strictNullChecks` , `noImplicitAny` , etc.).

Q: What is the difference between `type` and `interface` ?

Both describe the shape of an object. Differences:

- **interface** can be **re-opened** / **merged** (declaration merging); `type` cannot.
- **interface** is primarily for object shapes; **type** can alias **any** type — unions, primitives, tuples, etc.
- Both support extension (`extends` for interface, `&` for type).

```
interface User { name: string; }
interface User { age: number; }    // merged → { name, age }

type ID = string | number;         // union — only 'type' can do this
type Point = { x: number; y: number };
```

Rule of thumb: use `interface` for object/class contracts, `type` for unions and complex compositions.

Q: Explain `any` vs `unknown` vs `never` vs `void` .

- **any** — opts out of type checking entirely (avoid it).
- **unknown** — type-safe `any` ; you must narrow it before use.
- **never** — represents values that never occur (function that always throws or never returns).
- **void** — function returns nothing meaningful.

```
let a: any = 5;      a.foo();           // allowed (unsafe)
let u: unknown = 5;  // u.foo();         // Error until narrowed
if (typeof u === "string") u.trim();    // OK after narrowing

function fail(): never { throw new Error("x"); }
function log(): void { console.log("hi"); }
```

Q: What is type inference?

TypeScript automatically **infers** types when you don't annotate them, based on the assigned value. This reduces boilerplate while keeping safety.


```
let count = 10;           // inferred as number
// count = "ten";        // Error
const names = ["a"];      // inferred string[]
```

You only need explicit annotations where inference can't help (function params, ambiguous cases).

Q: What are literal types?

Literal types restrict a value to **specific exact values**, not just a broad type. Often combined with unions.

```
let dir: "left" | "right";
dir = "left";           // OK
// dir = "up";          // Error

type Status = 200 | 404 | 500;
```

Useful for Angular component inputs with a fixed set of allowed values.

Q: What are union and intersection types?

- **Union** (`|`) — value can be **one of** several types.
- **Intersection** (`&`) — value must satisfy **all** types combined.

```
type Id = string | number;           // union

interface A { a: number; }
interface B { b: string; }
type AB = A & B;                     // intersection → { a, b }
const ab: AB = { a: 1, b: "x" };
```

Q: What are tuples?

A tuple is a fixed-length array with a **known type at each position**.

```
let pair: [string, number] = ["age", 30];
pair[0].toUpperCase();    // string
pair[1].toFixed();        // number

// useState-style return
function useToggle(): [boolean, () => void] {
  return [true, () => {}];
}
```

Q: What are enums?

Enums define a set of named constants. Numeric by default; can be string enums (preferred for readability/debugging).

```
enum Direction { Up, Down }         // 0, 1
enum Role { Admin = "ADMIN", User = "USER" }

let r: Role = Role.Admin;           // "ADMIN"
```

`const enum` inlines values at compile time for performance. Many teams use union literal types instead of enums to avoid extra generated code.

Functions

Q: How do you type functions in TypeScript?

You annotate parameters and the return type. You can also type a function variable using a function-type signature.

```
function greet(name: string): string {  
  return `Hi ${name}`;  
}  
  
// function type  
const multiply: (a: number, b: number) => number = (a, b) => a * b;
```

Q: How do optional and default parameters work?

- **Optional** params use `?` and may be `undefined`.
- **Default** params provide a fallback value (and become optional).

```
function build(name: string, age?: number) {           // optional  
  return age ? `${name} ${age}` : name;  
}  
  
function greet(name: string, greeting = "Hello") {    // default  
  return `${greeting} ${name}`;  
}
```

Optional params must come **after** required ones.

Q: What are rest parameters?

Rest params collect remaining arguments into a typed array.

```
function sum(...nums: number[]): number {  
  return nums.reduce((a, b) => a + b, 0);  
}  
sum(1, 2, 3);    // 6
```

Q: What is function overloading?

Multiple function signatures for the same function name, with one implementation that handles all cases. Useful when return type depends on input type.

```
function parse(input: string): string[];
function parse(input: number): number;
function parse(input: string | number): string[] | number {
  return typeof input === "string" ? input.split("") : input * 2;
}

parse("ab");    // string[]
parse(5);       // number
```

OOP

Q: How do you define a class in TypeScript?

```
class User {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  greet(): string {
    return `Hi ${this.name}`;
  }
}
```

TypeScript adds access modifiers, parameter properties, abstract members and more on top of ES classes.

Q: Explain the access modifiers: public, private, protected, readonly.

- **public** (default) — accessible everywhere.
- **private** — only within the class.
- **protected** — within the class and its subclasses.
- **readonly** — can be set only in declaration/constructor, then immutable.

```
class Account {
  public id: number;
  private pin: number;
  protected balance: number;
  readonly bank = "ABC";

  // parameter properties shorthand
  constructor(id: number, pin: number, balance: number) {
    this.id = id; this.pin = pin; this.balance = balance;
  }
}
```

Angular uses this for DI: `constructor(private http: HttpClient) {}` declares and assigns in one line.

Q: What are abstract classes?

A base class that **cannot be instantiated** directly and can define abstract members that subclasses **must implement**. It can also contain shared concrete logic.

```

abstract class Shape {
  abstract area(): number;           // must be implemented
  describe() { return `Area: ${this.area()}`; } // shared
}

class Circle extends Shape {
  constructor(private r: number) { super(); }
  area() { return Math.PI * this.r ** 2; }
}

```

Q: Interface vs abstract class — when to use which?

Interface	Abstract class
Only declarations, no implementation	Can have implemented methods + state
A class can implement many	A class extends one
Erased at compile time (no JS output)	Generates real JS
Pure contract	Shared base behavior

Use an interface for a contract; use an abstract class when you want to share code among subclasses.

Q: Difference between **implements** and **extends** ?

- **extends** — inherit from a class (or interface from interface). Brings implementation.
- **implements** — promise to satisfy an interface's contract; no implementation inherited.

```

interface Logger { log(msg: string): void; }

class Base { id = 1; }
class Service extends Base implements Logger {
  log(msg: string) { console.log(msg); }
}

```

Q: How do getters and setters work?

Accessors let you run logic when reading/writing a property using normal property syntax.

```

class Temp {
  private _c = 0;
  get celsius() { return this._c; }
  set celsius(v: number) {
    if (v < -273) throw new Error("too low");
    this._c = v;
  }
}

const t = new Temp();
t.celsius = 25;           // calls setter
console.log(t.celsius);   // calls getter

```

Q: What are static members?

Static properties/methods belong to the **class itself**, not instances. Accessed via the class name.

```
class MathUtil {  
  static PI = 3.14;  
  static square(n: number) { return n * n; }  
}  
MathUtil.square(4);    // 16, no instance needed
```

Advanced Types

Q: What are generics? Give an example.

Generics let you write **reusable**, **type-safe** code that works over multiple types while preserving the specific type. A type parameter (`<T>`) acts as a placeholder.

```
function identity<T>(value: T): T {  
  return value;  
}  
identity<string>("hi");    // T = string  
identity(10);              // T inferred as number  
  
// generic class  
class Box<T> {  
  constructor(public content: T) {}  
}  
const b = new Box<number>(5);
```

Angular's `Observable<User>` and `HttpClient.get<User>()` are generics in action.

Q: What are generic constraints?

Restrict a generic type to types that have certain properties, using `extends` .

```
function getLength<T extends { length: number }>(item: T): number {  
  return item.length;  
}  
getLength("abc");          // OK  
getLength([1, 2]);         // OK  
// getLength(5);          // Error: number has no length  
  
// keyof constraint  
function getProp<T, K extends keyof T>(obj: T, key: K): T[K] {  
  return obj[key];  
}
```

Q: Explain the common utility types: Partial, Required, Pick, Omit, Record, Readonly.

```
interface User { id: number; name: string; email: string; }

Partial<User>;    // all props optional → { id?, name?, email? }
Required<User>;  // all props required
Pick<User, "id" | "name">;    // { id, name }
Omit<User, "email">;          // { id, name }
Readonly<User>;              // all props immutable
Record<string, number>;      // { [key: string]: number }
```

- **Partial** — great for update/patch payloads.
- **Pick / Omit** — derive a smaller type.
- **Record** — build a map/dictionary type.
- **Readonly** — immutable copy.

Q: What are mapped types?

They create new types by **transforming each property** of an existing type. This is how utility types like **Partial** are built.

```
type Optional<T> = { [K in keyof T]?: T[K] };
type Stringify<T> = { [K in keyof T]: string };

interface User { id: number; name: string; }
type UserStrings = Stringify<User>;    // { id: string; name: string }
```

Q: What are conditional types?

Types that choose between two types based on a condition, using the ternary-like **T extends U ? X : Y**.

```
type IsString<T> = T extends string ? "yes" : "no";
type A = IsString<string>;    // "yes"
type B = IsString<number>;    // "no"

// extract non-null
type NonNullableX<T> = T extends null | undefined ? never : T;
```

Q: What are type guards?

Runtime checks that **narrow** a type within a block. Built-in guards: **typeof**, **instanceof**, **in**. You can also write custom guards with a type predicate (**x is Type**).

```
function isString(x: unknown): x is string {
    return typeof x === "string";
}

function print(val: string | number) {
    if (isString(val)) val.toUpperCase(); // narrowed to string
    else val.toFixed(2);                 // number
}
```

Q: Explain `typeof`, `keyof`, and `in`.

- `typeof` — get the type of a variable/value.
- `keyof` — get a union of an object type's keys.
- `in` — iterate keys in mapped types / check property existence at runtime.

```
const config = { url: "x", retries: 3 };
type Config = typeof config;           // { url: string; retries: number }
type Keys = keyof Config;              // "url" | "retries"

type Flags = { [K in Keys]: boolean }; // 'in' in a mapped type

if ("retries" in config) {}           // runtime 'in'
```

Q: What is type assertion (`as`)?

Tells the compiler to treat a value as a specific type when **you** know more than it does. It does **no** runtime conversion.

```
const el = document.getElementById("app") as HTMLInputElement;
el.value = "hello";

const data = JSON.parse(str) as User;
```

Avoid overusing it — it bypasses type safety. Prefer type guards where possible.

Q: What is the non-null assertion operator (`!`)?

`!` tells the compiler a value is **not null/undefined**, even though the type says it could be. Use only when you're certain.

```
function getName(user?: { name: string }) {
  return user!.name; // assert user is defined
}

@ViewChild('ref') ref!: ElementRef; // common in Angular
```

The `ref!: ElementRef` form is "definite assignment assertion" — promises it will be assigned later.

Q: What are index signatures?

Allow objects with **dynamic keys** of a known type when you don't know the property names ahead of time.

```
interface StringMap {
  [key: string]: string;
}

const headers: StringMap = { "Content-Type": "application/json" };
headers["Authorization"] = "Bearer ...";
```

Decorators, Declaration Files, Modules

Q: What are decorators? How does Angular use them?

Decorators are special functions prefixed with `@` that **add metadata or modify** classes, methods, properties or parameters. They require `experimentalDecorators` in tsconfig. Angular is built around them:

```
@Component({                // class decorator
  selector: 'app-user',
  template: '...'
})
export class UserComponent {
  @Input() name!: string;    // property decorator
  @Output() saved = new EventEmitter();
  @ViewChild('ref') ref!: ElementRef;

  constructor(@Inject(TOKEN) private cfg: Config) {} // parameter decorator
}

@Injectable({ providedIn: 'root' }) // marks a service for DI
export class UserService {}
```

The decorator metadata tells Angular how to instantiate and wire up the class.

Q: What are declaration files (`.d.ts`)?

`.d.ts` files contain **only type declarations** (no implementation). They describe the shape of JavaScript libraries so TypeScript can type-check usage of plain JS packages. Many libs ship their own, or you install `@types/...` from DefinitelyTyped.

```
// types.d.ts
declare module "my-js-lib" {
  export function doThing(x: number): string;
}

declare const VERSION: string; // ambient declaration
```

```
npm i -D @types/lodash # community type definitions
```

Q: Namespaces vs modules — what's the difference?

- **Modules** (ES Modules) — file-based; use `import` / `export`. This is the **standard, recommended** approach and what Angular uses.
- **Namespaces** — an older TS-internal way to group code under a single global name (`namespace X {}`). Mostly legacy; avoid in modern apps.


```
// module (preferred)
export class UserService {}
import { UserService } from './user.service';

// namespace (legacy)
namespace Utils {
  export function format() {}
}
Utils.format();
```

Practical / Angular

Q: How does TypeScript specifically help in Angular development?

- **Strongly typed DI** — constructor injection with typed services.
- **Typed HTTP** — `http.get<User[]>(url)` gives typed responses.
- **Typed Inputs/Outputs** and template type checking (with strict templates).
- **Interfaces/models** for API data ensure consistency across components.
- **Generics** in RxJS (`Observable<T>` , `BehaviorSubject<T>`).
- **Decorators** enable the entire component/service/module architecture.
- **Refactoring safety** across large enterprise apps — renames and signature changes are caught at compile time.

```
interface User { id: number; name: string; }

@Injectable({ providedIn: 'root' })
export class UserService {
  constructor(private http: HttpClient) {}
  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('/api/users');
  }
}
```

Q: What are common TypeScript compiler errors and how do you fix them?

1. **Object is possibly 'null' or 'undefined'** (strictNullChecks)

```
const el = document.querySelector('.x');
// el.textContent = 'hi';           // Error
el?.textContent;                   // fix: optional chaining
if (el) el.textContent = 'hi';     // or narrow
```

2. **Type 'string' is not assignable to type 'number'** — wrong type assigned; fix the value or the declared type.

3. **Parameter 'x' implicitly has an 'any' type** (noImplicitAny) — add an explicit type:

```
function f(x: number) {}
```

4. **Property 'foo' does not exist on type ...** — typo, missing interface field, or needs a type assertion/guard.

5. **Cannot find module '...' or its type declarations** — install `@types/...` or add a `.d.ts` declaration.
6. **Property has no initializer and is not definitely assigned** (`strictPropertyInitialization`) — use the definite assignment `!:`

```
@Input() name!: string;
```

Q: How do you type an HTTP response and handle optional API fields?

Define an interface mirroring the API and mark optional fields with `?`. Use `Partial` for patch payloads.

```
interface Product {  
  id: number;  
  name: string;  
  description?: string; // optional  
}  
  
updateProduct(id: number, changes: Partial<Product>) {  
  return this.http.patch<Product>(`/api/products/${id}`, changes);  
}
```

This keeps components, services and templates aligned on a single source of truth for the data shape.

Angular — Interview Q&A

Primary skill file. Tailored for a Wipro lateral (4+ years) role. Experience covered: Angular (migration to Angular 16), Ionic, NgRx, Angular Formly, reactive forms, RxJS, Kendo UI, custom reusable components, unit testing.

Core Concepts

Q: What is Angular and how is it different from AngularJS?

Angular is a TypeScript-based, component-driven SPA framework for building scalable web (and via Ionic, mobile) applications. AngularJS (1.x) was JavaScript-based and used controllers, `$scope`, and directives with two-way digest-cycle binding.

Key differences:

- **Language:** Angular uses TypeScript; AngularJS used plain JavaScript.
- **Architecture:** Angular uses Components + Services; AngularJS used Controllers + `$scope`.
- **DI:** Angular has a hierarchical, typed DI system; AngularJS used string-based injection.
- **Binding/CD:** Angular uses unidirectional data flow with zone.js change detection; AngularJS used the digest cycle (`$digest`) which was slower.
- **Mobile:** Angular is mobile-friendly by design; AngularJS was not.
- **CLI & Tooling:** Angular ships a powerful CLI, AOT compilation, lazy loading, and tree-shaking.

Q: What is a Single Page Application (SPA)?

A SPA loads a single HTML shell once, then dynamically rewrites the view in the browser using JavaScript instead of requesting full new pages from the server. The Angular Router swaps components in/out without full page reloads.

Benefits: fast navigation, rich UX, less server load. Trade-offs: heavier initial bundle, SEO needs SSR (Angular Universal), and routing/state must be handled client-side.

Q: What is a Component in Angular?

A component is the fundamental building block of UI — a class decorated with `@Component` that controls a section of the screen (a "view"). It bundles a template (HTML), styles, and logic.

```
@Component({
  selector: 'app-user-card',
  templateUrl: './user-card.component.html',
  styleUrls: ['./user-card.component.scss'],
})
export class UserCardComponent {
  @Input() name = '';
}
```

Every component has a selector (how it's used in templates), a template, and a class that holds data and behavior.

Q: What is an NgModule?

An `@NgModule` is a container that groups related components, directives, pipes, and services. It tells Angular how to compile and run a section of the app.

```
@NgModule({
  declarations: [AppComponent, UserCardComponent], // components/directives/pipes owned by
  this module
  imports: [BrowserModule, FormsModule],           // other modules this module needs
  providers: [UserService],                         // services
  bootstrap: [AppComponent],                       // root component (root module only)
})
export class AppModule {}
```

- `declarations`: what belongs to this module.
- `imports`: bring in other modules' exported features.
- `exports`: make declarations available to importing modules.
- `providers`: register services.

Q: What are standalone components and why do they matter?

Introduced in Angular 14 and stabilized by 15/16, standalone components remove the need for NgModules. The component declares its own dependencies via the `imports` array and uses `standalone: true`.

```
@Component({
  selector: 'app-user-card',
  standalone: true,
  imports: [CommonModule, RouterLink],
  template: `<a [routerLink]="['/user', id]">{{ name }}</a>`,
})
export class UserCardComponent {
  @Input() id!: number;
  @Input() name = '';
}
```

Benefits: less boilerplate, simpler mental model, easier lazy loading (`loadComponent`), better tree-shaking. During my Angular 16 migration this was a major modernization step — bootstrapping with `bootstrapApplication()` instead of `AppModule`.

Q: What are decorators in Angular?

Decorators are functions prefixed with `@` that attach metadata to classes, properties, methods, or parameters, telling Angular how to process them.

- **Class decorators:** `@Component`, `@NgModule`, `@Injectable`, `@Directive`, `@Pipe`.
- **Property decorators:** `@Input`, `@Output`, `@ViewChild`, `@ContentChild`, `@HostBinding`.
- **Method decorators:** `@HostListener`.
- **Parameter decorators:** `@Inject`, `@Optional`, `@Self`, `@Host`.

Q: What is the Angular CLI and which commands do you use most?

The Angular CLI scaffolds, builds, tests, and serves Angular apps with best practices baked in.

```

ng new my-app           # create a project
ng generate component x # or ng g c x - scaffold a component
ng g s services/user    # generate a service
ng serve                # dev server with live reload
ng build --configuration production # production build (AOT + optimization)
ng test                 # run unit tests (Karma/Jasmine)
ng update               # migrate dependencies/versions
ng add @angular/material # add a library with schematics

```

Q: Describe a typical Angular project structure.

```

src/
  app/
    components/    # reusable presentational components
    pages/         # routed feature components
    services/      # data/business logic, HTTP
    models/        # interfaces & types
    store/         # NgRx actions/reducers/effects/selectors
    shared/        # shared module, pipes, directives
    app.component.ts
    app.routes.ts (or app-routing.module.ts)
  assets/          # static files
  environments/    # environment.ts / environment.prod.ts
  main.ts          # bootstrap
  styles.scss      # global styles
  angular.json     # workspace/build config

```

Components & Templates

Q: Explain all the Angular lifecycle hooks in order with their use cases.

Hooks run in this order:

1. **ngOnChanges(changes)** — called before **ngOnInit** and whenever an **@Input** changes. Use it to react to input changes; receives a **SimpleChanges** object.
2. **ngOnInit()** — called once after the first **ngOnChanges**. Best place for initialization: fetch data, set up subscriptions, initialize forms. Inputs are available here (not in the constructor).
3. **ngDoCheck()** — runs on every change detection cycle. Use for custom change detection; keep it cheap (runs very often).
4. **ngAfterContentInit()** — once after projected content (**ng-content**) is initialized. **@ContentChild** is available here.
5. **ngAfterContentChecked()** — after every check of projected content.
6. **ngAfterViewInit()** — once after the component's view and child views are initialized. **@ViewChild** is available here (e.g., access a DOM element or child component).
7. **ngAfterViewChecked()** — after every check of the component's view.
8. **ngOnDestroy()** — just before the component is destroyed. Clean up: unsubscribe from observables, detach event handlers, clear timers. Critical for preventing memory leaks.

```
export class ExampleComponent implements OnInit, OnChanges, OnDestroy, AfterViewInit {
  ngOnChanges(changes: SimpleChanges) { /* react to input changes */ }
  ngOnInit() { /* init data, forms, subscriptions */ }
  ngAfterViewInit() { /* access @ViewChild DOM */ }
  ngOnDestroy() { /* unsubscribe, cleanup */ }
}
```

Q: Why initialize in ngOnInit instead of the constructor?

The constructor is a TypeScript/JS feature used only for dependency injection — when it runs, `@Input` properties are not yet set and the component isn't fully wired up. `ngOnInit` runs after Angular initializes the component and binds inputs, so it's the correct place for setup logic, HTTP calls, and form initialization. This also keeps construction cheap and makes testing easier.

Q: What are the types of data binding in Angular?

1. **Interpolation** `{{ value }}` — component → view (text).
2. **Property binding** `[property]="value"` — component → view (DOM property).
3. **Event binding** `(event)="handler()"` — view → component.
4. **Two-way binding** `[(ngModel)]="value"` — both directions (syntactic sugar for `[value]` + `(valueChange)`).

```
<h1>{{ title }}</h1>                                <!-- interpolation -->
<img [src]="imageUrl" />                             <!-- property -->
<button (click)="save()">Save</button>               <!-- event -->
<input [(ngModel)]="name" />                         <!-- two-way -->
```

Q: How does two-way binding actually work under the hood?

`[(ngModel)]="x"` is the "banana in a box" syntax. It expands to a property binding plus an event binding:

```
<input [(ngModel)]="x" [(ngModelChange)]="x = $event" />
```

Any directive/component can support two-way binding if it exposes an `@Input() prop` and a matching `@Output() propChange` EventEmitter.

Q: What are template reference variables?

A template reference variable (declared with `#`) gives you a reference to a DOM element, component, or directive within the template.

```
<input #emailInput type="email" />
<button (click)="log(emailInput.value)">Log</button>
```

It's scoped to the template. For component access in the class, use `@ViewChild`.

Q: What is the difference between ViewChild and ContentChild?

- `@ViewChild` queries elements/components/directives in the component's own template (its view). Available in `ngAfterViewInit`.
- `@ContentChild` queries projected content passed via `ng-content`. Available in `ngAfterContentInit`.

```
@ViewChild('emailInput') emailInput!: ElementRef;
@ViewChild(ChildComponent) child!: ChildComponent;
@ContentChild(HeaderComponent) header!: HeaderComponent;
```

Use `@ViewChildren` / `@ContentChildren` (returning a `QueryList`) for multiple matches.

Q: What is ng-content and content projection?

Content projection lets a component render markup that the parent supplies, making components reusable (like a slot).

`<ng-content>` marks where projected content appears.

```
<!-- card.component.html -->
<div class="card">
  <ng-content select="[card-header]"></ng-content>
  <ng-content></ng-content> <!-- default slot -->
</div>
```

```
<!-- usage -->
<app-card>
  <h2 card-header>Title</h2>
  <p>Body content projected here</p>
</app-card>
```

I used this heavily for custom reusable components — generic cards, modals, and layout wrappers.

Q: What is the difference between ng-template and ng-container?

- `<ng-template>` defines a template that is not rendered until explicitly used (e.g., by `*ngIf` else, or `ngTemplateOutlet`). It's a blueprint.
- `<ng-container>` is a logical, non-rendered grouping element — it groups elements or hosts a structural directive without adding an extra DOM node.

```
<ng-container *ngIf="user; else loading">{{ user.name }}</ng-container>
<ng-template #loading>Loading...</ng-template>

<!-- avoid wrapper div -->
<ng-container *ngFor="let item of items">{{ item }}</ng-container>
```

Q: What is the difference between structural and attribute directives?

- **Structural directives** change the DOM layout by adding/removing elements. Prefixed with `*` (desugars to `<ng-template>`). Examples: `*ngIf`, `*ngFor`, `*ngSwitch`.
- **Attribute directives** change the appearance or behavior of an existing element. Examples: `ngClass`, `ngStyle`, `ngModel`, and custom directives like `[appHighlight]`.

```
@Directive({ selector: '[appHighlight]' })
export class HighlightDirective {
  @HostListener('mouseenter') onEnter() { this.el.nativeElement.style.background = 'yellow'; }
  constructor(private el: ElementRef) {}
}
```

Q: Explain `ngIf`, `ngFor`, and `trackBy`.

`*ngIf` conditionally renders an element (with optional `else`). `*ngFor` iterates over a collection.

```
<div *ngIf="isLoggedIn; else guest">Welcome</div>
<ng-template #guest>Please log in</ng-template>

<li *ngFor="let item of items; let i = index; trackBy: trackById">
  {{ i }} - {{ item.name }}
</li>
```

trackBy tells Angular how to identify items so it reuses DOM nodes instead of re-rendering the whole list when the array changes. This is a key performance optimization for large lists.

```
trackById(index: number, item: Item) { return item.id; }
```

Q: What is the difference between `ngClass` and `ngStyle`?

- `ngClass` adds/removes CSS classes conditionally.
- `ngStyle` sets inline styles conditionally.

```
<div [ngClass]="{ active: isActive, disabled: !isEnabled }">...</div>
<div [ngStyle]="{ color: textColor, 'font-size.px': size }">...</div>
```

Q: In Angular 17+, what are the new control flow blocks?

Angular 17 introduced built-in control flow that replaces structural directives with a cleaner syntax: `@if / @else`, `@for` (with mandatory `track`), and `@switch`. It's faster and doesn't require importing `CommonModule`.

```
@if (user) {
  <p>{{ user.name }}</p>
} @else {
  <p>Loading...</p>
}

@for (item of items; track item.id) {
  <li>{{ item.name }}</li>
}
```

(Worth mentioning even from a 16 background to show awareness of the upgrade path.)

Component Communication

Q: How do parent and child components communicate?

- Parent → Child: `@Input()` properties.
- Child → Parent: `@Output()` with `EventEmitter`.


```
// child
@Input() count = 0;
@Output() countChange = new EventEmitter<number>();
increment() { this.countChange.emit(++this.count); }
```

```
<!-- parent -->
<app-counter [(count)="value" [(countChange)="onChange($event)"]></app-counter>
```

Q: How do sibling components communicate?

Through a shared parent (parent mediates via Input/Output) or, more commonly, through a **shared service** holding state in a `Subject / BehaviorSubject`. Sibling A pushes; sibling B subscribes.

```
@Injectable({ providedIn: 'root' })
export class MessageService {
  private msg = new BehaviorSubject<string>('');
  msg$ = this.msg.asObservable();
  send(m: string) { this.msg.next(m); }
}
```

For complex app-wide state, NgRx is the better answer.

Q: How can a service act as shared state between components?

A singleton service (`providedIn: 'root'`) is shared across the app. Holding a `BehaviorSubject` inside it lets any component read the current value and react to changes via the exposed observable — a lightweight alternative to NgRx for small/medium state.

Services & Dependency Injection

Q: What is dependency injection in Angular?

DI is a design pattern where a class receives its dependencies from an external injector rather than creating them itself. Angular's injector creates and supplies service instances based on type and provider configuration.

```
@Injectable({ providedIn: 'root' })
export class UserService {
  constructor(private http: HttpClient) {}
}

@Component({ /* ... */ })
export class UserComponent {
  constructor(private userService: UserService) {} // injected automatically
}
```

Benefits: loose coupling, testability (swap mocks), and singleton reuse.

Q: Explain `providedIn: 'root'` vs `module` vs `component`.

- `providedIn: 'root'` — single app-wide singleton, tree-shakable (removed if unused). Default and preferred.

- **Provided in a module** (`providers: []` of an `NgModule`) — one instance per that module's injector; with lazy modules you get a separate instance.
- **Provided in a component** (`providers: []` in `@Component`) — a new instance per component instance (and its children). Use when each component needs its own state.

Q: What are hierarchical injectors?

Angular has a tree of injectors mirroring the component tree (plus the root/module injectors). When a dependency is requested, Angular walks up from the component's injector until it finds a provider. This means you can override a service at a lower level and child components get the closer instance. Resolution modifiers: `@Self`, `@SkipSelf`, `@Optional`, `@Host`.

Q: What is a singleton service?

A service with a single shared instance across the application — achieved with `providedIn: 'root'`. All injectors resolve to the same instance, making it ideal for shared state, caching, and cross-component communication.

Q: What is an injection token and when do you use it?

`InjectionToken` provides a DI key for things that aren't classes — interfaces, primitives, or config objects (which have no runtime type to inject by).

```
export const API_URL = new InjectionToken<string>('api.url');

// provide
{ provide: API_URL, useValue: 'https://api.example.com' }

// inject
constructor(@Inject(API_URL) private apiUrl: string) {}
```

Routing

Q: How do you configure routes in Angular?

Define a `Routes` array mapping paths to components, then register it.

```
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'users/:id', component: UserDetailComponent },
  { path: '**', component: NotFoundComponent }, // wildcard
];
// standalone: provideRouter(routes); NgModule: RouterModule.forRoot(routes)
```

The `<router-outlet>` marks where the matched component renders.

Q: What is routerLink and how do you navigate programmatically?

`routerLink` is a directive for declarative navigation in templates. For code-driven navigation use the `Router` service.

```
<a [routerLink]="['/users', id]" routerLinkActive="active">Profile</a>
```

```
constructor(private router: Router) {}
goHome() { this.router.navigate(['/home']); }
```

Q: How do you read route params vs query params?

```
constructor(private route: ActivatedRoute) {}

ngOnInit() {
  // route param: /users/:id
  this.route.paramMap.subscribe(p => this.id = p.get('id'));
  // query param: /search?q=angular
  this.route.queryParamMap.subscribe(q => this.term = q.get('q'));
}
```

Use the observable form when the component is reused across param changes; `snapshot` is fine for one-time reads.

Q: What are child routes?

Nested routes rendered inside a parent component's own `<router-outlet>`. Use the `children` array.

```
{
  path: 'dashboard',
  component: DashboardComponent,
  children: [
    { path: 'stats', component: StatsComponent },
    { path: 'settings', component: SettingsComponent },
  ],
}
```

Q: What are route guards? Explain CanActivate, CanDeactivate, and Resolve.

Guards control navigation.

- `CanActivate` — can the user enter this route? (auth checks).
- `CanActivateChild` — guards child routes.
- `CanDeactivate` — can the user leave? (warn about unsaved form changes).
- `Resolve` — pre-fetch data before the route activates so the component loads with data ready.
- `CanMatch` (modern) — decide whether a route definition matches at all (great for feature flags / lazy loading).

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.isLoggedIn() ? true : router.parseUrl('/login');
};
```

Angular 15+ favors functional guards (`CanActivateFn`) over class-based ones.

Q: What is lazy loading and why use it?

Lazy loading loads a feature module/component only when its route is visited, instead of in the initial bundle. This shrinks the initial bundle, speeds up first load, and is essential for large apps.

```
// module-based
{ path: 'admin', loadChildren: () => import('./admin/admin.module').then(m =>
m.AdminModule) }
// standalone (Angular 14+)
{ path: 'admin', loadComponent: () => import('./admin/admin.component').then(c =>
c.AdminComponent) }
```

Forms

Q: Template-driven vs reactive forms — what's the difference?

- **Template-driven:** logic lives in the template using `ngModel`; Angular builds the model implicitly. Good for simple forms. Asynchronous, harder to unit test.
- **Reactive (model-driven):** the form model is defined in the component class (`FormGroup` , `FormControl`). Explicit, synchronous, type-safe, scalable, and easy to test. Preferred for complex/dynamic forms.

I primarily use reactive forms (and Angular Formly on top for dynamic, config-driven forms).

Q: Explain FormControl, FormGroup, and FormArray.

- **FormControl** — a single input field with value and validation state.
- **FormGroup** — a collection of controls keyed by name (a form or sub-section).
- **FormArray** — a dynamic, indexed list of controls/groups (e.g., add/remove rows).

```
form = new FormGroup({
  name: new FormControl('', Validators.required),
  emails: new FormArray([ new FormControl('') ]),
});
addEmail() { (this.form.get('emails') as FormArray).push(new FormControl('')); }
```

Q: What is FormBuilder?

A service that provides shorthand to build form controls/groups/arrays with less boilerplate.

```
constructor(private fb: FormBuilder) {}
form = this.fb.group({
  name: ['', Validators.required],
  address: this.fb.group({ city: ['', zip: [] ]}),
  hobbies: this.fb.array([]),
});
```

Q: How do validators work — built-in, custom, and async?

Built-in: `Validators.required` , `minLength` , `maxLength` , `pattern` , `email` , `min` , `max` .

Custom (sync) — a function returning an error object or null:

```
function noSpaces(c: AbstractControl): ValidationErrors | null {
  return c.value?.includes(' ') ? { noSpaces: true } : null;
}
new FormControl('', [Validators.required, noSpaces]);
```

Async — returns an Observable/Promise; useful for server checks like username availability:

```
function uniqueUsername(api: ApiService): AsyncValidatorFn {
  return c => api.checkUsername(c.value).pipe(
    map(taken => taken ? { taken: true } : null)
  );
}
new FormControl('', { asyncValidators: [uniqueUsername(api)] });
```

Q: How do you build dynamic forms? (Angular Formly)

Dynamic forms are generated from configuration/metadata rather than hardcoded markup. **Angular Formly** drives the form from a JSON-like field config array, which is ideal when form structure comes from an API or changes frequently.

```
fields: FormlyFieldConfig[] = [
  { key: 'firstName', type: 'input', props: { label: 'First Name', required: true } },
  { key: 'email', type: 'input', props: { label: 'Email', type: 'email' } },
];
```

```
<form [formGroup]="form">
  <formly-form [form]="form" [fields]="fields" [model]="model"></formly-form>
</form>
```

In my projects I used Formly with custom field types and wrappers to render complex, config-driven forms (including Kendo UI controls), which dramatically reduced repetitive template code.

RxJS & Observables

Q: What is the difference between an Observable and a Promise?

Observable	Promise
Emits multiple values over time (stream)	Resolves a single value
Lazy — runs only when subscribed	Eager — runs immediately
Cancellable (unsubscribe)	Not cancellable
Rich operators (map, filter, etc.)	<code>.then</code> / <code>.catch</code> only

Angular's `HttpClient` returns Observables.

Q: What is the difference between Subject, BehaviorSubject, and ReplaySubject?

- Subject** — multicast; subscribers only get values emitted after they subscribe. No initial value.

- **BehaviorSubject** — requires an initial value and emits the latest/current value immediately to new subscribers. Ideal for state.
- **ReplaySubject** — replays a configurable buffer of previous values to new subscribers.
- (**AsyncSubject** — emits only the last value, on completion.)

```
const b = new BehaviorSubject<number>(0);
b.subscribe(v => console.log('A', v)); // A 0
b.next(1);                               // A 1
b.subscribe(v => console.log('B', v)); // B 1 (gets current value)
```

Q: Explain map, filter, switchMap, mergeMap, concatMap, and exhaustMap — when do you use each?

- **map** — transform each emitted value.
- **filter** — let values through only if they pass a predicate.

The "flattening" operators handle inner observables (e.g., HTTP calls triggered by a stream):

- **switchMap** — cancels the previous inner observable when a new value arrives. Best for type-ahead search and routing — you only care about the latest.
- **mergeMap** (flatMap) — runs all inner observables concurrently, no ordering. Use for independent parallel work (e.g., multiple uploads).
- **concatMap** — queues inner observables and runs them one at a time in order. Use when order matters and operations must not overlap (sequential writes).
- **exhaustMap** — ignores new values while the current inner observable is running. Best for preventing duplicate submits (e.g., login button spam).

```
this.searchControl.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(term => this.api.search(term)) // cancel stale searches
).subscribe(results => this.results = results);
```

Q: How do you subscribe and unsubscribe? Why does it matter?

You subscribe to consume values. If you never unsubscribe from long-lived observables, the subscription persists after the component is destroyed → **memory leak** and unexpected callbacks.

Cleanup options:

1. **async pipe** (preferred) — auto-subscribes/unsubscribes in the template.
2. **takeUntil** pattern with **ngOnDestroy**.
3. **takeUntilDestroyed()** (Angular 16+) — ties subscription lifetime to the injection context.
4. Manual **subscription.unsubscribe()** in **ngOnDestroy**.

Q: What is the async pipe and why prefer it?

The **async** pipe subscribes to an observable/promise in the template, returns the latest value, and unsubscribes automatically when the component is destroyed.

```
<div *ngIf="user$ | async as user">{{ user.name }}</div>
```

It removes manual subscription management, prevents leaks, and works well with OnPush change detection.

Q: Explain the takeUntil pattern for preventing memory leaks.

Emit on a destroy `Subject` in `ngOnDestroy`, and pipe `takeUntil(destroy$)` into every subscription so they all complete together.

```
private destroy$ = new Subject<void>();

ngOnInit() {
  this.service.data$.pipe(takeUntil(this.destroy$)).subscribe(d => this.data = d);
}
ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

Angular 16's `takeUntilDestroyed()` is the modern, less boilerplate equivalent.

Q: How do you handle errors in RxJS?

Use `catchError` to intercept and recover (return a fallback or rethrow), and `retry` / `retryWhen` to re-attempt.

```
this.api.getData().pipe(
  retry(2),
  catchError(err => {
    this.toast.error('Failed to load');
    return of([]); // graceful fallback
  })
).subscribe(data => this.data = data);
```

State Management (NgRx)

Q: What is NgRx and what are its core building blocks?

NgRx is a Redux-style, reactive state management library for Angular built on RxJS. It provides a single immutable store as the source of truth, with a strict unidirectional data flow.

Core pieces:

- **Store** — the single immutable state container.
- **Actions** — describe unique events (`{ type, payload }`).
- **Reducers** — pure functions that take current state + action → new state.
- **Selectors** — pure, memoized functions to query slices of state.
- **Effects** — handle side effects (HTTP, async) by listening to actions and dispatching new ones.

```

// action
export const loadUsers = createAction('[Users] Load');
export const loadUsersSuccess = createAction('[Users] Load Success', props<{ users: User[]
}>());

// reducer
const reducer = createReducer(initialState,
  on(loadUsersSuccess, (state, { users }) => ({ ...state, users })));

// selector
export const selectUsers = createSelector(selectUserState, s => s.users);

// effect
loadUsers$ = createEffect(() => this.actions$.pipe(
  ofType(loadUsers),
  switchMap(() => this.api.getUsers().pipe(
    map(users => loadUsersSuccess({ users })),
    catchError(() => of(loadUsersFailure()))
  ))
));

```

Q: What are NgRx effects for?

Effects isolate side effects (HTTP calls, navigation, local storage) from components and reducers. They listen to dispatched actions, perform async work, and dispatch result actions. Reducers stay pure; components just dispatch and select.

Q: What are selectors and why are they memoized?

Selectors are pure functions that derive/compose data from the store. `createSelector` memoizes the result — it only recomputes when its input slices change, avoiding unnecessary recalculation and triggering change detection efficiently.

Q: When do you use NgRx vs a simple service with a BehaviorSubject?

Use **NgRx** when: state is large/shared across many features, there are complex interactions, you need time-travel debugging, predictability, and clear separation of side effects. Use a **service + BehaviorSubject** when: state is simple/local, the team is small, and NgRx's boilerplate isn't justified. Don't add NgRx by default — it's overhead for small apps.

HTTP

Q: How do you use HttpClient?

`HttpClient` provides typed, observable-based HTTP methods. Inject it and call `get/post/put/delete`.


```

@Inject({ providedIn: 'root' })
export class UserService {
  constructor(private http: HttpClient) {}
  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('/api/users');
  }
  createUser(u: User) {
    return this.http.post<User>('/api/users', u);
  }
}

```

Requests are lazy — they fire only on subscription (or via `async` pipe).

Q: What are HTTP interceptors? Give auth and error examples.

Interceptors sit in the HTTP pipeline and can transform every outgoing request and incoming response — ideal for attaching auth tokens, logging, and centralized error handling.

```

// functional interceptor (Angular 15+)
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).getToken();
  const cloned = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;
  return next(cloned);
};

export const errorInterceptor: HttpInterceptorFn = (req, next) =>
  next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) inject(Router).navigate(['/login']);
      return throwError(() => err);
    })
  );

```

Register via `provideHttpClient(withInterceptors([authInterceptor, errorInterceptor]))`.

Q: How do you handle HTTP errors gracefully?

Combine interceptor-level global handling (401/403/500, toasts) with local `catchError` for context-specific fallbacks, plus `retry` for transient failures.

```

this.userService.getUsers().pipe(
  catchError(err => { this.error = 'Could not load users'; return of([]); })
).subscribe(users => this.users = users);

```

Pipes

Q: What are pipes? Name common built-in ones.

Pipes transform displayed values in templates without changing the underlying data. Built-ins: `date`, `uppercase` / `lowercase`, `titlecase`, `currency`, `decimal` / `number`, `percent`, `json`, `slice`, `async`,

keyvalue .

```
<p>{{ today | date:'dd/MM/yyyy' }}</p>
<p>{{ price | currency:'INR' }}</p>
<p>{{ name | uppercase }}</p>
```

Q: What is the difference between pure and impure pipes?

- **Pure** (default) — runs only when the input reference changes. Fast and cache-friendly.
- **Impure** (`pure: false`) — runs on every change detection cycle, even if the reference didn't change. Needed for mutable data (e.g., filtering an array mutated in place) but can hurt performance. `async` pipe is impure by design.

Q: How do you create a custom pipe?

Implement `PipeTransform` and decorate with `@Pipe` .

```
@Pipe({ name: 'truncate', standalone: true })
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit = 20): string {
    return value.length > limit ? value.slice(0, limit) + '...' : value;
  }
}
```

```
{{ longText | truncate:50 }}
```

Performance & Change Detection

Q: What is change detection? Default vs OnPush.

Change detection is how Angular keeps the DOM in sync with component data. By **default**, after any async event (click, HTTP, timer), Angular checks the entire component tree.

OnPush tells Angular to skip a component unless: an `@Input` reference changes, an event originates in the component, or an observable bound via `async` pipe emits. This drastically reduces checks in large trees.

```
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, /* ... */ })
```

OnPush works best with immutable data and the `async` pipe — a key technique for performant reusable components.

Q: What is zone.js and its role in change detection?

zone.js monkey-patches async APIs (events, timers, XHR/fetch, promises) so Angular knows when something might have changed state and can trigger change detection automatically — without manual calls. Angular 16+ introduces **signals** and is moving toward optional zoneless change detection for finer control and performance.

Q: How does trackBy improve performance?

Without `trackBy` , `*ngFor` re-creates all DOM nodes when the array reference changes. With `trackBy` returning a stable identity (like an id), Angular reuses existing DOM nodes for unchanged items and only updates what changed — major win for large/frequently-updated lists.

Q: What is the difference between AOT and JIT compilation?

- **JIT (Just-in-Time)** — compiles templates in the browser at runtime. Used historically in dev. Larger bundle, slower startup, runtime template errors.
- **AOT (Ahead-of-Time)** — compiles during the build. Smaller, faster, more secure (no eval), and catches template errors at build time. AOT is the **default for production** (and since Angular 9+ with Ivy, default for dev too).

Q: What techniques do you use to optimize an Angular app?

- Lazy load feature routes (`loadChildren` / `loadComponent`).
- `OnPush` change detection + immutable data + `async` pipe.
- `trackBy` in lists.
- Unsubscribe / use `async` pipe to avoid leaks.
- Pure pipes over method calls in templates.
- AOT + production build (tree-shaking, minification).
- Bundle analysis (`source-map-explorer`), code splitting, preloading strategies.
- Virtual scrolling (CDK) for large lists.
- Defer non-critical work; debounce expensive streams.

Testing

Q: What tools are used for Angular unit testing?

Jasmine is the testing/assertion framework (`describe` , `it` , `expect` , spies), and **Karma** is the test runner that executes tests in a real browser. The Angular CLI wires both up via `ng test` . (Many teams now switch to **Jest** for speed.)

Q: What is TestBed?

TestBed is Angular's primary testing utility — it configures and creates a testing module to instantiate components/services with their dependencies (and mocks).

```
describe('UserCardComponent', () => {
  let fixture: ComponentFixture<UserCardComponent>;

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [UserCardComponent],           // standalone
      providers: [{ provide: UserService, useValue: mockUserService }],
    });
    fixture = TestBed.createComponent(UserCardComponent);
  });

  it('should create', () => {
    expect(fixture.componentInstance).toBeTruthy();
  });
});
```

Q: How do you test a service that uses HttpClient?

Use `HttpClientTestingModule` and `HttpTestingController` to mock requests and assert on them, avoiding real network calls.

```
TestBed.configureTestingModule({ imports: [HttpClientTestingModule] });
const httpMock = TestBed.inject(HttpTestingController);
service.getUsers().subscribe(users => expect(users.length).toBe(2));
httpMock.expectOne('/api/users').flush([{}], {});
```

Migration & Versioning

Q: How do you upgrade/migrate Angular versions? (Your Angular 16 migration)

The supported path is incremental, one major version at a time, using the CLI and the official update guide.

```
ng update @angular/core @angular/cli
```

`ng update` runs **schematics** that automatically apply code migrations (e.g., updated APIs, renamed symbols, removed deprecations). My approach:

1. Use `update.angular.io` to check breaking changes between versions.
2. Update one major at a time, build and run tests after each.
3. Update third-party libs (Kendo UI, Formly, NgRx, Ionic) to compatible versions.
4. Adopt new features incrementally (standalone components, functional guards/interceptors, `inject()`, `takeUntilDestroyed`).
5. Verify with the unit test suite and manual smoke tests.

Q: What notable changes came across recent Angular versions?

- **Angular 14** — standalone components (preview), typed reactive forms, `inject()` function, functional route guards.
- **Angular 15** — standalone APIs stabilized, functional `HttpClient` providers/interceptors, directive composition API, `NgOptimizedImage`.
- **Angular 16** — **signals** (developer preview), `takeUntilDestroyed`, required inputs, non-destructive hydration for SSR, esbuild-based dev server, self-closing tags.
- **Angular 17** — new control flow (`@if/@for/@switch`), deferrable views (`@defer`), new branding/docs, Vite + esbuild by default, improved SSR.
- **Angular 18/19** — zoneless change detection (experimental), signal-based APIs maturing, stable material 3.

Q: What was your overall role in the Angular 16 migration?

Summary answer: I led/contributed to migrating legacy Angular apps to Angular 16 — upgrading incrementally with `ng update`, resolving breaking changes, updating third-party dependencies (Kendo UI, Formly, NgRx, Ionic), refactoring toward standalone components and modern functional guards/interceptors, fixing failing unit tests, and validating performance with OnPush and proper RxJS cleanup. This improved bundle size, type safety (typed forms), and maintainability.

Quick-Fire Bonus

Q: What is the difference between declarations, imports, and providers in NgModule?

`declarations` = components/directives/pipes that belong to this module; `imports` = other modules whose exported features this module uses; `providers` = services registered with the injector.

Q: What is the difference between `forRoot` and `forChild`?

`RouterModule.forRoot(routes)` is called once in the root module and sets up the router service + root routes. `forChild(routes)` is used in feature/lazy modules and registers additional routes without re-creating the router service.

Q: What is the `inject()` function?

A modern alternative to constructor injection that retrieves a dependency from the current injection context — usable in functional guards, interceptors, field initializers, and factories.

```
private userService = inject(UserService);
```

Q: What is the purpose of environment files?

`environment.ts` / `environment.prod.ts` hold per-build configuration (API URLs, feature flags). The CLI swaps them via `fileReplacements` during production builds.

Q: How does Ionic relate to Angular in your experience?

Ionic is a UI framework for building cross-platform mobile/PWA apps using Angular (its components, routing, and lifecycle integrate with Angular). I built Angular-based Ionic apps using Ionic UI components, lifecycle events (`ionViewWillEnter`, etc.), and Capacitor for native features — reusing Angular services, reactive forms, and NgRx across web and mobile.

Core Java — Interview Q&A

A focused set of Core Java questions and answers for a Wipro lateral (4+ years) backend / full-stack role. Answers are concise and oriented toward day-to-day Spring Boot development.

OOP Concepts

Q: What are the four pillars of OOP? Give a short example of each.

Encapsulation, Abstraction, Inheritance, Polymorphism.

- **Encapsulation** — bundling data with the methods that operate on it, hiding internal state via private fields and public getters/setters.
- **Abstraction** — exposing *what* an object does while hiding *how* (interfaces/abstract classes).
- **Inheritance** — a subclass reuses and extends a parent type.
- **Polymorphism** — one interface, many implementations (overriding / overloading).

```
// Encapsulation
class Account {
    private double balance;           // hidden state
    public double getBalance() { return balance; }
    public void deposit(double amt) { // controlled mutation
        if (amt > 0) balance += amt;
    }
}

// Abstraction
interface Payment { void pay(double amount); }

// Inheritance + Polymorphism
class UpiPayment implements Payment {
    public void pay(double amount) { /* UPI logic */ }
}
class CardPayment implements Payment {
    public void pay(double amount) { /* card logic */ }
}

Payment p = new UpiPayment(); // polymorphic reference
p.pay(500);
```

Q: What is the difference between abstraction and encapsulation?

- **Abstraction** is about *design* — hiding complexity and exposing only essential behaviour. Achieved with interfaces and abstract classes.
- **Encapsulation** is about *implementation* — hiding the internal data/state by restricting access (private fields + accessors).

In short: abstraction hides **complexity**, encapsulation hides **data**. Abstraction is solved at the design level; encapsulation at the data-protection level.

Q: Abstract class vs interface — when do you use which?

Aspect	Abstract class	Interface
Multiple inheritance	No (single)	Yes (multiple)
Fields	Instance fields allowed	Only <code>public static final</code> constants
Constructors	Yes	No
Methods	Concrete + abstract	Abstract + <code>default</code> + <code>static</code> (Java 8+), <code>private</code> (Java 9+)
Access modifiers	Any	Methods implicitly <code>public</code>

Use an **abstract class** when you have shared state/common code and a clear "is-a" hierarchy. Use an **interface** when you only define a contract or need a type to mix in multiple behaviours.

```
abstract class Shape {  
    String name;                // shared state  
    abstract double area();     // contract  
    void describe() { System.out.println(name + " area=" + area()); }  
}  
  
interface Drawable { void draw(); }    // pure contract
```

Q: What are default and static methods in interfaces?

Introduced in Java 8 so interfaces could evolve without breaking implementers.

- **default** — a method with a body that implementing classes inherit (and may override).
- **static** — a utility method that belongs to the interface itself, called as `Interface.method()`.

```
interface Calculator {  
    int apply(int a, int b);  
  
    default int applyTwice(int a, int b) {    // inherited, overridable  
        return apply(apply(a, b), b);  
    }  
  
    static Calculator adder() {              // factory utility  
        return (a, b) -> a + b;  
    }  
}
```

If a class implements two interfaces with the same default method, it must override it to resolve the conflict (the "diamond" problem).

Q: Overloading vs overriding?

- **Overloading** — same method name, different parameter list, in the *same* class. Resolved at **compile time** (static binding). Return type alone cannot distinguish overloads.

- **Overriding** — subclass redefines a superclass method with the *same* signature. Resolved at **runtime** (dynamic dispatch). Use `@Override`.

```
class Printer {  
    void print(int x) {}           // overload  
    void print(String x) {}       // overload  
}  
  
class Base { void run() {} }  
class Derived extends Base {  
    @Override void run() {}       // override  
}
```

Overriding rules: same signature, return type covariant or same, access not more restrictive, cannot throw broader checked exceptions.

Q: What do `super` and `this` mean?

- **this** — reference to the current object; used to access fields/methods and to call another constructor (`this(...)`) in the same class.
- **super** — reference to the parent class; used to call the parent constructor (`super(...)`) or access an overridden parent method/field.

```
class Vehicle {  
    int wheels;  
    Vehicle(int w) { this.wheels = w; }  
}  
class Car extends Vehicle {  
    String model;  
    Car(String model) {  
        super(4);           // parent constructor  
        this.model = model;  // current field  
    }  
}
```

Q: Explain constructors and constructor chaining.

A **constructor** initializes a new object; it has the class name and no return type. If you write none, the compiler adds a default no-arg constructor.

Chaining is invoking one constructor from another:

- `this(...)` — call another constructor in the **same** class.
- `super(...)` — call a **parent** constructor (must be the first statement).


```

class User {
    String name;
    int age;
    User() { this("Unknown", 0); }           // this-chaining
    User(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

```

The constructor call always travels up to **Object** first, so parent state is initialized before the child.

Q: Static vs instance members?

- **static** — belong to the class, shared across all objects, loaded once. Accessed via the class name. Cannot use **this** or directly access instance members.
- **instance** — belong to each object, one copy per instance.

```

class Counter {
    static int total;           // shared
    int id;                     // per object

    Counter() { id = ++total; }
}

```

static blocks run once at class-load; instance blocks run before each constructor body.

Q: Difference between final, finally, and finalize?

- **final** — a keyword/modifier: a **final** variable is a constant, a **final** method can't be overridden, a **final** class can't be extended.
- **finally** — a block that always executes after try/catch, used for cleanup.
- **finalize()** — a method (deprecated since Java 9) the GC may call before reclaiming an object. Unreliable — never depend on it; use try-with-resources/**Cleaner** instead.

```

final int MAX = 100;           // final

try { risky(); }
catch (Exception e) { log(e); }
finally { closeResources(); }  // finally

```

Core Java

Q: JDK vs JRE vs JVM?

- **JVM** (Java Virtual Machine) — the abstract engine that loads, verifies, and executes bytecode; provides platform independence.
- **JRE** (Java Runtime Environment) — JVM + core libraries needed to *run* Java apps.

- **JDK** (Java Development Kit) — JRE + development tools (`javac` , `jar` , `javadoc` , debugger) needed to *build* Java apps.

Containment: **JDK** $\supset$ **JRE** $\supset$ **JVM**.

Q: How does Java achieve platform independence?

Source `.java` is compiled by `javac` into **bytecode** (`.class`), which is platform-neutral. Each OS has its own JVM that interprets/JIT-compiles that same bytecode to native instructions. Hence "**write once, run anywhere**" — the bytecode is portable; only the JVM is platform-specific.

Q: String vs StringBuilder vs StringBuffer?

	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Performance	Slow for concat	Fastest	Slower than builder

Use **String** for constants, **StringBuilder** for single-threaded concatenation in loops, **StringBuffer** only when multiple threads mutate the same buffer.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) sb.append(i);
String result = sb.toString();    // "01234"
```

Q: Why are Strings immutable and what is the string pool?

Strings are immutable so they can be safely shared, cached, used as HashMap keys, and remain secure (e.g., in class loading, network connections). Once created, a String's value never changes — any "modification" creates a new object.

The **string pool** is a special area in the heap where string literals are interned and reused. Two identical literals point to the same pooled object; `new String("x")` forces a separate heap object.

```
String a = "hello";
String b = "hello";
System.out.println(a == b);           // true  (same pool object)

String c = new String("hello");
System.out.println(a == c);           // false (new heap object)
System.out.println(a == c.intern());  // true  (interned to pool)
```

Q: == vs equals()?

- `==` compares **references** (whether two variables point to the same object) for objects, or raw values for primitives.
- `equals()` compares **logical equality** as defined by the class. `Object.equals` defaults to `==` , but classes like `String` and the wrappers override it for value comparison.

```
Integer x = 1000, y = 1000;
System.out.println(x == y);           // false (different objects)
System.out.println(x.equals(y));      // true (same value)
```

Q: Explain the equals() and hashCode() contract.

1. If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` **must** be true.
2. Equal hashCodes do **not** require equal objects (collisions are allowed).
3. Both must be consistent across multiple calls.

If you override `equals`, you must override `hashCode`, otherwise hash-based collections (HashMap, HashSet) break.

```
class Point {
    int x, y;
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }
    @Override public int hashCode() { return Objects.hash(x, y); }
}
```

Q: What is autoboxing and unboxing?

Automatic conversion between primitives and their wrapper objects.

- Autoboxing — `int` → `Integer`.
- Unboxing — `Integer` → `int`.

```
List<Integer> nums = new ArrayList<>();
nums.add(5);           // autobox int -> Integer
int n = nums.get(0);   // unbox Integer -> int
```

Watch out: unboxing a `null` wrapper throws `NullPointerException`, and the `Integer` cache (-128..127) can make `==` behave unexpectedly.

Q: Is Java pass-by-value or pass-by-reference?

Java is **always pass-by-value**. For objects, the *value of the reference* (the pointer) is copied. So you can mutate the object's internal state through the copied reference, but reassigning the parameter does not affect the caller's variable.

```
void mutate(StringBuilder sb) { sb.append("X"); } // affects caller's object
void reassign(StringBuilder sb) { sb = new StringBuilder("new"); } // no effect outside
```

Q: What are wrapper classes and why are they needed?

Wrapper classes (`Integer`, `Long`, `Double`, `Boolean`, `Character`, etc.) wrap primitives in objects. Needed because generics and collections work only with objects (`List<Integer>`, not `List<int>`), and wrappers provide utility

methods (`Integer.parseInt` , `Integer.MAX_VALUE`) and allow `null` . They are immutable and cached for small values.

Collections

Q: Describe the Collections framework hierarchy.

`Iterable` → `Collection` splits into **List**, **Set**, **Queue**. `Map` is a separate root (not a `Collection`).

- **List:** `ArrayList` , `LinkedList` , `Vector`
- **Set:** `HashSet` , `LinkedHashSet` , `TreeSet`
- **Queue/Deque:** `PriorityQueue` , `ArrayDeque` , `LinkedList`
- **Map:** `HashMap` , `LinkedHashMap` , `TreeMap` , `Hashtable` , `ConcurrentHashMap`

Q: List vs Set vs Map?

- **List** — ordered, indexed, allows duplicates.
- **Set** — no duplicates; may be unordered (`HashSet`) or ordered (`TreeSet`/`LinkedHashSet`).
- **Map** — key→value pairs, unique keys.

```
List<String> list = new ArrayList<>(); // [a, a, b] allowed
Set<String> set = new HashSet<>(); // {a, b}
Map<String,Integer> map = new HashMap<>(); // {a=1, b=2}
```

Q: ArrayList vs LinkedList?

	ArrayList	LinkedList
Backing	Dynamic array	Doubly linked list
Random access (get)	O(1)	O(n)
Insert/delete at ends	Amortized O(1) / O(n) middle	O(1) at ends
Memory	Less overhead	Extra node pointers

Use **ArrayList** for read-heavy/index access (the common default). Use **LinkedList** only for frequent add/remove at the ends or as a Deque/Queue.

Q: How does HashMap work internally?

- It stores entries in an array of **buckets**. The key's `hashCode()` is spread (`hash = h ^ (h >>> 16)`) and `& (n-1)` gives the bucket index.
- On collision, entries form a **linked list** in the bucket. Since Java 8, when a bucket exceeds **8** entries (and capacity ≥ 64) it **treeifies** into a red-black tree → O(log n) lookups instead of O(n).
- `get` / `put` use `hashCode()` to find the bucket, then `equals()` to find the exact key.

- Default capacity **16**, load factor **0.75**; exceeding `capacity * loadFactor` triggers a **resize** (doubling + rehash).

```
Map<String,Integer> m = new HashMap<>();
m.put("a", 1);           // bucket = hash("a") & 15
m.get("a");              // hash -> bucket -> equals match
```

Q: HashMap vs Hashtable vs ConcurrentHashMap?

	HashMap	Hashtable	ConcurrentHashMap
Thread-safe	No	Yes (method-level sync)	Yes (fine-grained)
Null key/values	1 null key, many null values	None	None
Performance	Fastest single-thread	Slow (whole-map lock)	Best concurrent
Legacy	No	Yes (avoid)	No

ConcurrentHashMap locks only segments/bins (CAS + bin-level synchronization), so reads are mostly lock-free and writes don't block the whole map. Prefer it over **Hashtable** for concurrency.

Q: HashSet vs TreeSet?

- **HashSet** — backed by a HashMap; O(1) operations; **no ordering**; allows one `null`.
- **TreeSet** — backed by a red-black tree (**NavigableSet**); O(log n); **sorted** by natural order or a **Comparator** ; no `null` ; offers `first()` , `last()` , `ceiling()` , `floor()` .

Use HashSet for fast membership checks, TreeSet when you need sorted/range queries.

Q: Comparable vs Comparator?

- **Comparable** — natural ordering, implemented by the class itself via `compareTo()` . One sort order.
- **Comparator** — external ordering, defined separately via `compare()` . Multiple sort orders, doesn't modify the class.

```
class Emp implements Comparable<Emp> {
    int salary; String name;
    public int compareTo(Emp o) { return Integer.compare(salary, o.salary); } // natural
}

// Comparator: sort by name, then salary
Comparator<Emp> byName = Comparator.comparing((Emp e) -> e.name)
                                   .thenComparingInt(e -> e.salary);

list.sort(byName);
```

Q: Fail-fast vs fail-safe iterators?

- **Fail-fast** — throw **ConcurrentModificationException** if the collection is structurally modified during iteration (uses a `modCount`). Examples: **ArrayList** , **HashMap** . They iterate over the original.

- **Fail-safe** — iterate over a **copy/snapshot**, so they don't throw on modification (but may not reflect recent changes).

Examples: `CopyOnWriteArrayList`, `ConcurrentHashMap`.

```
for (String s : list) {  
    list.remove(s);           // ConcurrentModificationException (fail-fast)  
}  
// Use iterator.remove() or a concurrent collection instead.
```

Exceptions

Q: Checked vs unchecked exceptions?

- **Checked** — extend `Exception` (not `RuntimeException`); checked at compile time; must be caught or declared with `throws`. E.g., `IOException`, `SQLException`.
- **Unchecked** — extend `RuntimeException`; not enforced by the compiler; usually programming bugs. E.g., `NullPointerException`, `IllegalArgumentException`.
- **Errors** — `OutOfMemoryError`, `StackOverflowError`; serious, not meant to be caught.

Q: throw vs throws?

- `throw` — actually throws an exception instance (a statement).
- `throws` — declares in a method signature that it may throw exceptions (delegates handling to the caller).

```
void validate(int age) throws IllegalAccessException { // throws (declaration)  
    if (age < 18) throw new IllegalAccessException("Minor"); // throw (action)  
}
```

Q: What is try-with-resources?

A construct (Java 7+) that auto-closes resources implementing `AutoCloseable`, eliminating manual `finally` close. Resources close in reverse order of declaration, even on exception.

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {  
    return br.readLine();  
} // br.close() called automatically
```

This avoids resource leaks and suppressed-exception bugs common with manual `finally`.

Q: How do you create a custom exception?

Extend `Exception` (checked) or `RuntimeException` (unchecked). In Spring Boot, custom unchecked exceptions are common and mapped to HTTP responses via `@ControllerAdvice` / `@ExceptionHandler`.

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String msg) { super(msg); }
}

@ResponseStatus(HttpStatus.NOT_FOUND)
@ExceptionHandler(ResourceNotFoundException.class)
public ErrorResponse handle(ResourceNotFoundException ex) {
    return new ErrorResponse(ex.getMessage());
}
```

Q: Describe the exception hierarchy.

`Throwable` is the root, split into:

- **Error** — JVM-level, unrecoverable (`OutOfMemoryError` , `StackOverflowError`).
- **Exception** — application-level.
- **Checked** — direct subclasses of `Exception` (e.g., `IOException`).
- **Unchecked** — `RuntimeException` and its subclasses (e.g., `NullPointerException`).

Q: What happens with finally and a return statement?

`finally` always runs, even if `try` / `catch` has a `return` . If `finally` itself returns, it **overrides** the value from `try/catch` (an anti-pattern — avoid returning from `finally`).

```
int test() {
    try { return 1; }
    finally { return 2; } // returns 2, swallows the 1
}
```

`finally` does not run only for `System.exit()` or JVM crash.

Java 8+ Features

Q: What are lambda expressions?

Anonymous functions that provide a concise implementation of a functional interface (single abstract method). They reduce boilerplate of anonymous classes.

```
Runnable r = () -> System.out.println("run");
Comparator<Integer> c = (a, b) -> a - b;
list.forEach(item -> System.out.println(item));
```

Q: What are the built-in functional interfaces (Predicate/Function/Consumer/Supplier)?

From `java.util.function` :

- **Predicate**\<T> — `boolean test(T)` — a condition.
- **Function**\<T,R> — `R apply(T)` — transforms input to output.

- **Consumer**\<T> — `void accept(T)` — consumes, no return.
- **Supplier**\<T> — `T get()` — supplies a value, no input.

```
Predicate<Integer> isEven = n -> n % 2 == 0;
Function<String,Integer> len = String::length;
Consumer<String> printer = System.out::println;
Supplier<LocalDate> today = LocalDate::now;
```

Also `BiFunction`, `UnaryOperator`, `BinaryOperator`.

Q: Explain the Stream API with filter/map/collect/reduce examples.

A Stream is a pipeline for processing collections functionally (not a data structure). Operations are **lazy** until a terminal operation runs.

```
List<Employee> emps = ...;

// filter + map + collect
List<String> names = emps.stream()
    .filter(e -> e.getSalary() > 50000)
    .map(Employee::getName)
    .collect(Collectors.toList());

// reduce - total salary
double total = emps.stream()
    .map(Employee::getSalary)
    .reduce(0.0, Double::sum);

// grouping
Map<String, List<Employee>> byDept = emps.stream()
    .collect(Collectors.groupingBy(Employee::getDept));
```

Q: What are method references?

Shorthand for lambdas that just call an existing method. Four kinds:

```
Function<String,Integer> a = Integer::parseInt;           // static
Supplier<String> b = str::toUpperCase;                   // instance of particular object
Function<String,Integer> c = String::length;             // instance of arbitrary object
Supplier<ArrayList<String>> d = ArrayList::new;           // constructor
```

Q: What is Optional and why use it?

`Optional<T>` is a container that may or may not hold a value, designed to avoid `NullPointerException` and make "absence" explicit in APIs.


```
Optional<User> user = repo.findById(id);

String name = user.map(User::getName)
                  .orElse("Guest");

user.ifPresent(u -> log(u));

User u = repo.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Not found"));
```

Prefer `orElseGet` / `orElseThrow` / `map` ; avoid `get()` without a presence check. Don't use `Optional` for fields or parameters.

Q: Why were default methods added in Java 8?

To allow interfaces to add new methods **without breaking** existing implementations — critical for evolving the JDK (e.g., adding `stream()` , `forEach()` to `Collection`). Implementers inherit the default behaviour automatically.

Q: Intermediate vs terminal stream operations?

- **Intermediate** — return a new Stream, are **lazy**, chainable: `filter` , `map` , `sorted` , `distinct` , `limit` , `peek` .
- **Terminal** — trigger execution and produce a result/side-effect, **consume** the stream: `collect` , `forEach` , `reduce` , `count` , `findFirst` , `anyMatch` .

Without a terminal operation, no intermediate operation executes (lazy evaluation). A stream cannot be reused after a terminal op.

Multithreading

Q: Thread vs Runnable — which is better?

`Runnable` is preferred because:

- Java allows only single class inheritance — extending `Thread` blocks extending anything else.
- `Runnable` separates the task from the thread mechanism and works with thread pools / `ExecutorService` .

```
Runnable task = () -> System.out.println("running");
new Thread(task).start();    // preferred

class MyThread extends Thread {    // less flexible
    public void run() { /* ... */ }
}
```

Q: What is the thread lifecycle?

NEW → **RUNNABLE** (ready/running) → **RUNNING**; can go to **BLOCKED** (waiting for a monitor lock), **WAITING** (`wait()` , `join()`), or **TIMED_WAITING** (`sleep(ms)` , `wait(ms)`), then back to **RUNNABLE**, finally **TERMINATED** when `run()` completes.

Q: What does synchronized do?

Provides mutual exclusion and visibility: only one thread can hold an object's monitor lock at a time, preventing race conditions. Can be applied to a method or a block.

```
class Counter {
    private int count;
    public synchronized void increment() { count++; }    // method-level

    private final Object lock = new Object();
    public void dec() {
        synchronized (lock) { count--; }                // block-level
    }
}
```

Block-level locking is preferred for finer granularity.

Q: What is the volatile keyword?

volatile guarantees **visibility** — reads/writes go directly to main memory, so changes by one thread are immediately seen by others. It prevents instruction reordering for that variable. It does **not** provide atomicity for compound operations (`count++` is still unsafe).

```
private volatile boolean running = true;    // visible across threads
public void stop() { running = false; }
```

Use **volatile** for simple flags; use **AtomicInteger** /locks for compound updates.

Q: Explain wait() and notify().

Inter-thread communication methods on **Object**, used inside a **synchronized** block on that object's monitor.

- **wait()** — releases the lock and suspends the thread until notified.
- **notify()** / **notifyAll()** — wake one / all waiting threads.

```
synchronized (queue) {
    while (queue.isEmpty()) queue.wait();    // always wait in a loop
    process(queue.poll());
}
// producer:
synchronized (queue) { queue.add(item); queue.notifyAll(); }
```

Always call **wait()** in a loop to guard against spurious wakeups.

Q: What is a deadlock and how do you avoid it?

A deadlock occurs when two or more threads each hold a lock the other needs, so all wait forever.

```
// Thread 1: lock A then B   | Thread 2: lock B then A   -> deadlock
```

Avoid by: acquiring locks in a **consistent global order**, using `tryLock()` with timeouts, reducing lock scope, or using higher-level concurrency utilities (`java.util.concurrent`).

Q: What is ExecutorService and why use thread pools?

`ExecutorService` manages a pool of reusable threads, decoupling task submission from execution. Thread pools avoid the cost of creating threads per task and cap concurrency.

```
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit(() -> compute());
Integer result = f.get();           // blocks for result
pool.shutdown();
```

Common factories: `newFixedThreadPool` , `newCachedThreadPool` , `newSingleThreadExecutor` , `newScheduledThreadPool` . For control, configure `ThreadPoolExecutor` directly.

Q: Callable vs Runnable?

	Runnable	Callable<V>
Method	<code>void run()</code>	<code>V call()</code>
Return value	No	Yes
Checked exceptions	Cannot throw	Can throw
Used with	<code>Thread</code> , executor	<code>executor.submit()</code> → <code>Future</code>

```
Callable<Integer> task = () -> 42;
Future<Integer> future = pool.submit(task);
int value = future.get();
```

Q: synchronized vs Lock (ReentrantLock)?

	synchronized	Lock
Acquire/release	Implicit (block scope)	Explicit <code>lock()</code> / <code>unlock()</code>
Try / timeout	No	<code>tryLock(timeout)</code>
Interruptible	No	<code>lockInterruptibly()</code>
Fairness	No	Configurable
Read/write split	No	<code>ReadWriteLock</code>

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try { /* critical section */ }
finally { lock.unlock(); } // always unlock in finally
```

`Lock` is more flexible; `synchronized` is simpler and sufficient for most cases.

Memory & Garbage Collection

Q: Stack vs heap memory?

- **Stack** — per-thread; stores method frames, local primitives, and object **references**. LIFO, fast, auto-freed when a method returns. Overflow → `StackOverflowError`.
- **Heap** — shared across threads; stores all **objects** and instance fields. Managed by the garbage collector. Exhaustion → `OutOfMemoryError`.

```
void m() {
    int x = 5;           // x on stack
    Person p = new Person(); // reference on stack, object on heap
}
```

Q: How does garbage collection work?

The GC automatically reclaims heap memory occupied by objects that are no longer **reachable** from GC roots (stack references, static fields, etc.).

- The heap is generational: **Young** (Eden + 2 Survivor spaces) and **Old/Tenured**.
 - Most objects die young → **Minor GC** cleans the Young gen; surviving objects are promoted to Old gen → **Major/Full GC**.
 - Uses mark-and-sweep (and compaction). Modern collectors: **G1** (default), **ZGC**, **Shenandoah** for low pauses.
 - You cannot force GC; `System.gc()` is only a hint. Prevent leaks by clearing references and using try-with-resources.
-

Spring & Spring Boot — Interview Q&A

Spring Core

Q: What is the Spring Framework?

Spring is a lightweight, open-source Java framework for building enterprise applications. Its core feature is **Inversion of Control (IoC)**: the framework manages object creation and wiring instead of your code doing it manually. It provides modules for DI, AOP, transactions, data access (JDBC/JPA), MVC, security, and more, letting you write loosely coupled, testable code.

Key benefits: loose coupling, testability (easy mocking), declarative transactions/security, and a huge ecosystem.

Q: What is IoC (Inversion of Control) and DI (Dependency Injection)?

IoC means control of object lifecycle and dependency wiring is inverted — handed to the Spring container instead of being done in your code with `new`.

DI is the pattern Spring uses to implement IoC: the container *injects* an object's dependencies rather than the object creating them.

```
// Without DI – tight coupling
class OrderService {
    private PaymentService payment = new PaymentService(); // hard-coded
}

// With DI – container injects it
@Service
class OrderService {
    private final PaymentService payment;
    OrderService(PaymentService payment) { this.payment = payment; } // injected
}
```

Result: `OrderService` doesn't know how `PaymentService` is built, so it's easy to swap/mock.

Q: ApplicationContext vs BeanFactory?

Both are IoC containers, but:

BeanFactory	ApplicationContext
Basic container, lazy init by default	Superset of BeanFactory
No enterprise features	Eager singleton init, event publishing, i18n, AOP, annotation support
Rarely used directly	Standard choice in real apps

`ApplicationContext` is what you use in practice (Spring Boot creates one for you). `BeanFactory` is the lower-level foundation.

Q: What are the bean scopes in Spring?

- **singleton** (default) — one shared instance per container.
- **prototype** — new instance every time the bean is requested.
- **request** — one instance per HTTP request (web only).
- **session** — one instance per HTTP session (web only).
- **application** — one per ServletContext.
- **websocket** — one per WebSocket session.

```
@Service
@Scope("prototype")
public class ReportBuilder { }
```

Q: Explain the Spring bean lifecycle.

1. Container instantiates the bean.
2. Populates dependencies (DI).
3. Aware interfaces called (`BeanNameAware` , `ApplicationContextAware`).
4. `@PostConstruct` / `InitializingBean.afterPropertiesSet()` / custom init.
5. Bean is ready to use.
6. On shutdown: `@PreDestroy` / `DisposableBean.destroy()` / custom destroy.

```
@Component
public class CacheManager {
    @PostConstruct
    public void init() { /* warm up cache */ }
    @PreDestroy
    public void cleanup() { /* flush cache */ }
}
```

Q: @Component vs @Service vs @Repository vs @Controller?

All are stereotype annotations — specializations of `@Component` that mark a class as a Spring-managed bean. They differ in *intent* (and some add behavior):

- **@Component** — generic Spring bean.
- **@Service** — business/service layer (semantic only).
- **@Repository** — DAO layer; adds **exception translation** (converts JDBC/JPA exceptions to Spring's `DataAccessException`).
- **@Controller** — web MVC controller (returns views); `@RestController` = `@Controller` + `@ResponseBody` .

Use the most specific one for readability and to get layer-specific behavior.

Q: @Autowired — field vs constructor vs setter injection? Which is best and why?

- **Field injection** — `@Autowired` on the field. Concise but hard to test, hides dependencies, can't be `final` .
- **Setter injection** — good for optional dependencies.
- **Constructor injection** — *recommended*. Dependencies are explicit, can be `final` (immutable), guarantees the bean is fully initialized, and makes unit testing trivial (just pass mocks to the constructor). Since Spring 4.3 `@Autowired` is optional on a single constructor.

```

@Service
public class OrderService {
    private final PaymentService payment;
    private final InventoryService inventory;

    // constructor injection – no @Autowired needed for single ctor
    public OrderService(PaymentService payment, InventoryService inventory) {
        this.payment = payment;
        this.inventory = inventory;
    }
}

```

Q: What is @Qualifier?

When multiple beans of the same type exist, `@Autowired` is ambiguous. `@Qualifier` names the exact bean to inject.

```

@Service
public class NotificationService {
    private final MessageSender sender;
    public NotificationService(@Qualifier("smsSender") MessageSender sender) {
        this.sender = sender;
    }
}

```

Q: What is @Primary?

Marks one bean as the default when multiple candidates of a type exist, so you don't need `@Qualifier` everywhere.

```

@Bean
@Primary
public MessageSender emailSender() { return new EmailSender(); }

@Bean
public MessageSender smsSender() { return new SmsSender(); }
// emailSender is injected by default; use @Qualifier("smsSender") for the other

```

Q: @Configuration and @Bean?

`@Configuration` marks a class as a source of bean definitions. `@Bean` methods explicitly create beans — useful for third-party classes you can't annotate with `@Component`.

```

@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

```

Q: What is AOP (Aspect-Oriented Programming)?

AOP lets you modularize **cross-cutting concerns** (logging, security, transactions, metrics) that span many classes, keeping them out of business logic.

Key terms: **Aspect** (the module), **Advice** (action: `@Before`, `@After`, `@Around`), **Pointcut** (where it applies), **JoinPoint** (a point in execution).

```
@Aspect
@Component
public class LoggingAspect {
    @Around("execution(* com.app.service.*(..))")
    public Object logTime(ProceedingJoinPoint jp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = jp.proceed();
        log.info("{} took {} ms", jp.getSignature(), System.currentTimeMillis() - start);
        return result;
    }
}
```

`@Transactional` itself is implemented with AOP.

Spring Boot

Q: What is Spring Boot and what are its advantages over Spring?

Spring Boot is an opinionated layer on top of Spring that removes boilerplate configuration and lets you build production-ready, stand-alone apps quickly.

Advantages:

- **Auto-configuration** — sensible defaults based on the classpath.
- **Starters** — curated dependency bundles (no version juggling).
- **Embedded server** — runs as a `java -jar`, no external Tomcat.
- **No XML** — convention over configuration.
- **Actuator** — built-in production monitoring/health.

Q: How does auto-configuration work?

Driven by `@EnableAutoConfiguration` (inside `@SpringBootApplication`). At startup Spring Boot scans `META-INF/spring/...AutoConfiguration.imports` for auto-config classes, then applies them conditionally using `@Conditional` annotations:

- `@ConditionalOnClass` — a class is on the classpath.
- `@ConditionalOnMissingBean` — you haven't defined your own.
- `@ConditionalOnProperty` — a property is set.

Example: if `spring-boot-starter-data-jpa` and an H2 driver are on the classpath and you defined no `DataSource`, Boot auto-configures one. Your own beans always win, so you can override defaults.

Q: What are Spring Boot starters?

Starters are convenient dependency descriptors that pull in a coherent set of libraries for a feature, with compatible versions managed for you.

Examples: `spring-boot-starter-web` (REST + embedded Tomcat + Jackson), `spring-boot-starter-data-jpa`, `spring-boot-starter-security`, `spring-boot-starter-webflux`, `spring-boot-starter-test`.

Q: What does @SpringBootApplication consist of?

It's a meta-annotation combining three:

- **@Configuration** — class can define beans.
- **@EnableAutoConfiguration** — turns on auto-configuration.
- **@ComponentScan** — scans the current package and sub-packages for components.

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

Q: application.properties vs application.yml?

Both configure the app externally; just different formats. YAML is hierarchical and cleaner for nested config; properties is flat key-value. If both exist, properties take precedence for the same key.

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/shop
```

```
server:
  port: 8081
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/shop
```

Q: What are Spring profiles?

Profiles let you define environment-specific configuration (dev, test, prod). Use `application-{profile}.properties` and activate with `spring.profiles.active`.

```
# application-prod.properties
spring.datasource.url=jdbc:postgresql://prod-db:5432/shop
```

```
@Service
@Profile("dev")
public class MockPaymentService implements PaymentService { }
```

Activate: `--spring.profiles.active=prod` or via env var.

Q: How does the embedded server work and how do you change it?

Boot embeds a servlet container (Tomcat by default) so the app runs as an executable jar. To switch to Jetty or Undertow, exclude Tomcat from the web starter and add the other starter.

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions><exclusion>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
  </exclusion></exclusions>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>

```

Q: How do you change the server port?

```
server.port=8082
```

Or `server.port=0` for a random free port, or via command line: `java -jar app.jar --server.port=9090`.

Q: What is Spring Boot Actuator?

Actuator exposes production-ready endpoints for monitoring and management over HTTP/JMX: `/actuator/health`, `/actuator/metrics`, `/actuator/info`, `/actuator/env`, `/actuator/loggers`.

```
management.endpoints.web.exposure.include=health,info,metrics
```

Add `spring-boot-starter-actuator`. Useful with Prometheus/Grafana and load-balancer health checks.

Q: What is CommandLineRunner?

A functional interface whose `run()` executes once after the application context is ready — handy for startup tasks (seeding data, warming caches). `ApplicationRunner` is similar but gives parsed `ApplicationArguments`.

```

@Component
public class DataSeeder implements CommandLineRunner {
    private final UserRepository repo;
    public DataSeeder(UserRepository repo) { this.repo = repo; }

    @Override
    public void run(String... args) {
        repo.save(new User("admin"));
    }
}

```

REST APIs

Q: @RestController vs @Controller?

`@Controller` is for traditional MVC returning view names; methods need `@ResponseBody` to return data.

`@RestController` = `@Controller` + `@ResponseBody`, so every method returns serialized data (JSON) directly in the

response body — ideal for REST APIs.

Q: What are the request mapping annotations?

`@RequestMapping` is the general one; the HTTP-specific shortcuts are cleaner:

- `@GetMapping` — read
- `@PostMapping` — create
- `@PutMapping` — full update
- `@PatchMapping` — partial update
- `@DeleteMapping` — delete

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping("/{id}") public User get(@PathVariable Long id) { ... }
    @PostMapping         public User create(@RequestBody User u) { ... }
}
```

Q: @PathVariable vs @RequestParam vs @RequestBody?

- `@PathVariable` — binds a value from the URI path: `/users/42` -> `id=42`.
- `@RequestParam` — binds a query parameter: `/users?status=active` -> `status="active"`.
- `@RequestBody` — deserializes the request body (JSON) into an object, used for POST/PUT.

```
@GetMapping("/users/{id}")
public User get(@PathVariable Long id,
               @RequestParam(defaultValue = "false") boolean detailed) { ... }

@PostMapping("/users")
public User create(@RequestBody @Valid UserDto dto) { ... }
```

Q: What is ResponseEntity and why use it?

`ResponseEntity<T>` represents the full HTTP response — status code, headers, and body. Use it for explicit control over status codes and headers instead of returning a bare object.

```
@PostMapping("/users")
public ResponseEntity<User> create(@RequestBody @Valid UserDto dto) {
    User saved = service.create(dto);
    return ResponseEntity.status(HttpStatus.CREATED)
        .header("Location", "/api/users/" + saved.getId())
        .body(saved);
}
```

Q: Which HTTP status codes should a REST API use?

- **200 OK** — successful GET/PUT.
- **201 Created** — successful POST that creates a resource.
- **204 No Content** — successful DELETE / update with no body.
- **400 Bad Request** — validation/malformed input.
- **401 Unauthorized** — not authenticated.

- **403 Forbidden** — authenticated but not allowed.
- **404 Not Found** — resource missing.
- **409 Conflict** — duplicate/version conflict.
- **500 Internal Server Error** — unexpected server error.

Q: How do you handle exceptions globally?

Use `@RestControllerAdvice` with `@ExceptionHandler` methods to centralize error handling and return consistent error responses, instead of try/catch in every controller.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiError(404, ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldError().getDefaultMessage();
        return ResponseEntity.badRequest().body(new ApiError(400, msg));
    }
}
```

Q: How do you do request validation?

Add Bean Validation annotations on the DTO and trigger them with `@Valid` on the controller parameter. Failures throw `MethodArgumentNotValidException` (handle it in `@RestControllerAdvice`).

```
public class UserDto {
    @NotBlank private String name;
    @Email private String email;
    @Min(18) private int age;
}

@PostMapping("/users")
public User create(@RequestBody @Valid UserDto dto) { ... }
```

Requires `spring-boot-starter-validation`.

Q: What is idempotency in REST?

An operation is idempotent if repeating it produces the same result as doing it once.

- **GET, PUT, DELETE** — idempotent (PUT sets the same state; deleting an already-deleted resource yields the same final state).
- **POST** — not idempotent (each call creates a new resource).

To make POST idempotent for safety (e.g. payments), use an **idempotency key** header that the server stores and de-duplicates.

Q: What are some REST API best practices?

- Use nouns and plurals for resources: `/api/users` , `/api/users/{id}/orders` .
- Use HTTP methods/verbs correctly (don't put actions in URLs).
- Return proper status codes and consistent error bodies.
- Support pagination, filtering, sorting for collections.
- Version the API.
- Stateless (no server session); use tokens.
- Use DTOs, never expose entities directly.

Q: How do you version a REST API?

Common approaches:

- **URI versioning** — `/api/v1/users` (most common, explicit).
- **Header versioning** — `Accept: application/vnd.app.v1+json` .
- **Request param** — `/api/users?version=1` .

```
@RestController
@RequestMapping("/api/v1/users")
public class UserV1Controller { ... }
```

Spring Data JPA

Q: What is Spring Data JPA?

Spring Data JPA is an abstraction over JPA/Hibernate that eliminates boilerplate DAO code. You declare a repository *interface* and Spring generates the implementation at runtime, including CRUD, derived queries, pagination, and sorting.

Q: JpaRepository vs CrudRepository vs PagingAndSortingRepository?

A hierarchy:

- **CrudRepository** — basic CRUD (`save` , `findById` , `findAll` , `delete`).
- **PagingAndSortingRepository** — adds pagination and sorting.
- **JpaRepository** — adds JPA-specific features (`flush` , `saveAll` , batch deletes, `findAll` returning `List`). Usually you extend `JpaRepository` .

```
public interface UserRepository extends JpaRepository<User, Long> { }
```

Q: Explain @Entity, @Id, and @GeneratedValue.

- **@Entity** — maps the class to a database table.
- **@Id** — marks the primary key field.
- **@GeneratedValue** — how the PK is generated: `IDENTITY` (DB auto-increment, common in MySQL), `SEQUENCE` (DB sequence, common in Postgres), `AUTO` , `TABLE` .

```

@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;
}

```

Q: What are derived query methods?

Spring Data parses the method name and generates the query automatically.

```

public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByStatus(String status);
    User findByEmail(String email);
    List<User> findByAgeGreaterThanAndStatus(int age, String status);
    List<User> findByNameContainingIgnoreCase(String name);
    long countByStatus(String status);
}

```

Q: When and how do you use @Query (JPQL vs native)?

Use `@Query` when derived methods get unwieldy or you need custom logic. JPQL works on entities/fields; native SQL works on tables/columns.

```

// JPQL - uses entity names
@Query("SELECT u FROM User u WHERE u.status = :status")
List<User> findActive(@Param("status") String status);

// Native SQL
@Query(value = "SELECT * FROM users WHERE age > ?1", nativeQuery = true)
List<User> findOlderThan(int age);

// Modifying query
@Modifying
@Transactional
@Query("UPDATE User u SET u.status = :s WHERE u.id = :id")
int updateStatus(@Param("id") Long id, @Param("s") String s);

```

Q: Explain JPA relationships.

- **@OneToOne** — e.g. User ↔ Profile.
- **@OneToMany** / **@ManyToOne** — e.g. one Department has many Employees. **@ManyToOne** is the owning side (holds the FK).
- **@ManyToMany** — e.g. Students ↔ Courses, via a join table.

```

@Entity
public class Department {
    @Id @GeneratedValue private Long id;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Employee> employees = new ArrayList<>();
}

@Entity
public class Employee {
    @Id @GeneratedValue private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}

```

Q: Lazy vs Eager loading?

- **EAGER** — related entity is loaded immediately with the parent.
- **LAZY** — related entity is loaded only when accessed (a proxy is used until then).

Defaults: `@ManyToOne` / `@OneToOne` are EAGER; `@OneToMany` / `@ManyToMany` are LAZY. Prefer **LAZY** for collections to avoid loading unnecessary data; fetch what you need explicitly. Accessing a lazy field outside a transaction causes `LazyInitializationException`.

Q: What is the N+1 query problem and how do you fix it?

When you load N parent entities and then trigger 1 query per parent to load its lazy association — resulting in N+1 queries.

Fixes:

- **JOIN FETCH** in JPQL.
- **@EntityGraph**.
- Batch fetching (`@BatchSize` / `hibernate.default_batch_fetch_size`).

```

@Query("SELECT d FROM Department d JOIN FETCH d.employees")
List<Department> findAllWithEmployees();

// or
@EntityGraph(attributePaths = "employees")
List<Department> findAll();

```

Q: What is @Transactional and how does propagation work?

`@Transactional` declaratively wraps a method in a database transaction — commit on success, rollback on a runtime exception. Implemented via AOP proxy.

Propagation controls how an existing transaction is handled:

- **REQUIRED** (default) — join existing or create new.
- **REQUIRES_NEW** — always start a new, suspend the current.
- **SUPPORTS** — join if one exists, else run non-transactional.
- **MANDATORY** — must run inside an existing one.
- **NESTED** — nested transaction with savepoints.

```
@Transactional
public void transfer(Long from, Long to, BigDecimal amt) {
    accountRepo.debit(from, amt);
    accountRepo.credit(to, amt); // both commit or both roll back
}
```

Note: by default rollback happens only on unchecked (`RuntimeException`) exceptions; use `rollbackFor` for checked ones. Self-invocation bypasses the proxy.

Q: How do you implement pagination?

Pass a `Pageable` to a repository method and return a `Page<T>` (which includes total count, total pages, etc.).

```
public interface UserRepository extends JpaRepository<User, Long> {
    Page<User> findByStatus(String status, Pageable pageable);
}

// usage
Pageable pageable = PageRequest.of(0, 20, Sort.by("name").ascending());
Page<User> page = userRepository.findByStatus("active", pageable);
List<User> content = page.getContent();
long total = page.getTotalElements();
```

Security & Misc

Q: Spring Security basics — authentication vs authorization?

- **Authentication (authn)** — verifying *who you are* (login, credentials, token).
- **Authorization (authz)** — verifying *what you're allowed to do* (roles/permissions).

Spring Security works as a filter chain in front of the app. You configure it via a `SecurityFilterChain` bean.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.csrf(c -> c.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/public/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        ).httpBasic(Customizer.withDefaults());
    return http.build();
}
```

Q: Explain the JWT authentication flow at a high level.

1. Client sends credentials to `/login`.
2. Server validates and returns a signed **JWT** (header.payload.signature).
3. Client stores the token and sends it on each request: `Authorization: Bearer <token>`.
4. A filter validates the signature/expiry on each request and sets the `SecurityContext` — no server-side session (stateless).

JWT is self-contained (carries claims like user id and roles) and signed so it can't be tampered with. Good for stateless, scalable REST APIs.

Q: DTO vs Entity — why separate them?

- **Entity** — maps to a DB table; tied to persistence.
- **DTO (Data Transfer Object)** — shapes data for the API layer.

Reasons to separate: avoid exposing internal schema, prevent over-posting / unwanted field updates, decouple API contract from DB changes, avoid lazy-loading serialization issues, and tailor request/response payloads. Map between them manually or with MapStruct.

```
public record UserDto(Long id, String name, String email) {}

UserDto toDto(User u) {
    return new UserDto(u.getId(), u.getName(), u.getEmail());
}
```

Q: How do you handle CORS in Spring Boot?

CORS controls which origins (domains) can call your API from a browser. Configure per-controller or globally.

```
// Per controller/method
@CrossOrigin(origins = "https://app.example.com")
@RestController
public class UserController { }

// Global
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("https://app.example.com")
            .allowedMethods("GET", "POST", "PUT", "DELETE");
    }
}
```

With Spring Security, configure CORS there too (`http.cors(...)`).

Reactive (Spring WebFlux)

Q: Spring WebFlux vs Spring MVC?

- **Spring MVC** — synchronous, blocking, thread-per-request, built on the Servlet API. Simple and great for most apps.
- **Spring WebFlux** — asynchronous, non-blocking, reactive (Project Reactor), runs on Netty by default. Uses few threads to handle many concurrent connections (event loop). Better for high-concurrency, streaming, or I/O-bound workloads where downstream calls are also non-blocking.

Use WebFlux when you need scalability under heavy concurrent I/O; otherwise MVC is simpler.

Q: Mono vs Flux?

Both are reactive publisher types from Project Reactor representing async streams:

- **Mono** — emits **0 or 1** element (e.g. a single user, or a completion).
- **Flux** — emits **0..N** elements (e.g. a stream/list of users).

They are lazy — nothing happens until subscribed (the framework subscribes for you).

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping("/{id}")
    public Mono<User> getById(@PathVariable String id) {
        return userService.findById(id);    // 0 or 1
    }

    @GetMapping
    public Flux<User> getAll() {
        return userService.findAll();        // 0..N, can stream
    }
}
```

AWS (Basics) — Interview Q&A

Note: AWS is listed as a **basic / working-knowledge** skill on the resume. Focus on fundamentals and honest answers. If you haven't used a service hands-on, say "I understand the concept; I've used it at a basic level" rather than overselling.

Cloud & AWS Fundamentals

Q: What is cloud computing?

On-demand delivery of computing resources (servers, storage, databases, networking) over the internet, with pay-as-you-go pricing. Instead of buying and maintaining physical hardware, you rent resources from a provider like AWS and scale up or down as needed.

Q: What are the benefits of cloud computing?

- **No upfront cost** — pay only for what you use.
- **Elasticity / scalability** — scale resources up or down on demand.
- **High availability & reliability** — data centers across the globe.
- **Speed & agility** — provision resources in minutes.
- **Managed services** — AWS handles patching, backups, hardware.

Q: What is IaaS, PaaS, and SaaS?

- **IaaS (Infrastructure as a Service):** you manage OS, runtime, app — provider manages hardware. *Example: EC2.*
- **PaaS (Platform as a Service):** provider manages OS & runtime, you deploy code. *Example: Elastic Beanstalk, AWS Lambda (close to it).*
- **SaaS (Software as a Service):** fully managed software you just use. *Example: Gmail, Salesforce.*

Q: What is AWS?

Amazon Web Services — the world's most widely used cloud platform, offering 200+ services for compute, storage, databases, networking, ML, and more, on a pay-as-you-go model.

Q: What is a Region and an Availability Zone (AZ)?

- **Region:** a geographic area (e.g., `ap-south-1` Mumbai) containing multiple data centers.
- **Availability Zone:** one or more isolated data centers within a Region, with independent power/networking.
- You deploy across **multiple AZs** for high availability — if one AZ fails, the others keep running.

Q: What is the AWS Free Tier?

A set of services free for 12 months or always-free within limits — e.g., 750 hours/month of a `t2.micro` EC2 instance, 5 GB S3 storage, and 1M Lambda requests/month. Great for learning and small projects.

Q: What is the AWS Shared Responsibility Model?

- **AWS is responsible for security of the cloud** — physical hardware, networking, data centers.
 - **You are responsible for security in the cloud** — your data, access management (IAM), OS patching (on EC2), encryption, and app security.
-

IAM (Identity & Access Management)

Q: What is IAM?

A service to securely control **who** (authentication) can do **what** (authorization) in your AWS account. It manages users, groups, roles, and permission policies — and it's free.

Q: What are IAM users, roles, and policies?

- **User:** a person or app with long-term credentials.
- **Group:** a collection of users sharing permissions.
- **Role:** temporary credentials assumed by a user, service, or app (e.g., an EC2 instance assumes a role to access S3 — no hardcoded keys).
- **Policy:** a JSON document defining allowed/denied actions on resources.

Q: What is the principle of least privilege?

Grant only the **minimum permissions** needed to do a task — nothing more. It limits the blast radius if credentials are compromised.

EC2 (Elastic Compute Cloud)

Q: What is EC2?

A service providing **resizable virtual servers (instances)** in the cloud. You choose the OS, CPU, RAM, and storage, and you fully control the instance — ideal for hosting applications like a Spring Boot backend.

Q: What is an AMI?

Amazon Machine Image — a template containing the OS, software, and configuration used to launch an EC2 instance. You can use AWS-provided AMIs or create your own.

Q: What are EC2 instance types?

Families optimized for different workloads:

- **General purpose** (t3, m5) — balanced, for web/app servers.
- **Compute optimized** (c5) — CPU-heavy work.
- **Memory optimized** (r5) — in-memory databases, caching.
- **Storage optimized** (i3) — high disk I/O.

Q: What is a Security Group?

A **virtual firewall** at the instance level controlling inbound and outbound traffic by port, protocol, and IP. It's **stateful** — if inbound is allowed, the response is automatically allowed out. *Example: allow port 22 (SSH) from your IP and 8080 for your app.*

Q: Security Group vs NACL?

- **Security Group:** instance-level, **stateful**, allow-rules only.
- **NACL (Network ACL):** subnet-level, **stateless**, supports allow *and* deny rules.

Q: What is a key pair?

A public/private key used to securely SSH into a Linux EC2 instance. AWS stores the public key; you keep the private `.pem` file.

Q: What is an Elastic IP?

A **static, public IPv4 address** you can attach to an instance. Unlike the default public IP (which changes on stop/start), an Elastic IP stays the same.

Q: When would you use EC2 vs Lambda?

- **EC2:** long-running apps, full control over the server, predictable/steady load. *Example: a Spring Boot API running 24/7.*
- **Lambda:** short, event-driven tasks, no server management, spiky/occasional workloads. *Example: resizing an image when uploaded to S3.*

S3 (Simple Storage Service)

Q: What is S3?

Object storage for any amount of data — files (objects) stored in **buckets**. Highly durable (99.999999999% — "11 nines"), scalable, and accessed via HTTP/REST APIs. Common for images, backups, logs, and static websites.

Q: What are buckets and objects?

- **Bucket:** a globally-unique-named container for storing objects.
- **Object:** the actual file plus metadata, identified by a **key** (its name/path).

Q: What are S3 storage classes?

- **S3 Standard** — frequent access.
- **S3 Standard-IA / One Zone-IA** — infrequent access, cheaper.
- **S3 Glacier / Glacier Deep Archive** — archival, lowest cost, slower retrieval.
- **S3 Intelligent-Tiering** — auto-moves data between tiers based on usage.

Q: Can S3 host a website?

Yes — **static website hosting** serves HTML/CSS/JS directly from a bucket. *This is exactly how you'd deploy an Angular build* — upload the `dist/` folder to S3, often fronted by CloudFront (CDN) for speed and HTTPS.

Q: What is S3 versioning?

Keeps **multiple versions** of an object so you can recover from accidental deletes/overwrites. Once enabled, deleting an object just adds a "delete marker."

Q: What is a presigned URL?

A time-limited URL that grants temporary access to a private S3 object without making the bucket public — useful for secure downloads/uploads.

Q: S3 vs EBS vs EFS?

- **S3**: object storage, accessed over HTTP, unlimited scale.
 - **EBS (Elastic Block Store)**: a virtual hard disk attached to **one** EC2 instance.
 - **EFs (Elastic File System)**: a shared file system mountable by **many** EC2 instances.
-

Lambda (Serverless)

Q: What is AWS Lambda?

A **serverless compute** service that runs your code in response to events — no servers to provision or manage. You upload a function, AWS runs and scales it automatically, and you pay only for execution time.

Q: How does Lambda work / what triggers it?

An **event source** invokes the function — e.g., an S3 upload, an API Gateway HTTP request, a DynamoDB change, or a scheduled CloudWatch event. Lambda spins up, runs your handler, and shuts down.

Q: What is a cold start?

The delay when Lambda initializes a new execution environment for the first request (or after idle). Subsequent "warm" invocations are faster. Choosing a lighter runtime and smaller package reduces it.

Q: What are typical Lambda use cases?

Image/file processing on upload, lightweight REST APIs (with API Gateway), scheduled jobs, real-time stream processing, and backend automation.

Q: What are Lambda's limitations?

- Max execution timeout (15 minutes).
 - Limited memory/temp storage (configurable, with caps).
 - Stateless — no persistent local state between invocations.
 - Cold starts for latency-sensitive apps.
-

RDS (Relational Database Service)

Q: What is RDS?

A **managed relational database** service. AWS handles provisioning, patching, backups, and scaling, so you focus on your data. Supports **MySQL**, **PostgreSQL** (both in your stack), MariaDB, Oracle, SQL Server, and Amazon Aurora.

Q: RDS vs running a database on EC2?

- **RDS:** AWS manages backups, patching, failover, monitoring — less operational work.
- **DB on EC2:** full control and customization, but **you** manage everything (backups, patching, HA). Use only when you need special configs RDS doesn't support.

Q: What is Multi-AZ in RDS?

A **standby replica in another AZ** for **high availability**. AWS automatically fails over to the standby if the primary fails. It's for **resilience**, not read scaling.

Q: What is a Read Replica?

A **read-only copy** of the database used to **scale read traffic** (e.g., reporting/dashboards). It's for **performance**, not automatic failover.

Q: Multi-AZ vs Read Replica — what's the difference?

- **Multi-AZ:** disaster recovery / high availability (automatic failover, synchronous).
- **Read Replica:** scaling reads (asynchronous, can serve read queries).

Q: What about RDS backups?

RDS supports **automated daily backups** (with point-in-time recovery) and **manual snapshots** you trigger yourself.

CloudWatch

Q: What is CloudWatch?

AWS's **monitoring and observability** service. It collects **metrics, logs, and events** from AWS resources and your apps, and lets you set **alarms** and dashboards.

Q: Metrics vs Logs vs Alarms?

- **Metrics:** numeric data over time (CPU %, memory, request count).
- **Logs:** text output from apps/services (e.g., your Spring Boot logs via CloudWatch Logs).
- **Alarms:** trigger actions when a metric crosses a threshold (e.g., email/SNS when CPU > 80%, or auto-scale).

Q: How would you monitor an application with CloudWatch?

Send app logs to **CloudWatch Logs**, watch key **metrics** (CPU, latency, error rate), set **alarms** for thresholds, and visualize everything on a **dashboard**. Alarms can notify via SNS or trigger auto-scaling.

Q: CloudWatch vs CloudTrail?

- **CloudWatch:** monitors **performance & health** (what's happening *in* resources).
 - **CloudTrail:** records **API calls / who did what** (auditing & governance).
-

Putting It Together

Q: How do these services fit a typical web application?

A simple architecture for your stack:

```
[ Angular app ] → S3 + CloudFront (static hosting + CDN, HTTPS)
      |
      v (REST API calls)
[ Spring Boot API ] → runs on EC2 (behind a Load Balancer)
      |
      v
[ MySQL / Postgres ] → Amazon RDS (Multi-AZ for HA)
```

Monitoring/logs across all → CloudWatch (metrics, logs, alarms)

Occasional tasks (e.g., image processing) → Lambda triggered by S3

Access control everywhere → IAM (least privilege)

Q: How do you keep AWS costs low?

- Use the **right instance size** (don't over-provision); stop unused instances.
- Use **Auto Scaling** so capacity matches demand.
- Pick the **right S3 storage class** and lifecycle rules.
- Use **Reserved/Savings Plans** for steady workloads.
- Set **billing alarms** in CloudWatch.

Q: What are basic AWS security best practices?

- Enable **MFA** and never use the root account for daily work.
- Apply **least privilege** IAM policies; use **roles** instead of hardcoded keys.
- **Encrypt** data at rest (S3, RDS) and in transit (HTTPS/TLS).
- Keep **security groups** tight (open only required ports).
- Enable **CloudTrail** for auditing.

Microservices (Basics) — Interview Q&A

Note: Microservices is listed as a **basic** skill. Focus on core concepts and honest, conceptual answers. Tie examples to Spring Boot / Spring Cloud where natural, since that's your backend stack.

Fundamentals

Q: What are microservices?

An architectural style where an application is built as a **collection of small, independent services**, each responsible for one business capability, communicating over the network (usually REST or messaging). Each service can be **developed, deployed, and scaled independently**.

Q: Monolith vs Microservices?

	Monolith	Microservices
Codebase	Single, unified	Many small services
Deployment	Deploy whole app	Deploy each service independently
Scaling	Scale entire app	Scale only the service that needs it
Tech stack	Usually one	Each service can use its own
Failure	One bug can crash all	Failure is isolated
Complexity	Simple to start	More operational complexity

Q: What are the advantages of microservices?

- **Independent deployment** — ship one service without redeploying everything.
- **Independent scaling** — scale only the hot service.
- **Technology freedom** — different languages/DBs per service.
- **Fault isolation** — one service failing doesn't bring down the whole system.
- **Team autonomy** — small teams own small services.

Q: What are the disadvantages / challenges?

- **Distributed system complexity** — network calls, latency, partial failures.
- **Data consistency** — no single database; eventual consistency.
- **Testing & debugging** harder across services.
- **Deployment & monitoring** overhead (many moving parts).
- **DevOps maturity** required (CI/CD, containers, orchestration).

Q: When should you NOT use microservices?

- Small/simple applications or early-stage products — a monolith is faster to build.
- Small teams without DevOps maturity.
- When the domain boundaries aren't clear yet. *Start with a monolith, split later when needed (see strangler pattern).*

Q: What are the key characteristics of a microservice?

Single responsibility, loosely coupled, independently deployable, owns its own data, communicates via well-defined APIs, and is **resilient** (handles failures of others gracefully).

Q: Microservices vs SOA?

Both break apps into services, but **SOA** typically uses a heavy central **ESB (Enterprise Service Bus)** and shared databases, while **microservices** are lighter, smaller, decentralized, each owning its own data, communicating via simple protocols (REST/messaging).

Communication Between Services

Q: How do microservices communicate?

- **Synchronous** — one service calls another and waits (usually **REST/HTTP**, sometimes gRPC).
- **Asynchronous** — services exchange **messages/events** via a broker (Kafka, RabbitMQ), without waiting.

Q: Synchronous vs Asynchronous — when to use which?

- **Synchronous (REST)**: simple, immediate response needed. Risk: tight coupling, cascading failures.
- **Asynchronous (messaging)**: decoupled, resilient, good for events ("OrderPlaced"), background work. Risk: eventual consistency, more complexity.

Q: What is a message broker?

Middleware that lets services communicate via messages without direct calls. **Kafka** (high-throughput event streaming) and **RabbitMQ** (traditional message queue) are common. It decouples producers from consumers.

Q: What is an API contract?

The agreed-upon **interface** (endpoints, request/response formats) between services. Keeping contracts stable (and versioning them) lets services evolve independently. Tools like OpenAPI/Swagger document them.

Spring Cloud Building Blocks

Q: What is service discovery? (Eureka)

In a dynamic environment, service instances come and go with changing IPs. A **service registry** like **Netflix Eureka** lets services **register themselves** and **discover** others by name instead of hardcoding URLs.

Service A → asks Eureka "where is PAYMENT-SERVICE?" → gets instance address → calls it

Q: What is an API Gateway and why use it?

A **single entry point** for all client requests that routes them to the right backend service. **Spring Cloud Gateway** is common. Benefits: centralized **authentication, rate limiting, logging, routing, CORS**, and it hides internal service structure from clients.

Q: What is a Config Server?

A **centralized configuration** service (Spring Cloud Config) that stores config for all microservices (often in Git). Services fetch their config at startup, so you manage settings in one place and across environments (dev/test/prod) without redeploying.

Q: What is client-side load balancing?

The calling service picks which instance of the target service to call (e.g., **Spring Cloud LoadBalancer**, formerly Ribbon), often using the registry's list of instances — distributing load without a central load balancer.

Q: What is a Feign client?

A **declarative REST client** in Spring Cloud. You define an interface with annotations, and Feign generates the HTTP call code — making inter-service REST calls clean:

```
@FeignClient(name = "payment-service")
public interface PaymentClient {
    @GetMapping("/payments/{id}")
    Payment getPayment(@PathVariable Long id);
}
```

Q: What is a Circuit Breaker and why is it needed?

If a downstream service is slow/failing, repeated calls pile up and can cascade. A **circuit breaker** (e.g., **Resilience4j**, formerly Hystrix) "trips" after repeated failures and **fails fast** (or returns a fallback) instead of waiting — protecting the whole system. States: **Closed → Open → Half-Open**.

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
public Payment getPayment(Long id) { ... }
```

Common Patterns

Q: What is the API Gateway pattern?

Route all external traffic through one gateway that handles cross-cutting concerns (auth, routing, throttling) and aggregates responses — instead of clients calling many services directly.

Q: What is "database per service"?

Each microservice **owns its own database**; no other service accesses it directly. This keeps services loosely coupled and independently deployable — but means **no cross-service joins** and the need for eventual consistency.

Q: What is the Saga pattern?

A way to manage **distributed transactions** across services without a global lock. A business transaction is split into a sequence of **local transactions**; if one fails, **compensating transactions** undo the previous steps. Two styles: **choreography** (events) and **orchestration** (a coordinator).

Q: What is CQRS? (brief)

Command Query Responsibility Segregation — separate the **write model** (commands) from the **read model** (queries), often with different data stores optimized for each. Useful for high-read systems; adds complexity.

Q: What is the Strangler pattern?

A migration strategy: incrementally replace a monolith by building new functionality as microservices and **routing traffic gradually** away from the monolith until it's "strangled" out. Lower risk than a big-bang rewrite. *(Relevant to your migration experience.)*

Q: What are retry and bulkhead patterns? (brief)

- **Retry**: automatically retry a failed call (with backoff) for transient errors.
- **Bulkhead**: isolate resources (e.g., separate thread pools) per dependency so one failing service can't exhaust all resources.

Data & Consistency

Q: Why does each service own its database?

To keep services **independent and loosely coupled** — a schema change in one service shouldn't break another. The trade-off is you lose ACID joins across services and must handle consistency at the application level.

Q: What is eventual consistency?

In distributed systems you often can't have instant consistency everywhere. **Eventual consistency** means data across services becomes consistent **after some delay** (e.g., via events). Acceptable for many business cases (e.g., an order summary updating moments later).

Q: Why are distributed transactions hard?

There's no single database to roll back. A transaction spanning services can partially fail, so you need patterns like **Saga** with compensating actions instead of a traditional commit/rollback.

Cross-Cutting Concerns

Q: How do you handle logging across microservices?

Centralized logging — each service ships logs to a central system (e.g., ELK stack: Elasticsearch + Logstash + Kibana, or CloudWatch). This lets you search logs across all services in one place.

Q: What is distributed tracing?

Tracking a single request as it flows through multiple services using a **correlation/trace ID**. Tools like **Spring Cloud Sleuth** + **Zipkin** add and propagate the ID so you can see the full path and find bottlenecks.

Q: How is security handled in microservices?

Typically **token-based (JWT)**. The API Gateway authenticates the user, then the **token is propagated** to downstream services, each validating it. OAuth2 is common for authorization.

Q: How does Docker fit microservices?

Docker packages each service with its dependencies into a **container** that runs identically anywhere. Each microservice → its own image → its own container, enabling consistent, isolated, independently deployable units. (*You used Docker on the Subsea project.*)

Q: What is Kubernetes (very brief)?

A **container orchestration** platform that automates deploying, scaling, and managing containers across machines. It handles load balancing, self-healing (restarting failed containers), rolling updates, and service discovery — the standard way to run microservices at scale.

Quick Honest Framing for the Interview

"I've worked primarily on the application side — building REST APIs with Spring Boot and consuming them from Angular, and I used Docker on the Subsea project. I understand microservices concepts — independent deployment, service discovery, API gateway, circuit breakers, database-per-service, and the trade-offs around consistency. I'm comfortable picking these up hands-on with Spring Cloud."

Coding Problems — Solutions (JavaScript & Java)

Common coding-round problems for a Wipro lateral interview, easy → medium, with approach, complexity, and clean code.

Tips for the coding round

- **Clarify first:** ask about input type, size, edge cases (empty, null, negatives, duplicates).
- **Think aloud:** explain your approach before coding — interviewers grade reasoning.
- **Start simple, then optimize:** a brute-force that works beats an elegant solution that doesn't.
- **State complexity:** mention time & space (Big-O) for your solution.
- **Dry-run:** walk through one example to prove correctness.
- **Handle edge cases:** empty array, single element, all duplicates, etc.

Strings

Problem: Reverse a string

Approach: Two pointers, or use built-ins. **Complexity:** $O(n)$ time, $O(n)$ space.

```
function reverseString(str) {  
  return str.split('').reverse().join('');  
}  
  
// Without built-ins:  
function reverse(str) {  
  let res = '';  
  for (let i = str.length - 1; i >= 0; i--) res += str[i];  
  return res;  
}
```

```
public static String reverse(String str) {  
  return new StringBuilder(str).reverse().toString();  
}
```

Problem: Check if a string is a palindrome

Approach: Compare characters from both ends. **Complexity:** $O(n)$ time, $O(1)$ space.

```
function isPalindrome(str) {  
  let i = 0, j = str.length - 1;  
  while (i < j) {  
    if (str[i] !== str[j]) return false;  
    i++; j--;  
  }  
  return true;  
}
```

```

public static boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i) != s.charAt(j)) return false;
        i++; j--;
    }
    return true;
}

```

Problem: Count vowels and consonants

Complexity: $O(n)$.

```

function countVowelsConsonants(str) {
    let vowels = 0, consonants = 0;
    for (const ch of str.toLowerCase()) {
        if ("aeiou".includes(ch)) vowels++;
        else if (ch >= "a" && ch <= "z") consonants++;
    }
    return { vowels, consonants };
}

```

Problem: Check if two strings are anagrams

Approach: Sort both, or compare character frequency. **Complexity:** $O(n)$ with frequency map.

```

function isAnagram(a, b) {
    if (a.length !== b.length) return false;
    const count = {};
    for (const ch of a) count[ch] = (count[ch] || 0) + 1;
    for (const ch of b) {
        if (!count[ch]) return false;
        count[ch]--;
    }
    return true;
}

```

```

public static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    char[] x = a.toCharArray(), y = b.toCharArray();
    Arrays.sort(x); Arrays.sort(y);
    return Arrays.equals(x, y);
}

```

Problem: First non-repeating character

Complexity: $O(n)$.

```
function firstNonRepeating(str) {
  const count = {};
  for (const ch of str) count[ch] = (count[ch] || 0) + 1;
  for (const ch of str) if (count[ch] === 1) return ch;
  return null;
}
```

Problem: Remove duplicate characters from a string

```
function removeDuplicates(str) {
  return [...new Set(str)].join("");
}
```

Problem: Reverse words in a sentence

```
function reverseWords(sentence) {
  return sentence.trim().split(/\s+/).reverse().join(" ");
}
```

Problem: Check if two strings are rotations of each other

Approach: `b` is a rotation of `a` if it's a substring of `a + a`. **Complexity:** $O(n)$.

```
function areRotations(a, b) {
  return a.length === b.length && (a + a).includes(b);
}
```

Problem: Count occurrences of each character (frequency map)

```
function charFrequency(str) {
  const map = {};
  for (const ch of str) map[ch] = (map[ch] || 0) + 1;
  return map;
}
```

Arrays

Problem: Find the maximum and minimum in an array

Complexity: $O(n)$.

```
function maxMin(arr) {
  return { max: Math.max(...arr), min: Math.min(...arr) };
}
```

Problem: Find the second largest element

Approach: Single pass, track largest & second largest. **Complexity:** $O(n)$.


```
function secondLargest(arr) {
  let first = -Infinity, second = -Infinity;
  for (const n of arr) {
    if (n > first) { second = first; first = n; }
    else if (n > second && n !== first) second = n;
  }
  return second;
}
```

Problem: Reverse an array in place

```
function reverseArray(arr) {
  let i = 0, j = arr.length - 1;
  while (i < j) { [arr[i], arr[j]] = [arr[j], arr[i]]; i++; j--; }
  return arr;
}
```

Problem: Find the missing number (1..n)

Approach: Sum of 1..n minus actual sum. **Complexity:** O(n) time, O(1) space.

```
function missingNumber(arr, n) {
  const expected = (n * (n + 1)) / 2;
  const actual = arr.reduce((a, b) => a + b, 0);
  return expected - actual;
}
```

Problem: Two Sum (return indices of two numbers adding to target)

Approach: HashMap of value → index. **Complexity:** O(n).

```
function twoSum(nums, target) {
  const map = new Map();
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (map.has(need)) return [map.get(need), i];
    map.set(nums[i], i);
  }
  return [];
}
```

```
public static int[] twoSum(int[] nums, int target) {
  Map<Integer, Integer> map = new HashMap<>();
  for (int i = 0; i < nums.length; i++) {
    int need = target - nums[i];
    if (map.containsKey(need)) return new int[]{map.get(need), i};
    map.put(nums[i], i);
  }
  return new int[]{};
}
```

Problem: Move all zeros to the end

Complexity: $O(n)$.

```
function moveZeros(arr) {
  let pos = 0;
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] !== 0) { [arr[pos], arr[i]] = [arr[i], arr[pos]]; pos++; }
  }
  return arr;
}
```

Problem: Find duplicates in an array

```
function findDuplicates(arr) {
  const seen = new Set(), dups = new Set();
  for (const n of arr) seen.has(n) ? dups.add(n) : seen.add(n);
  return [...dups];
}
```

Problem: Maximum subarray sum (Kadane's algorithm)

Approach: Track current sum; reset if it drops below 0. Complexity: $O(n)$.

```
function maxSubArray(nums) {
  let maxSoFar = nums[0], current = nums[0];
  for (let i = 1; i < nums.length; i++) {
    current = Math.max(nums[i], current + nums[i]);
    maxSoFar = Math.max(maxSoFar, current);
  }
  return maxSoFar;
}
```

Problem: Rotate an array by k positions

Complexity: $O(n)$.

```
function rotate(arr, k) {
  k = k % arr.length;
  return [...arr.slice(-k), ...arr.slice(0, -k)];
}
```

Problem: Merge two sorted arrays

Complexity: $O(n + m)$.

```
function mergeSorted(a, b) {
  const res = []; let i = 0, j = 0;
  while (i < a.length && j < b.length) {
    res.push(a[i] <= b[j] ? a[i++] : b[j++]);
  }
  return [...res, ...a.slice(i), ...b.slice(j)];
}
```

Problem: Find a pair with a given sum

```
function hasPairWithSum(arr, sum) {
  const set = new Set();
  for (const n of arr) {
    if (set.has(sum - n)) return true;
    set.add(n);
  }
  return false;
}
```

Problem: Majority element (appears > n/2 times) — Boyer-Moore

Complexity: O(n) time, O(1) space.

```
function majorityElement(nums) {
  let count = 0, candidate = null;
  for (const n of nums) {
    if (count === 0) candidate = n;
    count += (n === candidate) ? 1 : -1;
  }
  return candidate;
}
```

Numbers

Problem: Factorial (iterative + recursive)

```
function factorialIter(n) { let r = 1; for (let i = 2; i <= n; i++) r *= i; return r; }
function factorialRec(n) { return n <= 1 ? 1 : n * factorialRec(n - 1); }
```

```
public static long factorial(int n) {
  long r = 1;
  for (int i = 2; i <= n; i++) r *= i;
  return r;
}
```

Problem: Fibonacci (iterative + recursive + memoized)

```
function fibIter(n) {
  let a = 0, b = 1;
  for (let i = 0; i < n; i++) [a, b] = [b, a + b];
  return a;
}
function fibMemo(n, memo = {}) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}
```

Problem: Check if a number is prime

Complexity: $O(\sqrt{n})$.

```
function isPrime(n) {  
  if (n < 2) return false;  
  for (let i = 2; i * i <= n; i++) if (n % i === 0) return false;  
  return true;  
}
```

Problem: Print primes in a range

```
function primesInRange(start, end) {  
  const res = [];  
  for (let n = start; n <= end; n++) if (isPrime(n)) res.push(n);  
  return res;  
}
```

Problem: Armstrong number (e.g., $153 = 1^3 + 5^3 + 3^3$)

```
function isArmstrong(n) {  
  const digits = String(n).split("");  
  const p = digits.length;  
  const sum = digits.reduce((a, d) => a + Math.pow(+d, p), 0);  
  return sum === n;  
}
```

Problem: Check if a number is a palindrome

```
function isNumberPalindrome(n) {  
  const s = String(n);  
  return s === s.split("").reverse().join("");  
}
```

Problem: Swap two numbers without a temp variable

```
let a = 5, b = 10;  
a = a + b; b = a - b; a = a - b; // a=10, b=5  
// or with XOR: a ^= b; b ^= a; a ^= b;
```

Problem: GCD and LCM

```
function gcd(a, b) { return b === 0 ? a : gcd(b, a % b); }  
function lcm(a, b) { return (a * b) / gcd(a, b); }
```

Problem: Count digits & sum of digits

```
function sumOfDigits(n) {  
  let sum = 0;  
  n = Math.abs(n);  
  while (n > 0) { sum += n % 10; n = Math.floor(n / 10); }  
  return sum;  
}
```

Problem: FizzBuzz

```
function fizzBuzz(n) {  
  for (let i = 1; i <= n; i++) {  
    if (i % 15 === 0) console.log("FizzBuzz");  
    else if (i % 3 === 0) console.log("Fizz");  
    else if (i % 5 === 0) console.log("Buzz");  
    else console.log(i);  
  }  
}
```

Sorting & Searching

Problem: Bubble sort

Complexity: $O(n^2)$.

```
function bubbleSort(arr) {  
  for (let i = 0; i < arr.length - 1; i++)  
    for (let j = 0; j < arr.length - 1 - i; j++)  
      if (arr[j] > arr[j + 1]) [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];  
  return arr;  
}
```

Problem: Binary search (sorted array)

Complexity: $O(\log n)$.

```
function binarySearch(arr, target) {  
  let lo = 0, hi = arr.length - 1;  
  while (lo <= hi) {  
    const mid = Math.floor((lo + hi) / 2);  
    if (arr[mid] === target) return mid;  
    if (arr[mid] < target) lo = mid + 1;  
    else hi = mid - 1;  
  }  
  return -1;  
}
```

```

public static int binarySearch(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

```

Problem: Quicksort (idea)

Pick a **pivot**, partition the array so smaller elements go left and larger go right, then recursively sort each partition.

Average $O(n \log n)$, worst case $O(n^2)$.

HashMap-based

Problem: Group anagrams

Approach: Use sorted word as the map key. **Complexity:** $O(n \cdot k \log k)$.

```

function groupAnagrams(words) {
    const map = {};
    for (const w of words) {
        const key = w.split('').sort().join('');
        (map[key] || []).push(w);
    }
    return Object.values(map);
}

```

Problem: First repeating element

```

function firstRepeating(arr) {
    const seen = new Set();
    for (const n of arr) {
        if (seen.has(n)) return n;
        seen.add(n);
    }
    return null;
}

```

Recursion

Problem: Sum of array elements (recursive)

```
function sumArray(arr, i = 0) {  
  return i === arr.length ? 0 : arr[i] + sumArray(arr, i + 1);  
}
```

Problem: Power(x, n)

Approach: Fast exponentiation. **Complexity:** $O(\log n)$.

```
function power(x, n) {  
  if (n === 0) return 1;  
  const half = power(x, Math.floor(n / 2));  
  return n % 2 === 0 ? half * half : x * half * half;  
}
```

Stack / Linked List

Problem: Check balanced parentheses

Approach: Push opening brackets, pop & match on closing. **Complexity:** $O(n)$.

```
function isBalanced(s) {  
  const stack = [];  
  const pairs = { "(": ")", "[": "]", "{": "}" };  
  for (const ch of s) {  
    if ("([{".includes(ch)) stack.push(ch);  
    else if (pairs[ch]) {  
      if (stack.pop() !== pairs[ch]) return false;  
    }  
  }  
  return stack.length === 0;  
}
```

Problem: Reverse a linked list (Java)

Complexity: $O(n)$ time, $O(1)$ space.

```
class Node { int val; Node next; Node(int v){ val = v; } }

public static Node reverse(Node head) {
    Node prev = null, curr = head;
    while (curr != null) {
        Node next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev; // new head
}
```

Problem: Find the duplicate in an array (Floyd's cycle / set)

```
function findDuplicate(nums) {
    const seen = new Set();
    for (const n of nums) {
        if (seen.has(n)) return n;
        seen.add(n);
    }
    return -1;
}
```


Projects — Deep-Dive Interview Q&A

Experience / "project difficulty" round. Interviewers cross-question on real decisions, trade-offs, and concrete technical detail. Speak in the first person, give numbers where you can, and always close a "how did you solve it" answer with the outcome.

1) RoboGebra — AI-Driven Math Learning Platform

Project pitch: RoboGebra is an AI-driven math learning platform that helps students learn step-by-step, with personalized dashboards and bilingual (Tamil/English) explanations. I work as a frontend-focused full-stack engineer building the Angular + Ionic client and the supporting Java/MySQL REST services. I own features like the personalized learning dashboard, the Study Reminder system, the Study List module, the AI math explanation engine integration, the interactive Quiz module, and real-time math computation with live graph visualization. My work directly improved learner retention through scheduled reminders and made complex problems easier to grasp via instant, interactive feedback.

Q: What was your exact role and the team size?

I was the primary frontend engineer (Angular/Ionic) and also wrote backend REST endpoints in Java with MySQL for my modules. The team was around 6–7: a couple of frontend devs, two backend devs, a designer, a QA, and a lead/PM. I owned the dashboard, reminders, study list, quiz, and the real-time computation features end to end — UI, the API contract, and the persistence layer for those.

Q: Describe the overall architecture.

It's a hybrid mobile/web app: Angular as the framework with Ionic for the mobile/cross-platform UI layer, talking over REST to Java services backed by MySQL. The AI explanation/quiz generation lives behind a service that the Java layer proxies, so the client never calls the model directly — that keeps keys server-side and lets us cache/normalize responses. On the frontend I used a service-per-domain pattern (DashboardService, ReminderService, StudyListService, QuizService), RxJS for async streams, and a shared state layer for cross-component data like the current learner profile.

Q: The real-time computation and graph visualization sounds expensive. How did you keep it performant?

The problem: when a student drags a slider or edits a value, we recompute the expression and redraw the graph plus fetch AI insights. Naively that fires a request and a re-render on every keystroke/drag tick, which janks the UI and hammers the AI endpoint. I split it into two streams — local math computation (instant, runs in the browser) and the AI insight call (debounced).

```
// Local recompute is instant; AI insight is debounced + cancellable
this.valueChange$.pipe(
  // redraw the graph immediately from the local math engine
  tap(v => this.redrawGraph(this.computeLocally(v))),
  debounceTime(400),
  distinctUntilChanged(),
  switchMap(v => this.quizApi.getAiInsight(v).pipe(
    catchError(() => of(null)) // never let one failed insight break the canvas
  ))
).subscribe(insight => this.insight = insight);
```

`switchMap` cancels the in-flight AI request when a newer value arrives, `debounceTime` collapses bursts, and `distinctUntilChanged` skips no-op changes. I also memoized the local compute and used `trackBy` / `OnPush` so the graph component only re-renders when its inputs actually change. Result: the graph feels instant and AI calls dropped by a large margin.

Q: How did you implement the bilingual (Tamil/English) rendering? Any gotchas?

Explanations come back as structured content (steps + formulas), so I keep the language as a key rather than baking translated strings into components. The AI engine returns both language variants (or we request a specific one), and the UI switches via a language signal/observable. The gotchas were font rendering and math: Tamil glyphs needed a font that supports the script, and mixing Tamil prose with LaTeX-style formulas meant I had to render the formula segments separately (MathJax/KaTeX) from the localized prose so the RTL/script handling didn't mangle the math. I segmented each step into "text" and "formula" tokens and rendered them in their own spans.

Q: How does the Study Reminder system work, and what REST design did you use?

It's flexible scheduling with notifications. I exposed a standard REST resource for reminders with full CRUD:

```
@RestController
@RequestMapping("/api/reminders")
public class ReminderController {
    @GetMapping           // list a user's reminders
    @PostMapping          // create
    @PutMapping("/{id}")  // update schedule/status
    @DeleteMapping("/{id}") // remove
}
```

A reminder stores a schedule (one-off or recurring), the target study item, and a status. A scheduler checks due reminders and pushes notifications. On the client I optimistically update the list on POST/PUT/DELETE and reconcile if the server rejects. Adding consistent reminders measurably improved retention because learners came back on cadence.

Q: How did you handle the Study List real-time sync?

The Study List lets learners create/organize/track goals and needs to reflect changes immediately across views (and devices). I kept a single source of truth in a shared service exposing a `BehaviorSubject` of the list; any mutation goes through that service so every subscribed component updates instantly. For cross-device sync, mutations hit the API and the service refreshes/merges so an edit on mobile shows on web. Reordering and completion toggles are optimistic with rollback on error.

Q: How do you manage state across these modules?

For most feature state I use RxJS services with `BehaviorSubject`s (a lightweight store pattern) rather than pulling in NgRx — the app's state is feature-scoped, so a store-per-domain keeps it simple and testable. Shared/global concerns (current learner, language, auth) live in singleton root-provided services. Components subscribe via the `async` pipe so subscriptions are cleaned up automatically.

Q: How do you handle API errors and loading states?

Every API call goes through typed service methods that return Observables; components render explicit loading/empty/error states rather than blank screens. I use a `catchError` to map server errors into a user-facing message and, where safe, fall back (e.g., the AI insight failing should not break the graph — see the snippet above). An HTTP interceptor centralizes auth headers, 401 handling, and a global error toast so I don't repeat that logic per call.

Q: How did you test this?

Unit tests for the services (Jasmine/Karma) with `HttpTestingController` to assert the right requests and to simulate error responses, and component tests for the dashboard/quiz logic. For the RxJS-heavy real-time code I tested the debounce/switchMap behavior with marble-style tests so I could prove old AI requests get cancelled. QA covered end-to-end flows on device.

Q: What would you do differently?

I'd introduce a proper state library (NgRx Component Store or Signals-based store) earlier — once several modules needed to share the learner/profile state, the hand-rolled subjects got repetitive. I'd also add response caching for AI explanations at the service layer from day one, since identical problems get explained repeatedly and that's wasted model cost and latency.

Q: How did you collaborate?

Agile with two-week sprints, Jira for stories/bugs, Git with feature branches and PR reviews. I'd pick up a story, agree the API contract with the backend dev, build behind a feature branch, get a review, and demo in sprint review.

2) Subsea — Marine Equipment & Operations Management

Project pitch: Subsea is a marine equipment and operations management platform used to track and schedule subsea assets and operations. I worked on the Angular frontend with a .NET API backend, containerized with Docker and deployed on Azure. My main contributions were building the Schedule-Manager module with its .NET APIs, implementing complex grids with Kendo UI, adding dynamic category/subcategory management, and fixing production bugs. I also refactored parts of the architecture to be more modular and reusable.

Q: What was your role and team size?

Full-stack with an Angular emphasis on a team of roughly 5–6. I owned the Schedule-Manager module front to back — Angular UI plus the .NET endpoints — and I was one of the people who picked up production defects.

Q: Describe the architecture.

Angular SPA, .NET Web API services, containerized with Docker and running on Azure. The frontend talked to REST endpoints; Docker gave us consistent local/CI/prod environments, and Azure hosted the containers. The Schedule-Manager was a feature module with its own routing, services, and Kendo-based grid components.

Q: You used Kendo UI Grid with complex/large data. What was the challenge and how did you solve it?

The grids showed large schedule/equipment datasets with sorting, filtering, grouping, and inline editing, and the default client-side binding got sluggish as rows grew. I moved the grid to server-side data operations — paging, sorting, and filtering done in the .NET API rather than loading everything into the browser:

```
// Push paging/sort/filter to the server via Kendo DataSourceRequest
loadGrid(state: State) {
  const query = toDataSourceRequestString(state); // Kendo helper -> querystring
  return this.http.get<GridDataResult>(`/api/schedule?${query}`);
}
```

```
// .NET side: let Kendo's DataSourceRequest do paging/sort/filter in the DB query
[HttpGet]
public DataSourceResult Get([DataSourceRequest] DataSourceRequest request)
=> _db.Schedules.ToDataSourceResult(request); // translates to SQL paging
```

That meant only one page of rows crossed the wire. Combined with virtual scrolling on the grid and only requesting the columns we display, the grid stayed responsive even with large datasets.

Q: How did the dynamic category/subcategory management work?

Categories and their subcategories were data-driven, not hardcoded — admins could add/edit them and the UI had to reflect that hierarchy live. I modeled it as a parent/child relationship from the API and rendered cascading selects where choosing a category filters its subcategories. On change I reload dependent dropdowns reactively rather than re-fetching the whole tree.

Q: How did Docker fit into your workflow?

Docker gave us a reproducible environment — the same image ran on my machine, CI, and Azure, which killed the "works on my machine" class of bugs, especially for the .NET runtime and dependencies. I worked with the Dockerfile/compose setup to run the API and its dependencies locally so I could integrate the frontend against a realistic backend.

Q: Tell me about a production bug you fixed.

A common pattern was grid/state defects where filters or edits didn't persist correctly after a reload. My approach was reproduce-first: get exact repro steps from the report, reproduce locally against the same data shape, narrow it to frontend vs API by inspecting the network calls, fix at the right layer, and add a guard/test so it didn't regress. Doing it methodically rather than guessing is what kept fixes from bouncing back.

Q: How did you handle state and API calls here?

Feature-scoped Angular services wrapping the .NET endpoints, Observables consumed with the async pipe, and the grid driven by server-side state objects. Error handling went through a shared interceptor plus per-call `catchError` mapping to user-facing messages.

Q: What did you refactor for modularity/reusability?

I pulled repeated grid configuration and category/subcategory selectors into shared components and helpers so other modules could reuse them instead of copy-pasting Kendo setup. I also tightened the feature module boundaries so Schedule-Manager didn't leak dependencies into unrelated areas.

Q: How did you test?

Unit tests for services and component logic, and manual/QA verification of the grid operations and schedule flows. For bug fixes I added regression coverage for the specific failing case.

Q: What would you do differently?

I'd standardize the grid data-binding pattern across the app from the start — we had a mix of client- and server-bound grids, and unifying on server-side operations earlier would have prevented the performance issues. I'd also add more automated integration tests around the .NET endpoints.

Q: How did you collaborate?

Agile sprints, Jira, Git PRs, and close coordination with the backend on the .NET API contracts since I was building both sides of Schedule-Manager.

3) Backoffice UI — Angular Version Migration

Project pitch: This was a focused migration project to bring four Backoffice repositories up to Angular 16. I upgraded a Ticketing application from Angular 2 to 16 and three SSG web-plugin apps from Angular 9 and 4 up to 16. The work was about navigating years of breaking changes safely and verifying nothing regressed through thorough testing. The payoff was modern, supported, faster-building apps on a single Angular version.

Q: What was your role and team size?

I led/owned the migrations across the four repos. It was a small effort — me driving the upgrades with the original feature teams and QA validating their apps. I was the person who understood the cross-version breaking changes and applied them consistently.

Q: What's your strategy for a multi-version jump like Angular 2/9 to 16?

I never jump straight across — Angular only officially supports one-major-at-a-time upgrades. I went incrementally (e.g., 9 → 10 → 11 ... → 16), using `ng update` per step which runs the migration schematics, building and running tests after each hop, and committing each version as its own checkpoint so I could bisect if something broke. The update.angular.io guide was my checklist for each step.

Q: What were the actual breaking changes you had to handle?

A lot. Key ones:

- **Ivy** became the default renderer (v9) — some libraries that relied on View Engine or non-Ivy-compatible metadata needed updates or replacement; I had to ensure all dependencies were Ivy-compatible.
- **RxJS 6 → 7** — typing changes and the move to fully pipeable operators; I had to remove any remaining deprecated operator imports and fix `toPromise()` usage (deprecated, later replaced with `firstValueFrom` / `lastValueFrom`).
- **Deprecated/removed APIs** — e.g. the `entryComponents` array became unnecessary with Ivy, `ViewChild`'s `static` flag semantics changed (v8/9), and various TypeScript strictness bumps surfaced type errors.
- **TypeScript and tooling version bumps** at each step, plus changes to the build (`angular.json`) schema and eventual move toward the newer build pipeline.
- **Strict mode and stricter templates** flagged previously-silent issues.

Q: How did RxJS 6→7 specifically bite you?

Mostly typing and deprecated patterns. `toPromise()` was deprecated, so I migrated to `firstValueFrom` / `lastValueFrom`. Some operator overloads got stricter so a few `map` / `reduce` chains that were loosely typed needed explicit types. The fix was mechanical but had to be done carefully across the codebases.

```
// before (RxJS 6, deprecated)
const data = await this.http.get('/api/tickets').toPromise();

// after (RxJS 7)
import { firstValueFrom } from 'rxjs';
const data = await firstValueFrom(this.http.get('/api/tickets'));
```

Q: How did you make the transition "seamless"? What was the testing strategy?

Build + test after every single version step, not just at the end — that localizes any breakage to one hop. I ran the existing unit tests, did smoke testing of the critical flows in each app, and leaned on QA for full regression on the Ticketing and plugin apps. Where tests were thin, I added coverage around the riskiest areas before upgrading so I'd notice regressions. Keeping each version in its own commit meant I could always roll back one step.

Q: The three plugin apps were SSG web plugins — anything special?

They were embedded plugins, so I had to make sure the upgraded Angular bundles still loaded correctly inside the host SSG environment and didn't clash with the host's runtime. Verifying the integration boundary (how the plugin is bootstrapped/mounted) was as important as the in-app behavior.

Q: How did you handle third-party libraries that weren't compatible?

I checked each dependency's Angular compatibility before each hop. If a library lagged, I either upgraded it to a compatible version, found a maintained alternative, or temporarily pinned and isolated it until a compatible release existed. Ivy compatibility was the big filter at v9.

Q: What would you do differently?

I'd add or strengthen automated tests before starting the migration, since strong test coverage is what makes a big upgrade safe and fast. I'd also script the per-step build/test loop in CI so each version bump is gated automatically rather than verified by hand.

Q: How did you manage this in Git?

A branch per repo, one commit per major version step with a clear message, and PRs reviewed by the owning team before merge. The granular commits doubled as documentation of exactly what changed at each version.

4) EasyVisa — Visa Management for Attorneys

Project pitch: EasyVisa is a visa case management platform for immigration attorneys — managing applications, documents, and the relationships between cases, applicants, and petitioners. I worked on the Angular frontend against a Grails/Groovy backend with PostgreSQL and Neo4j for graph relationships. I built the document portal module with accordion upload panels, created dynamic forms with Angular Formly including nested reactive forms, customized an image cropper, drove an Angular version upgrade, and built reusable components with unit tests and documentation. My work made complex, document-heavy visa workflows manageable for attorneys.

Q: What was your role and team size?

Frontend engineer on a team of around 6–8 (frontend, Grails/Groovy backend devs, QA, lead). I owned the document portal and the dynamic forms work, and I contributed shared/reusable components used across the app.

Q: Why Neo4j alongside Postgres — what's the architecture?

Postgres held the core relational/transactional data, while Neo4j modeled the relationships — applicants, petitioners, cases, family/employer links — because visa cases are fundamentally a graph of related entities. Querying "who is connected to this case and how" is natural in a graph and painful with deep relational joins. The Grails/Groovy backend exposed REST endpoints to the Angular frontend; I consumed those, and where the data was relationship-shaped (e.g., showing related parties on a case) it was backed by Neo4j traversals on the server.

Q: The dynamic nested Formly forms sound hard. Walk me through a challenge.

Visa forms are huge, conditional, and repeatable — a section can have N applicants, each with nested address/employment histories, and fields appear/disappear based on earlier answers. Hardcoding that is unmaintainable, so I used Angular Formly to render forms from a JSON field config, with `fieldGroup` for nesting and `fieldArray` for repeatable sections:

```
const fields: FormlyFieldConfig[] = [{
  key: 'applicants',
  type: 'repeat', // custom repeating section
  fieldArray: {
    fieldGroup: [
      { key: 'firstName', type: 'input', props: { label: 'First name', required: true } },
      { key: 'hasEmployer', type: 'checkbox', props: { label: 'Currently employed?' } },
      {
        key: 'employer', type: 'input', props: { label: 'Employer' },
        expressions: { hide: 'model.hasEmployer === false' } // conditional visibility
      }
    ]
  }
}];
```

The hard parts were conditional logic (Formly `expressions` for show/hide and dynamic required), keeping nested reactive form state valid as sections were added/removed, and validation bubbling up from nested groups to the submit button. I solved validation by relying on Formly's nested `FormGroup` aggregation and writing custom validators for the visa-specific rules, then surfacing errors at the right nesting level.

Q: Tell me about the document portal and large uploads.

Attorneys upload many documents per case, organized in accordion panels by document type. Large files were the issue — uploading naively blocks the UI and risks timeouts. I used `HttpClient` with `reportProgress` to stream progress per file, allowed concurrent-but-throttled uploads, and gave each panel its own progress/error state so one failed upload didn't sink the others:

```
this.http.post('/api/documents', formData, {
  reportProgress: true, observe: 'events'
}).subscribe(event => {
  if (event.type === HttpEventType.UploadProgress) {
    this.progress = Math.round(100 * event.loaded / (event.total ?? 1));
  } else if (event.type === HttpEventType.Response) {
    this.markUploaded();
  }
});
```

For very large files the right move is chunked/resumable upload, which I flagged as the next improvement.

Q: You customized an image cropper — why and how?

Certain documents/photos (e.g., applicant photos) had to meet specific dimension/aspect requirements, so I wrapped and customized an Angular image-cropper component to enforce the aspect ratio, output size, and format we needed, and wired its output into the upload flow. The customization was mostly constraining the crop box and post-processing the output blob before upload.

Q: How did you handle state, APIs, and errors?

Feature services wrapping the Grails REST endpoints, Observables with the async pipe, and reactive forms as the source of truth for form state. Errors were handled per-call with `catchError` mapping to user messages plus a shared interceptor for auth and global failures. For the forms, I kept the Formly model as the single state object so save/restore (drafts) was straightforward.

Q: You built reusable components — give an example and the benefit.

I extracted things like the upload-panel, the Formly field types, and common form controls into shared, configurable components. The benefit was consistency and speed: a new form section or document type became a config change rather than new bespoke code, and bugs got fixed once in the shared component.

Q: How did you test, and what about the version upgrade?

I wrote unit tests (Jasmine/Karma) for the components and form logic and documented the reusable pieces so other devs could adopt them. For the Angular upgrade I followed the incremental `ng update` approach, building and testing at each step and fixing breaking changes/deprecations as they surfaced.

Q: What would you do differently?

I'd move large uploads to chunked/resumable from the start, and I'd consider a schema-driven approach where the form JSON configs come from the backend so non-developers could adjust forms without a release. I'd also add more integration tests around the conditional form logic, since that's where regressions hide.

Q: How did you collaborate?

Agile with Jira and Git PR reviews, and tight coordination with the Grails/Groovy team on REST contracts and on what data came from Postgres vs Neo4j.

General project & experience questions

Q: What was the most challenging project and why?

RoboGebra's real-time computation-and-visualization feature, because it combined frontend performance (instant graph updates), async orchestration (debounced, cancellable AI calls), and bilingual rendering of mixed prose and formulas. Getting it to feel instant while not flooding the AI backend forced careful RxJS design — separating the instant local compute from the debounced/ `switchMap` -cancelled remote call.

Q: How do you approach building a new feature end to end?

Clarify the requirement and acceptance criteria, agree the API contract with the backend early, sketch the component/service breakdown, build behind a feature branch with the happy path plus loading/empty/error states, write unit tests as I go, self-review the diff, open a PR, address review comments, and demo in sprint review. I try to ship a thin vertical slice first, then iterate.

Q: How do you debug a production issue?

Reproduce first — get exact steps and the affected data shape, reproduce locally. Then isolate the layer using the network tab and logs (frontend vs API vs data), form a hypothesis, fix at the correct layer, and add a regression test so it can't silently come back. I avoid guess-and-deploy; methodical reproduction is what stops fixes from bouncing.

Q: What do you look for in code reviews?

Correctness first, then readability and maintainability: clear naming, proper unsubscription/async-pipe usage, error/loading handling, no obvious performance traps (unnecessary re-renders, client-side data loads that should be server-side), and adequate tests. I give specific, kind, actionable comments and distinguish blocking issues from nits.

Q: How do you ensure code quality?

Strong typing, small focused components/services, the async pipe and OnPush to avoid subscription leaks and extra renders, unit tests for logic, linting/formatting in CI, and reusing shared components instead of duplicating. Consistent patterns across the codebase matter as much as any single clever fix.

Q: Describe your Agile/Scrum process.

Two-week sprints with planning, daily standups, sprint review/demo, and retro. Work tracked in Jira as stories/bugs with estimates, code in Git with feature branches and PR reviews. Standup is where I flag blockers early.

Q: How do you handle changing requirements mid-sprint?

I clarify the change with the PM/stakeholder, assess impact on scope and the sprint commitment, and raise it transparently rather than silently absorbing it. Small changes I fold in; larger ones get re-estimated and either swapped for something or moved to the next sprint. Building things data/config-driven (like the Formly forms) also made many "changes" cheap.

Q: What was your biggest technical decision and the trade-off?

On EasyVisa, choosing Angular Formly with JSON-driven config for the visa forms instead of hand-coding each form. The trade-off was a steeper initial learning curve and some complexity in custom field types versus huge long-term savings — new forms and conditional logic became configuration, not new components. For the scale of forms there, it clearly paid off.

Q: How do you handle merge conflicts and what's your Git workflow?

Feature branches off main, small frequent commits, regular rebases/merges from main to keep the branch current so conflicts stay small. When conflicts happen I resolve them by understanding both sides' intent — not blindly picking one — re-run the build and tests, and if it's non-trivial I check with the other author. PRs are reviewed before merge.

Q: How do you estimate tasks?

I break a story into concrete sub-tasks, estimate each based on complexity and unknowns (relative sizing / story points), pad for testing and review, and flag risky areas as needing a spike. I'd rather surface uncertainty up front than commit to a number I can't hit. Past similar work calibrates the estimate.

Q: What's your experience with CI/CD and deployment?

On Subsea we used Docker for reproducible builds and deployed containers to Azure, so the same image ran locally, in CI, and in prod — which eliminated environment drift. Generally I work with pipelines that lint, build, and run tests on every PR so problems are caught before merge, and I treat a green pipeline as a gate to deploy.

Q: Tell me about a disagreement on a technical approach.

On grid performance in Subsea there was a question of client-side vs server-side data operations. I argued for server-side paging/sorting/filtering because client-side wouldn't scale with the dataset. Rather than just assert it, I demonstrated the lag with a realistic data volume, we agreed on the server-side approach, and the grid stayed responsive. I focus on evidence and the user impact, and I commit to the decision once it's made even if it isn't the one I proposed.

HR & Behavioral Interview — Questions & Sample Answers

These are the most common HR and managerial-round questions for an experienced (lateral) candidate, with sample answers written in first person and tailored to Priyanka's background. Use the STAR method (Situation, Task, Action, Result) where the question is about a real scenario. Adapt the wording so it sounds like you — don't recite robotically.

1. Tell me about yourself.

Sample answer:

"I'm a Full Stack Developer with over four years of experience, currently a Software Engineer at Provility Software Solutions in Chennai. I specialize in Angular on the front end and Java with Spring Boot on the back end. My most recent project is RoboGebra, an AI-driven math learning platform where I built the learning dashboard, a quiz module, and a study reminder system using REST APIs. I've also worked across .NET, microservices, and cloud on AWS and Azure. I'm now looking for a role with more scale and variety, which is what draws me to Wipro."

(See the dedicated 00-self-introduction.md for the full 60-second and 2-minute scripts.)

2. Why do you want to join Wipro?

Sample answer:

"Wipro is a global organization with a huge range of clients and domains, and that's exactly the kind of scale I want next. So far I've worked on focused products in a smaller setup, where I've gained strong end-to-end ownership. At Wipro I'd get exposure to enterprise-grade projects, structured engineering practices, and the chance to grow into more senior technical and architectural responsibilities. The breadth of technology Wipro works with — across cloud, full stack, and modern frameworks — maps very well to my Angular, Java and Spring Boot background, so I see it as a place where I can contribute from day one and also keep learning."

3. Why are you looking for a change?

Sample answer:

"I've had a great four years at Provility and I've grown a lot — from building reusable components on EasyVisa to leading an Angular migration and now working on AI-driven features in RoboGebra. At this stage I'm looking for a larger platform with bigger, more complex projects and clearer growth into senior roles. It's a move toward more scale and learning, not away from any problem. I want to keep building, just at a larger scale and with broader exposure."

4. What are your strengths?

Sample answer:

"My biggest strength is end-to-end ownership — I'm comfortable building a feature from the REST API in Spring Boot or .NET all the way up to the Angular UI, which means I can move fast without waiting on hand-offs. Second, I'm strong on reusable, maintainable front-end work — on EasyVisa I built custom Formly components and reusable form modules that

the whole team used. And third, I'm adaptable: across my projects I've picked up Angular, React, .NET, Grails and Groovy as the work demanded, so learning a new stack quickly is something I'm confident about."

5. What are your weaknesses?

Sample answer:

"I tend to be a bit of a perfectionist with code — I used to spend extra time polishing things that were already good enough. I've learned to manage it by being more deliberate about priorities: I ask what's truly important for the release, get it working and reviewed first, and refine later if there's time. I also lean on code reviews and feedback to know when something is genuinely 'done.' It's made me faster while still keeping quality high. The other thing I've been actively working on is public speaking and presenting in larger forums, which I'm improving by volunteering to demo features in our sprint reviews."

6. What is your biggest achievement?

Sample answer (STAR):

"Situation: On RoboGebra, one of the key differentiators was giving students math explanations in their own language. **Task:** I was responsible for building the front-end of the AI math explanation feature and the surrounding learning experience — dashboard, quiz, and reminders. **Action:** I built the personalized learning dashboard and quiz module in Angular and Ionic, integrated the AI explanation engine to deliver bilingual Tamil and English explanations, and wired up real-time math computation and visualization. I also built a study reminder system backed by REST APIs so students stayed on track. **Result:** The platform delivered a genuinely personalized, accessible learning experience, especially for Tamil-speaking students who could finally learn math in their native language. It's the project I'm most proud of because it combined clean engineering with real user impact."

7. Tell me about a time you had a conflict with a teammate.

Sample answer (STAR):

"Situation: On one project, a backend teammate and I disagreed on the API contract for a form-heavy module — they wanted one large endpoint, I wanted more granular ones for the nested reactive forms I was building. **Task:** We needed to agree quickly without blocking the sprint. **Action:** Instead of pushing my view, I set up a short call, walked through the actual UI flows and how the form state would be saved, and listened to their concern about round-trips and performance. We landed on a middle ground — a couple of focused endpoints plus one batch save. **Result:** It unblocked both of us within a day, the integration was clean, and we actually used that same pattern on later modules. I learned that most 'conflicts' are just two valid priorities that haven't been talked through yet."

8. How do you handle tight deadlines?

Sample answer:

"I break the work down first and figure out what's truly must-have versus nice-to-have, then communicate early if something is at risk — surprises are worse than bad news. A real example: during the Subsea project I had bug fixes plus the Schedule-Manager module to deliver in a short window. I prioritized the blocking bugs, got the core Schedule-

Manager flow with the Kendo UI grid working end to end first, and left polish for last. I also kept my lead updated daily so expectations were aligned. We shipped on time. Under pressure I stay calm, prioritize ruthlessly, and over-communicate."

9. Tell me about a time you failed, and what you learned.

Sample answer (STAR):

"**Situation:** Early in my career on EasyVisa, I built a reusable Formly component and assumed my local testing was enough, so I didn't write thorough unit tests for an edge case. **Task:** The component was meant to handle dynamic nested forms. **Action:** A configuration I hadn't tested broke in another part of the app during QA. I owned it immediately, fixed it, and then went back and added proper unit tests covering the edge cases. **Result:** It was caught before release, but it taught me to treat unit tests as part of 'done,' not an afterthought. Since then, writing tests for the components I own has been a standard part of my workflow, and it's saved me many times."

10. How do you handle feedback or criticism?

Sample answer:

"I genuinely value it — code reviews are one of the main ways I've grown. I try not to take feedback personally; I separate the code from my ego. When I get a review comment I don't agree with, I ask questions to understand the reasoning rather than just defending my approach, and often I end up learning something. When I do disagree after that, I explain my view with context and we decide together. Feedback is faster learning, so I'd rather hear it early than discover a problem in production."

11. Where do you see yourself in 5 years?

Sample answer:

"In five years I'd like to be a senior full-stack engineer or a technical lead — someone who not only delivers complex features but also makes architecture decisions and mentors junior developers. I've already had a taste of leadership through the Angular 16 migration I led, and I enjoyed owning that. At a company like Wipro, with its scale and structured growth paths, I see a clear runway to deepen my technical expertise while taking on more ownership and eventually guiding a team."

12. What are your salary expectations?

Sample answer:

"I'm looking for a package that reflects my four-plus years of full-stack experience and is aligned with industry standards for this role and location. I'm open and flexible, and I trust Wipro to make a fair offer based on my skills and the value I bring. If you can share the band for this position, I'm happy to discuss where I'd fit within it. Overall, the right role and growth opportunity matter as much to me as the number."

13. What is your notice period?

Sample answer:

"My current notice period is [your actual notice period, e.g. 60 days]. I'm committed to a smooth handover for my team, but I'd also do my best to wrap up responsibly and join as early as possible. If there's flexibility through a buyout or an early release, I'm open to exploring that."

Tip: state your real notice period and, if you can, signal willingness to expedite — companies value an earlier start.

14. Are you open to relocation?

Sample answer:

"Yes, I'm open to relocation for the right opportunity. I'm currently based in Chennai and comfortable there, but I'm flexible and would relocate to wherever the role is based. I'd just appreciate reasonable notice to plan the move. I'm also comfortable with hybrid or remote arrangements if that's how the team works."

Tip: adjust this honestly based on your actual willingness and any constraints.

15. Are you a team player or an individual contributor?

Sample answer:

"Both, depending on what the work needs. I can put my head down and deliver independently — I've owned full modules like the document portal on EasyVisa and the Schedule-Manager on Subsea end to end. But I also thrive in a team: I share reusable components, do code reviews, help unblock teammates, and I led the Angular 16 migration that touched four repositories and required coordinating across people. Good engineering is collaborative, so I'm comfortable switching between focused individual work and tight teamwork."

16. How do you stay updated with new technologies?

Sample answer:

"I follow the official docs and release notes for the frameworks I use — Angular especially, which is how I confidently led the migration to Angular 16. I read engineering blogs and follow the community, and I learn a lot hands-on by building small things to try new patterns, like RxJS operators or NgRx state patterns. I've also picked up new stacks on the job as projects demanded — React, .NET, Grails, Groovy — so I treat each project as a chance to learn. Staying current is just part of being a developer for me."

17. Why should we hire you?

Sample answer:

"Because I bring four-plus years of genuine full-stack delivery — Angular and Ionic on the front end, Java, Spring Boot and .NET on the back end — and a track record of owning features end to end across multiple domains: legal tech, marine operations, and AI-driven education. I ramp up quickly on new stacks, I write reusable and tested code, and I've already led work like an Angular migration. I'm low-maintenance, delivery-focused, and I communicate proactively. For a lateral

role you need someone who can contribute from week one while still growing into more responsibility — that's exactly what I offer."

18. Describe a time you took ownership or showed leadership.

Sample answer (STAR):

"**Situation:** Four of our repositories were on an older Angular version and accumulating tech debt. **Task:** I was given the job of migrating them to Angular 16. **Action:** I planned the upgrade repo by repo, worked through breaking changes and deprecated APIs, updated dependencies carefully, and tested each migration before moving on so the team's work was never blocked for long. **Result:** All four repos were successfully migrated to Angular 16, the codebase became easier to maintain, and we could use newer framework features. It was a project I largely drove myself, and it built my confidence in owning larger technical initiatives."

19. Tell me about a time you had to learn something new quickly.

Sample answer (STAR):

"**Situation:** When I moved to the Subsea project, the back end was .NET — and I'd been working primarily in Java and Grails. **Task:** I needed to build the Schedule-Manager module with .NET APIs and a Kendo UI grid. **Action:** I leaned on my general back-end fundamentals, went through the existing .NET codebase and docs, and built incrementally — starting with simple endpoints before the full category and subcategory management. I asked targeted questions when stuck rather than spinning. **Result:** I delivered the module on schedule, and adding ASP.NET Core to my toolkit made me a more versatile developer. It confirmed that I can pick up a new stack quickly when the work calls for it."

20. How do you prioritize when you have multiple tasks?

Sample answer:

"I look at impact and dependencies first — what's blocking others or the release goes to the top. On Subsea I was balancing bug fixes with building the Schedule-Manager module, so I cleared the blocking bugs first, then moved to feature work. I use Jira to keep everything visible and track status, and I align with my lead when priorities are unclear rather than guessing. I also time-box things so one task doesn't quietly eat the whole day. Clear priorities plus communication keeps me from getting overwhelmed."

21. How do you handle pressure or stress?

Sample answer:

"I deal with stress by getting organized — when things feel chaotic, breaking the work into a clear list immediately calms it down because I can see a path forward. I focus on what I can control, tackle the highest-priority item, and communicate early if a deadline is genuinely at risk. I've handled crunch periods, like delivering across bug fixes and a new module on Subsea in a short window, by staying methodical rather than panicking. And I make sure to step away briefly when I'm stuck — a short break often solves the bug faster than staring at it."

22. What do you know about Wipro?

Sample answer:

"Wipro is one of India's largest IT services and consulting companies, with a global presence and clients across many industries — banking, healthcare, retail, energy and more. It's well known for digital transformation, cloud, and engineering services, and for investing in employee learning and career growth. What appeals to me is the combination of scale, diversity of projects, and the structured environment to grow as an engineer. With my full-stack background in Angular, Java and cloud, I see a strong fit with the kind of work Wipro delivers."

Tip: glance at Wipro's latest focus areas and any role-specific service line before the interview so this feels current.

23. Tell me about a time you went above and beyond.

Sample answer (STAR):

Situation: On EasyVisa, several teammates kept rebuilding similar form components from scratch, which was slow and inconsistent. **Task:** My assigned work was just my own module, but I saw the repeated effort. **Action:** I took the initiative to build a set of reusable Angular Formly custom components and reusable form modules — including things like an image cropper and nested reactive form handling — and documented how to use them. **Result:** The team adopted them, which sped up development across modules and kept the UI consistent. It wasn't strictly my task, but it had a bigger payoff for the whole team."

24. How do you ensure code quality?

Sample answer:

"A few habits. I write unit tests for the components I own — I learned that lesson early on EasyVisa. I participate actively in code reviews, both giving and receiving feedback, and I build reusable components so logic isn't duplicated and bugs don't multiply. I follow the team's coding standards and keep things readable, because code is read far more than it's written. And I test edge cases, not just the happy path. Quality for me is about preventing problems early rather than firefighting later."

25. Do you have any questions for us?

Sample answer (always ask 2-3 — never say "no"):

"Yes, a few:

- What does the team I'd join look like, and what stack and projects would I be working on?
- How is success measured for this role in the first six months to a year?
- What do the growth and learning paths look like at Wipro for someone at my level?
- How does the team handle code reviews, testing, and engineering best practices?
- What are the biggest technical challenges the team is currently working on?"

Tip: pick the 2-3 most relevant to the conversation. Asking thoughtful questions signals genuine interest and seniority.

Quick Reminders for the HR Round

- **Stay positive about your current employer** — never badmouth Provility or teammates. Frame your move as growth, not escape.
- **Use real project names** — RoboGebra, Subsea, EasyVisa, the Angular 16 migration. Specifics make you credible.
- **Quantify or describe impact** where you can, even qualitatively ("reused across the team," "delivered on time," "all four repos migrated").
- **Keep answers 30-90 seconds.** Long enough to show substance, short enough to keep attention.
- **Be honest** about notice period, relocation, and salary — consistency builds trust.
- **Smile, make eye contact, and pause before answering** tough questions. Composure reads as seniority.

Java 17 — Version-Specific Features (Interview Q&A)

Berribot target stack lists Java 17. Your resume says "Java" — be ready for "what's new in Java 17 / since Java 8" questions. This file covers the headline features from Java 9 → 17, focused on what interviewers actually ask, with code.

Java 17 Basics

Q: Why is Java 17 important?

Java 17 is an **LTS (Long-Term Support)** release (Sept 2021), like Java 8, 11, and 21. LTS versions get extended support, so companies standardize on them. Java 17 bundles years of improvements (records, sealed classes, pattern matching, switch expressions, text blocks) into a stable, supported release.

Q: What is an LTS release?

A **Long-Term Support** version that receives security patches and updates for many years, making it safe for production. Non-LTS releases (e.g., 12–16, 18–20) are short-lived feature previews. Enterprises usually run **8 → 11 → 17 → 21**.

Q: How is Java 17 different from Java 8 (the big talking point)?

Java 8	Java 17
Lambdas, Streams, Optional introduced	Plus records, sealed classes, pattern matching
<code>switch</code> statements only	<code>switch</code> expressions with arrow syntax
String concatenation	Text blocks for multi-line strings
Verbose POJOs	Records remove boilerplate
<code>instanceof</code> + cast	Pattern matching for <code>instanceof</code>
Manual modularity	Module system (JPMS) since Java 9

Records (Java 16)

Q: What is a record?

An immutable data-carrier class. The compiler auto-generates the constructor, `getters` (accessor methods), `equals()`, `hashCode()`, and `toString()`. Perfect for **DTOs**.

```
public record Employee(Long id, String name, String dept) {}

// Usage
Employee e = new Employee(1L, "Priyanka", "Engineering");
e.name();           // accessor (no "get" prefix)
e.toString();       // Employee[id=1, name=Priyanka, dept=Engineering]
```

Q: What are the rules/limits of records?

- Fields are **final** (immutable); record is implicitly **final**.
- Can add static fields, methods, and a **compact constructor** for validation:

```
public record Employee(Long id, String name) {
    public Employee {           // compact constructor
        if (name == null) throw new IllegalArgumentException("name required");
    }
}
```

- Cannot extend another class (but can implement interfaces).

Q: Record vs Lombok `@Data` ?

Records are a **native language feature** (no library) and are immutable by design. Lombok is a third-party annotation processor that generates getters/setters for mutable classes. For immutable DTOs, records are cleaner; for JPA entities (which need a no-arg constructor and mutability), you still use regular classes/Lombok.

Sealed Classes (Java 17)

Q: What are sealed classes?

They restrict **which classes can extend or implement** a class/interface — giving you a controlled, known hierarchy.

```
public sealed interface Shape permits Circle, Square, Rectangle {}

public final class Circle implements Shape { }
public final class Square implements Shape { }
public non-sealed class Rectangle implements Shape { } // reopened for extension
```

Each permitted subclass must be **final**, **sealed**, or **non-sealed**. Great with pattern-matching **switch** — the compiler knows all cases.

Switch Expressions (Java 14)

Q: What are switch expressions?

switch can now **return a value** using arrow syntax — no fall-through, no **break**, more concise.

```
String size = switch (day) {
    case MONDAY, FRIDAY, SUNDAY -> "Busy";
    case TUESDAY                 -> "Light";
    default                     -> {
        yield "Normal";          // yield returns a value from a block
    }
};
```

Benefits: no accidental fall-through, exhaustive checks, returns a value directly.

Pattern Matching for instanceof (Java 16)

Q: What is pattern matching for `instanceof` ?

Combines the type check and cast into one step, removing boilerplate.

```
// Old
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}
// Java 16+
if (obj instanceof String s) {
    System.out.println(s.length());    // s is already cast
}
```

Q: What is pattern matching for switch? (preview in 17, final in 21)

Match on type directly in a switch — powerful with sealed classes:

```
String describe(Shape shape) {
    return switch (shape) {
        case Circle c    -> "Circle r=" + c.radius();
        case Square s    -> "Square side=" + s.side();
        case Rectangle r -> "Rectangle";
    };    // exhaustive – no default needed with sealed types
}
```

Text Blocks (Java 15)

Q: What are text blocks?

Multi-line string literals using `"""`, ideal for JSON, SQL, HTML — no `\n` or `+` concatenation.

```
String json = ""
{
    "name": "Priyanka",
    "role": "Full Stack Developer"
}
"";

String sql = ""
SELECT id, name FROM employees
WHERE dept = 'Engineering'
"";
```

Other Notable Features (Java 9–17)

Q: What is `var` (local variable type inference, Java 10)?

Lets the compiler infer the local variable type. Only for **local** variables with an initializer — improves readability, not dynamic typing.

```
var list = new ArrayList<String>(); // inferred ArrayList<String>
var count = 10; // int
```

Q: What is the Java Module System / JPMS (Java 9)?

A way to group packages into **modules** with explicit dependencies and exported APIs via `module-info.java`. Improves encapsulation and lets you build smaller runtimes (`jlink`). Most apps still use the classpath, but it's good to know.

Q: What collection factory methods were added (Java 9)?

Immutable collections in one line:

```
List<String> list = List.of("a", "b", "c");
Map<String, Integer> map = Map.of("x", 1, "y", 2);
Set<Integer> set = Set.of(1, 2, 3);
```

Q: What Stream/Optional improvements came after Java 8?

- `Stream.takeWhile()` / `dropWhile()` (Java 9), `Stream.ofNullable()`, `iterate` with predicate.
- `Optional.ifPresentOrElse()`, `Optional.or()`, `Optional.stream()` (Java 9).
- `Collectors.teeing()` (Java 12), `String` methods `strip()`, `isBlank()`, `lines()`, `repeat()` (Java 11/12).

Q: What are "helpful NullPointerExceptions" (Java 14)?

The JVM now tells you **exactly which variable was null** in an NPE message (e.g., `Cannot invoke "String.length()" because "name" is null`), making debugging far easier.

Q: What is the new HTTP Client (Java 11)?

A modern, built-in `java.net.http.HttpClient` supporting HTTP/2, async (`CompletableFuture`), and WebSockets — replacing the old `URLConnection` .

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder(URI.create("https://api.example.com")).build();
HttpResponse<String> res = client.send(req, HttpResponse.BodyHandlers.ofString());
```

Q: Can you run a single-file Java program now (Java 11)?

Yes — `java Hello.java` runs a source file directly without a separate `javac` step (great for scripts/learning).

Likely BerriBot Q&A

Q: What Java 17 features would you use in a Spring Boot microservice?

- **Records** for DTOs / API request-response objects and immutable value objects.
- **Text blocks** for inline JSON test data or native SQL queries.
- **Switch expressions** for clean mapping logic (status → action).
- **Sealed classes + pattern matching** for modeling a closed set of domain events.
- **var** for concise local variables in service methods.

Q: How would you migrate a Java 8 project to Java 17?

1. Update the JDK and build tool (Maven/Gradle) `source/target` to 17.
2. Check for **removed APIs** (e.g., some `javax` → `jakarta` namespace changes in newer Spring Boot 3.x, deprecated `Nashorn` JS engine, security manager).
3. Update dependencies (Spring Boot 3.x requires Java 17).
4. Run the full test suite; fix compilation warnings.
5. Gradually adopt new features (records for DTOs, etc.) — migration first, modernization second.

Honesty tip: If you've mostly used Java 8/11 features, say: *"I've worked extensively with Java 8 features — Streams, lambdas, Optional, functional interfaces. I've been upskilling on Java 17 — records, sealed classes, pattern matching, and switch expressions — and I'm comfortable using them."*

MongoDB — Interview Q&A

⚠ **Gap alert:** MongoDB is on the **Berribot stack** but **not on your current resume** (you list MySQL, Postgres, Neo4j). Expect questions. This file gives you solid fundamentals + Spring Data MongoDB so you can speak confidently. Be honest about hands-on depth, but know the concepts cold.

Fundamentals

Q: What is MongoDB?

A **NoSQL, document-oriented database**. Instead of tables and rows, it stores data as flexible **JSON-like documents** (BSON) inside **collections**. It's schema-less, horizontally scalable, and great for rapidly evolving data and large volumes.

Q: What is a document and a collection?

- **Document:** a record stored as a BSON (Binary JSON) object — key-value pairs, can be nested. The MongoDB equivalent of a "row."
- **Collection:** a group of documents — the equivalent of a "table" (but with no fixed schema).

```
// A document in the "employees" collection
{
  "_id": ObjectId("64f..."),
  "name": "Priyanka",
  "role": "Full Stack Developer",
  "skills": ["Angular", "Java", "Spring Boot"],
  "address": { "city": "Chennai", "country": "India" }
}
```

Q: What is BSON?

Binary JSON — MongoDB's storage format. It extends JSON with extra types (ObjectId, Date, binary, int/long/decimal) and is optimized for speed and space.

Q: What is `_id`?

A unique primary key automatically added to every document. By default it's a 12-byte **ObjectId** (containing a timestamp), but you can set your own value.

Q: SQL vs NoSQL (MySQL vs MongoDB) — the key interview question

	MySQL (SQL/Relational)	MongoDB (NoSQL/Document)
Data model	Tables, rows, columns	Collections, documents
Schema	Fixed, predefined	Flexible, dynamic
Relationships	Joins, foreign keys	Embedding or referencing
Scaling	Vertical (mostly)	Horizontal (sharding)
Transactions	Strong ACID	ACID (multi-doc since v4.0)
Query language	SQL	MongoDB Query Language (JSON)
Best for	Structured, relational data	Flexible, large-scale, evolving data

Q: When would you choose MongoDB over MySQL?

- **MongoDB:** flexible/changing schema, hierarchical/nested data, high write throughput, horizontal scale, content management, real-time analytics, catalogs.
- **MySQL:** strong relational integrity, complex joins, financial/transactional consistency, well-defined stable schema.
- *Many systems use both (polyglot persistence) — e.g., MySQL for orders, MongoDB for product catalog/logs.*

Q: What is ACID vs BASE?

- **ACID** (relational): Atomicity, Consistency, Isolation, Durability — strong guarantees.
- **BASE** (many NoSQL): Basically Available, Soft state, Eventual consistency — favors availability/scale over immediate consistency.
- MongoDB is ACID at the **single-document** level always, and supports **multi-document ACID transactions** since v4.0.

CRUD Operations

Q: Basic CRUD in MongoDB shell?

```
// Create
db.employees.insertOne({ name: "Priyanka", role: "Developer", exp: 4 });
db.employees.insertMany([ { name: "A" }, { name: "B" } ]);

// Read
db.employees.find({ role: "Developer" });
db.employees.findOne({ name: "Priyanka" });
db.employees.find({ exp: { $gte: 4 } });           // experience >= 4

// Update
db.employees.updateOne({ name: "Priyanka" }, { $set: { exp: 5 } });
db.employees.updateMany({ role: "Developer" }, { $inc: { exp: 1 } });

// Delete
db.employees.deleteOne({ name: "B" });
db.employees.deleteMany({ exp: { $lt: 2 } });
```


Q: Common query operators?

- Comparison: `$eq`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`, `$in`, `$nin`
- Logical: `$and`, `$or`, `$not`, `$nor`
- Element: `$exists`, `$type`
- Array: `$all`, `$elemMatch`, `$size`
- Update: `$set`, `$unset`, `$inc`, `$push`, `$pull`, `$addToSet`

Schema Design

Q: Embedding vs Referencing — how do you model relationships?

- **Embedding** (nested documents): store related data inside the parent. Best for **one-to-few**, data read together, that doesn't change independently.

```
{ "name": "Order1", "items": [ { "product": "Pen", "qty": 2 }, { "product": "Book", "qty": 1 } ] }
```

- **Referencing** (store the `_id` of another document): best for **one-to-many** / **many-to-many**, large or independently-changing data. Requires a second lookup (or `$lookup`).

```
{ "name": "Order1", "customerId": ObjectId("...") }
```

Q: What's the rule of thumb for embedding vs referencing?

- **Embed** when data is accessed together and bounded in size ("contains" relationship).
- **Reference** when data is large, shared, or grows unbounded, or needs to be queried on its own.
- Watch the **16 MB document size limit** — don't embed unbounded arrays.

Q: Does schema-less mean no design?

No — you still design your documents intentionally around **query patterns** ("model for how you read"). Schema flexibility helps evolution, but a bad data model still causes problems.

Indexing & Performance

Q: What are indexes in MongoDB?

Data structures (B-trees) that speed up queries by avoiding full collection scans. Without an index, MongoDB scans every document.

```
db.employees.createIndex({ name: 1 });           // ascending index
db.employees.createIndex({ role: 1, exp: -1 });  // compound index
```

Q: Types of indexes?

Single-field, **compound** (multiple fields), **multikey** (on array fields), **text** (full-text search), **geospatial**, **hashed** (for sharding), and **TTL** (auto-expire documents after a time).

Q: How do you analyze query performance?

Use `explain()`:

```
db.employees.find({ name: "Priyanka" }).explain("executionStats");
```

Look for `COLLSCAN` (bad — full scan) vs `IXSCAN` (good — index used).

Aggregation

Q: What is the aggregation pipeline?

A framework to process and transform documents through **stages** (like a Unix pipe). Used for grouping, filtering, computing — the equivalent of SQL `GROUP BY` + functions.

```
db.orders.aggregate([
  { $match: { status: "completed" } },           // filter (WHERE)
  { $group: { _id: "$customerId",                // group by
    total: { $sum: "$amount" } } },
  { $sort: { total: -1 } },                       // ORDER BY
  { $limit: 10 }
]);
```

Q: Common aggregation stages?

`$match`, `$group`, `$project`, `$sort`, `$limit`, `$skip`, `$unwind` (flatten arrays), `$lookup` (join with another collection), `$count`, `$addFields`.

Scaling & Reliability

Q: What is replication / a replica set?

A **replica set** is a group of MongoDB servers with the same data: one **primary** (handles writes) and multiple **secondaries** (copies). If the primary fails, an automatic **election** promotes a secondary — providing **high availability** and redundancy.

Q: What is sharding?

Horizontal scaling — splitting data across multiple servers (shards) based on a **shard key**. Each shard holds a subset of the data, allowing the database to handle huge datasets and high throughput. A `mongos` router directs queries to the right shard.

Q: Replication vs Sharding?

- **Replication:** copies of the **same** data → availability & read scaling.
- **Sharding:** **splits** data across servers → write scaling & large datasets.

Q: Do MongoDB transactions exist?

Yes — **multi-document ACID transactions** since v4.0 (on replica sets). Single-document operations are always atomic. Use transactions sparingly (they add overhead); good schema design often avoids needing them.

Spring Data MongoDB (most relevant for Berribot)

Q: How do you use MongoDB with Spring Boot?

Add `spring-boot-starter-data-mongodb`, configure the URI, and use `@Document` entities with `MongoRepository`.

```
spring.data.mongodb.uri=mongodb://localhost:27017/berribot
```

```
@Document(collection = "employees")
public class Employee {
    @Id
    private String id;
    private String name;
    private String role;
    private List<String> skills;
    // getters/setters
}
```

Q: What is MongoRepository?

A Spring Data interface giving you CRUD + derived query methods out of the box (like `JpaRepository` but for Mongo):

```
public interface EmployeeRepository extends MongoRepository<Employee, String> {
    List<Employee> findByRole(String role);
    List<Employee> findBySkillsContaining(String skill);
    List<Employee> findByExpGreaterThan(int exp);
}
```

Q: MongoRepository vs MongoTemplate?

- **MongoRepository:** high-level, derived query methods — quick and clean for standard CRUD.
- **MongoTemplate:** lower-level, fine-grained control — complex queries, aggregations, bulk ops, dynamic criteria.

```
Query query = new Query(Criteria.where("role").is("Developer").and("exp").gte(4));
List<Employee> list = mongoTemplate.find(query, Employee.class);
```

Q: How do you run aggregations in Spring Data?

Via `MongoTemplate` with the `Aggregation` builder:

```
Aggregation agg = Aggregation.newAggregation(
    Aggregation.match(Criteria.where("status").is("completed")),
    Aggregation.group("customerId").sum("amount").as("total")
);
AggregationResults<Document> results =
    mongoTemplate.aggregate(agg, "orders", Document.class);
```

Q: How would you map a MySQL-backed feature to MongoDB?

Identify the read pattern. If data is hierarchical and read together (e.g., a learning dashboard with nested progress), **embed** it in one document for a single fast read instead of multiple SQL joins. If entities are shared and large, **reference** them.

Honest Framing for the Interview

"My production database experience is with **MySQL, Postgres, and Neo4j**. I understand MongoDB's document model, indexing, the aggregation pipeline, and replica sets/sharding, and I've worked with Spring Data MongoDB concepts — `@Document` , `MongoRepository` , and `MongoTemplate` . I'm confident I can be productive with it quickly, and I already think in terms of NoSQL/graph modeling from my Neo4j work."

Tip: Your **Neo4j** experience is a genuine advantage here — it shows you already think beyond relational tables, which is exactly the mindset MongoDB needs.

BerriBot Assessment — Master Prep Guide

Target role: Senior Full Stack Developer (4.8+ years)

Stack: Java 17 · Spring Boot · Angular 16 · MongoDB · MySQL · Microservices · AWS S3

This guide maps the assessment's stack to your study files, flags gaps, and gives you a focused plan + key talking points.

1. Stack → Study-file map

Stack item	Status vs your resume	Study file(s)
Java 17	You list "Java" — brush up version features	12-java17-features.md + 05-java.md
Spring Boot	✅ On resume	06-spring-boot.md
Angular 16	✅ Strong — you migrated to Angular 16	04-angular.md + Angular 16 section below
MongoDB	⚠️ Gap — not on resume	13-mongodb.md
MySQL	✅ On resume	06-spring-boot.md (JPA)
Microservices	⚠️ Basic	08-microservices-basics.md
AWS S3	⚠️ Basic	07-aws-basics.md
Self-intro / HR	—	00 , 11
Coding round	—	09-coding-problems.md
Projects	✅	10-projects-deep-dive.md

2. Honest gap analysis (prioritize these)

- MongoDB** — biggest gap. Learn the document model, indexing, aggregation pipeline, and Spring Data MongoDB (`@Document` , `MongoRepository` , `MongoTemplate`). Your **Neo4j** experience is your bridge — you already think NoSQL.
 - Java 17 features** — you've used Java 8; learn records, sealed classes, pattern matching, switch expressions, text blocks.
 - Microservices depth** — you have the basics; be ready for service discovery, API gateway, circuit breaker, inter-service communication.
 - AWS S3 hands-on** — know buckets, the Spring Boot SDK upload/download flow, presigned URLs (example below).
- Everything else (Angular, Spring Boot, MySQL/JPA, REST) is your strength — lead with it.

3. Likely assessment format (typical for this profile)

Berribot/product-company assessments for a senior full-stack role usually run in these stages — prepare for each:

1. **Online coding test** (HackerRank/Codility style) — DSA + maybe a small Java/JS task. → [09-coding-problems.md](#)
2. **Technical Round 1 (Backend)** — Java 17, Spring Boot, REST, JPA/MongoDB, design of an API.
3. **Technical Round 2 (Frontend)** — Angular 16, RxJS, state management, component design.
4. **System/Full-stack design** — design a feature end-to-end (Angular → Spring Boot REST → MongoDB/MySQL → S3), microservices split, scaling.
5. **Managerial / project deep-dive** — your real projects, decisions, trade-offs. → [10](#)
6. **HR** — fit, expectations, notice period. → [11](#)

4. Angular 16 — version-specific talking points

Since they pin **Angular 16** (which you migrated to), be ready for:

- **Standalone components** — no NgModule needed; `standalone: true`, import dependencies directly. The future direction of Angular.
- **Angular Signals (developer preview in 16)** — a new reactive primitive for fine-grained change detection: `signal()`, `computed()`, `effect()`. Know it exists and the concept (reactivity without zone.js overhead).
- **Required inputs** — `@Input({ required: true })`.
- **Router data as input bindings** (`withComponentInputBinding`).
- `takeUntilDestroyed()` and `DestroyRef` — cleaner unsubscription.
- **Self-closing tags, esbuild-based dev server** (faster builds), Vite.
- **NgOptimizedImage** for image performance.

Talking point: "I led/contributed to migrating four repos to Angular 16. I dealt with breaking changes, RxJS upgrades, Ivy, and deprecated APIs, and I'm familiar with the new standalone-component direction and the signals preview."

5. Full-stack scenario you should be able to whiteboard

"Design a file-upload + document feature" (ties your EasyVisa document-portal experience to their stack):

```

[ Angular 16 client ]
- reactive form, file picker
- calls REST API; shows progress
  |
  v (HTTP, JWT auth)
[ Spring Boot service (Java 17) ]
- @RestController, @Valid DTOs (records)
- validates, stores metadata
- uploads file to S3 (presigned URL or SDK)
  |                               |
  v                               v
[ MongoDB ]                     [ AWS S3 ]
document metadata                actual file bytes
(filename, type, owner,          (key = userId/uuid)
s3Key, uploadedAt)

```

Key points to mention: store **metadata in MongoDB/MySQL**, **binary in S3** (never blobs in the DB); use **presigned URLs** so the client uploads/downloads directly from S3; secure with **JWT**; handle large files with multipart; add validation and a global exception handler.

S3 upload from Spring Boot (be ready to discuss)

```

@Service
public class S3Service {
    private final S3Client s3;    // AWS SDK v2

    public String upload(MultipartFile file, String key) throws IOException {
        s3.putObject(
            PutObjectRequest.builder().bucket("berribot-docs").key(key).build(),
            RequestBody.fromBytes(file.getBytes())
        );
        return key;
    }

    // Presigned URL so the client downloads directly from S3
    public String presignedDownloadUrl(String key) {
        S3Presigner presigner = S3Presigner.create();
        GetObjectPresignRequest req = GetObjectPresignRequest.builder()
            .signatureDuration(Duration.ofMinutes(15))
            .getObjectRequest(b -> b.bucket("berribot-docs").key(key))
            .build();
        return presigner.presignGetObject(req).url().toString();
    }
}

```

6. MongoDB + MySQL together (polyglot persistence)

They list **both** — expect "when do you use which?"

- **MySQL**: transactional, relational, strong consistency — users, payments, orders, structured relationships.
- **MongoDB**: flexible/nested data, high write volume, evolving schema — activity logs, content, catalogs, dashboards, embedded sub-documents.
- A single Spring Boot app can use **both** (Spring Data JPA + Spring Data MongoDB), each for the right data.

7. Focused 5-day plan for Berribot

Day	Focus
1	MongoDB (13) — model, queries, aggregation, Spring Data. This is your gap.
2	Java 17 (12) + revise Core Java (05)
3	Spring Boot + REST + JPA (06) + the S3/full-stack scenario above
4	Angular 16 (04) + the Angular 16 section above
5	Microservices (08) + coding practice (09) + self-intro & projects (00 , 10)

8. Senior-level talking points (4.8+ years framing)

At "senior," they expect **ownership and judgment**, not just coding:

- Talk about **design decisions and trade-offs** (why MongoDB vs MySQL, why a service split).
- Mention **mentoring, code reviews, and architecture** input (you "assisted in refining architecture" on Subsea).
- Show **end-to-end ownership** — API → DB → UI → deployment.
- Discuss **non-functionals**: performance, security (JWT), scalability, monitoring.
- Quantify impact where you can (retention from Study Reminders, migration of 4 repos, etc.).

9. Pre-interview checklist (Berribot edition)

- [] Can explain **MongoDB vs MySQL** and when to use each
- [] Can write a basic **aggregation pipeline** and a **MongoRepository** interface
- [] Can name **5 Java 17 features** and where you'd use them
- [] Can describe **Angular 16 standalone components & signals** at a high level
- [] Can whiteboard the **upload-to-S3 + metadata-in-DB** flow with presigned URLs
- [] Can split a feature into **microservices** and justify the boundaries
- [] Self-intro polished; one strong **STAR story** per project ready
- [] Honest about MongoDB/Java 17 depth — but concepts solid

Good luck with the Berribot assessment, Priyanka! Lead with your Angular + Spring Boot strength, bridge MongoDB through your Neo4j NoSQL mindset, and show senior-level ownership.